学校代码：10284
分 类 号：TP18
密　　级：公开
U D C：004.8
学　　号：502023330031



南京大学

硕士学位论文

论文题目　大模型推理Prefill-Decode双阶段KV Cache优化关键技术研究

作者姓名　李宇航
专业名称　计算机科学与技术
研究方向　大模型推理系统优化
导师姓名　顾荣　研究员

2026年5月25日

答辩委员会主席 程龚 教授

评　阅　人 程龚 教授

徐雷 教授

论文答辩日期　2026 年 5 月 14 日

研究生签名：李宇航

导师签名：欣荣

Research on Key KV Cache Optimization Techniques for Prefill-Decode Dual-Stage Large Language Model Inference

by

Yuhang Li

Supervised by

Rong Gu, Researcher

A dissertation submitted to

the graduate school of Nanjing University

in partial fulfilment of the requirements for the degree of

Master

in

Computer Science and Technology



School of Computer Science

Nanjing University

May 25, 2026

南京大学学位论文原创性声明

本人郑重声明，所提交的学位论文是本人在导师指导下独立进行科学研究工作所取得的成果。除本论文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得南京大学或其他教育机构的学位证书而使用过的材料。对本文的研究做出重要贡献的个人和集体，均已在论文的致谢部分明确标明。本人郑重申明愿承担本声明的法律责任。

研究生签名：李宇航

日期：2026年5月25日

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：大模型推理Prefill-Decode双阶段 KV Cache 优化关键技术研究

计算机科学与技术 专业 2023 级硕士生姓名：李宇航

指导教师（姓名、职称）：顾荣 研究员

摘 要

大模型凭借出色的语言理解、内容生成与复杂推理能力，正在深刻影响智能问答、代码生成和内容创作等多个领域。作为模型能力落地的关键环节，大模型推理系统直接决定模型服务的响应时延、吞吐能力与部署成本。在推理过程中，KV Cache 通过缓存历史 token 的键值状态来避免重复注意力计算，是支撑长上下文和高并发服务的关键运行时机制。然而，KV Cache 会随着输入长度、输出长度和并发请求数增长而迅速膨胀，不仅大量占用 GPU 显存，还会进一步影响前缀共享命中、批次组织与系统吞吐，成为制约推理效率与部署成本的核心瓶颈。围绕这一背景，本文面向 Prefill 与 Decode 双阶段的资源开销和执行特性，研究大模型推理系统中的 KV Cache 协同优化方法。

本文从三个层面展开研究。在体系结构层面，研究层次化 KV Cache 存储，以提升异构内存环境下的缓存承载能力；在数据结构层面，研究前缀 KV Cache 管理，以提高共享前缀识别、驻留与复用效率；在模型结构层面，研究响应长度预测与 KV Cache 负载感知请求路由，以缓解 Decode 阶段因输出长度长尾分布带来的多 GPU 负载失衡与资源浪费问题。围绕上述三个层面，本文进一步形成了 Prefill 阶段的显存–主存卸载与准入、Prefill 阶段的主存–显存预取与执行优化，以及 Decode 阶段的长度预测与负载感知路由三项核心工作，具体如下。

（1）针对 Prefill 阶段共享前缀 KV Cache 在 GPU 显存受限条件下容易被错误驱逐、简单卸载又会引入大量 GPU–CPU 传输开销的问题，本文提出面向 KV Cache 显存–主存卸载与准入的热度感知管理方法，通过轻量级热度追踪、热度感知驱逐和选择性主存准入，优先保留未来复用价值更高的前缀缓存。实验效果表明，该方法能够在较小命中率代价下显著降低异构内存传输时间，并改善端到端执行时延；例如在 MMLU 负载上，选择性准入将总时延从 1966.31 s 降至

556.07 s，提升约 3.54 倍，将卸载时间从 1357.76 s 降至 25.66 s，下降约 52.9 倍。

（2）针对主机内存中高价值 KV Cache 仅被动保留、难以及时转化为后续 Prefill 命中收益的问题，本文提出面向 KV Cache 主存–显存预取与执行优化的主动回迁方法，通过主存热点与显存冷点的匹配置换、Decode 阶段流水线重叠以及批量传输，降低缓存回迁对端到端执行路径的干扰。实验效果表明，该方法能够将前缀命中率优势稳定转化为时延下降和吞吐提升；在 C-EVAL 负载上，相较启用前缀共享的 vLLM，最高实现约 2.25 倍的时延优化与吞吐提升，相较 SGLang-LRU 最高实现约 2.00 倍的时延下降与吞吐提升。

（3）针对 Decode 阶段输出长度长尾分布导致多 GPU 间未来 KV Cache 增长量和解码工作量失衡的问题，本文提出响应长度预测与 KV Cache 负载感知多 GPU 路由调度方法，通过层–头级别预测能力扫描识别与响应长度强相关的 HotHead 表征，并结合不同生成场景的提示词位置与全局表征构造轻量长度预测器，将预测长度转化为未来 KV Cache 负载估计用于请求路由。实验效果表明，该方法在 Llama3.2-1B 和 Qwen2.5-3B 的 LongSeq、Reasoning 与 RL 三类场景中均取得最低平均绝对误差，最高相对最佳基线降低约 20.0%；在生成长度高倾斜的多 GPU 路由负载上，相较 SGLang 默认轮转路由，本文方法将整批完成时延降低约 33.3%，将 P99 尾部延迟降低约 61.9%，并将输出吞吐提升约 53.8%。

关键词：大语言模型推理；KV Cache；前缀共享；异构内存；响应长度预测；请求调度

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on Key KV Cache Optimization Techniques for Prefill-Decode Dual-Stage Large Language Model Inference

SPECIALIZATION: Computer Science and Technology

POSTGRADUATE: Yuhang Li

MENTOR: Rong Gu, Researcher

ABSTRACT

Large language models have profoundly influenced intelligent question answering, code generation, and content creation due to their strong capabilities in language understanding, content generation, and complex reasoning. As a critical stage for turning model capability into practical service, large language model inference systems directly determine response latency, throughput, and deployment cost. In the inference process, the KV cache stores historical key-value states to avoid redundant attention computation, and is therefore a key runtime mechanism for long-context and high-concurrency serving. However, as input length, output length, and request concurrency increase, the KV cache expands rapidly, consumes substantial GPU memory, and further affects prefix reuse, batch organization, and system throughput, making it a core bottleneck for inference efficiency and deployment cost. Motivated by these challenges, this thesis studies coordinated KV cache optimization for the Prefill and Decode stages.

The study is organized along three complementary dimensions. From the architecture perspective, it investigates hierarchical KV cache storage to expand effective cache capacity under heterogeneous memory settings. From the data-structure perspective, it studies prefix KV cache management to improve shared-prefix identification, retention, and reuse. From the model-structure perspective, it explores response length prediction and KV-cache-load-aware request routing to mitigate multi-GPU load imbalance and resource waste caused by long-tailed output lengths during decoding. Based on these dimensions, this thesis further develops three core techniques: GPU-to-host offloading and admission for the Prefill stage, host-to-GPU prefetching and execution optimiza-

tion for the Prefill stage, and response-length prediction with load-aware routing for the Decode stage, as described below.

(1) Targeting the problem that shared-prefix KV caches are easily evicted under limited GPU memory while naive offloading introduces substantial GPU-to-CPU transfer overhead, this thesis proposes a hotness-aware management method for GPU-to-host KV cache offloading and admission. The method uses lightweight hotness tracking, hotness-aware eviction, and selective host-memory admission to prioritize prefix caches with higher future reuse value. Experimental results show that the method significantly reduces heterogeneous-memory transfer time and improves end-to-end latency with only a small cache-hit-rate tradeoff. For example, on the MMLU workload, selective admission reduces total latency from 1966.31 s to 556.07 s, achieving a 3.54× improvement, and reduces offloading time from 1357.76 s to 25.66 s, a 52.9× reduction.

(2) Targeting the problem that high-value KV caches retained in host memory are only passively stored and cannot be promptly converted into subsequent Prefill cache-hit benefits, this thesis proposes an active host-to-GPU prefetching and execution optimization method. The method matches hot host-side nodes with cold GPU-side nodes, overlaps migration with the Decode stage through pipelining, and uses batched transfer to reduce the interference of cache migration on the end-to-end execution path. Experimental results show that the method stably converts prefix-cache hit-rate gains into latency reduction and throughput improvement. On the C-EVAL workload, compared with prefix-sharing-enabled vLLM, it achieves up to about 2.25× latency improvement and throughput improvement; compared with SGLang-LRU, it achieves up to about 2.00× latency reduction and throughput improvement.

(3) Targeting the problem that long-tailed output lengths in the Decode stage cause imbalance in future KV cache growth and decoding workload across multiple GPUs, this thesis proposes a response-length prediction and KV-cache-load-aware multi-GPU routing method. The method scans layer-head prediction capability to identify HotHead representations strongly correlated with response length, combines them with scenario-specific prompt-position and global representations to build a lightweight length predictor, and converts predicted length into future KV cache load for request routing. Ex-

perimental results show that the method achieves the lowest mean absolute error across LongSeq, Reasoning, and RL scenarios on both Llama3.2-1B and Qwen2.5-3B, reducing error by up to about 20.0% over the best baseline. Under a highly skewed generation-length workload for multi-GPU routing, compared with the default round-robin routing in SGLang, the proposed method reduces batch completion latency by about 33.3%, reduces P99 tail latency by about 61.9%, and improves output throughput by about 53.8%.

KEYWORDS: Large language model inference; KV cache; Prefix sharing; Heterogeneous memory; Response length prediction; Request scheduling

目　录

第一章 绪论 1

1.1 研究背景 1

1.2 相关工作及不足 3

1.2.1 大模型推理系统与 KV Cache 管理 3

1.2.2 前缀共享与异构内存协同优化 5

1.2.3 响应长度预测与长度感知调度 6

1.3 挑战点分析 7

1.4 本文主要研究内容 8

1.5 本文组织结构 9

第二章 背景知识与问题定义 11

2.1 大模型推理过程与 KV Cache 机制 11

2.1.1 Transformer 推理流程与显存开销 11

2.1.2 Prefill-Decode 双阶段执行特性 13

2.1.3 前缀共享机制与前缀树组织方式 15

2.2 大模型推理系统中的异构内存管理 16

2.2.1 GPU 显存与主机内存协同 16

2.2.2 KV Cache 卸载、准入与预取基础 17

2.2.3 请求批处理与调度机制 18

2.3 面向 Decode 阶段的长度感知推理优化 19

2.3.1 响应长度分布特征 19

2.3.2 响应长度预测方法基础 20

2.3.3 长度感知路由问题定义 21

2.4 本章小结 22

第三章 面向推理 Prefill 阶段的 KV Cache 显存-主存卸载与准入策略 24
3.1 问题背景与分析 24
3.2 基于轻量级过滤器的 KV Cache 热度动态追踪 27
3.2.1 基于布谷鸟过滤器的 KV Cache 热度记录结构 27
3.2.2 面向动态访问模式的 KV Cache 热度实时更新 29
3.3 基于热度的 KV Cache 精准卸载与高效准入 31
3.3.1 基于热度的 GPU 显存 KV Cache 卸载机制 31
3.3.2 基于热度的主机内存 KV Cache 准入方法 32
3.4 实验评估与分析 34
3.4.1 实验环境与实现细节 34
3.4.2 数据集、对比方法与评价指标 35
3.4.3 热度追踪与卸载策略有效性验证 36
3.4.4 准入策略对推理性能的影响分析 38
3.5 本章小结 39

第四章 面向推理 Prefill 阶段的 KV Cache 主存-显存预取与执行优化策略 40
4.1 问题背景与分析 40
4.2 基于异构内存热度对比的 KV Cache 预取决策 42
4.2.1 异构内存间冷热 KV Cache 的识别与匹配方法 42
4.2.2 基于热度的 KV Cache 预取机制 44
4.3 基于流水线与批量传输的 KV Cache 预取执行优化 44
4.3.1 基于 Prefill-Decode 特性的流水线执行方法 45
4.3.2 面向传输带宽优化的批量数据传输方法 46
4.4 实验评估与分析 47
4.4.1 实验环境与评价指标 47
4.4.2 预取决策策略有效性验证 48
4.4.3 端到端性能收益分析 49
4.4.4 消融实验 53

4.5 本章小结 54

第五章 面向推理 Decode 阶段的响应长度与 KV Cache 负载感知的多 GPU 请求调度 56

5.1 问题背景与分析 56

5.2 基于层-头表征增强的响应长度预测方法 57

5.2.1 基于层-头预测能力扫描的关键表征选择 58

5.2.2 基于场景自适应特征融合的长度预测模型 60

5.3 基于 KV Cache 负载感知的多 GPU 路由调度方法 62

5.3.1 基于预测响应长度的 KV Cache 负载估计 62

5.3.2 基于预测负载均衡的多 GPU 请求路由 64

5.4 实验评估与分析 65

5.4.1 实验环境、数据集、对比方法与评价指标 65

5.4.2 长度预测精度与现有方法的比较 66

5.4.3 新调度策略与 SGLang 原始调度策略的比较 68

5.5 本章小结 70

第六章 系统架构与关键模块实现 71

6.1 系统设计目标与总体架构 71

6.1.1 统一状态对象与模块接口 72

6.2 关键数据结构与状态转换 73

6.2.1 前缀树节点状态转换 73

6.2.2 请求控制块状态转换 74

6.3 面向 Prefill 阶段的异构内存协同实现 74

6.3.1 前缀树与热度记录模块实现 75

6.3.2 卸载、准入与预取执行链路实现 75

6.4 面向 Decode 阶段的长度预测与多 GPU 路由实现 76

6.4.1 长度预测服务实现 77

6.4.2 多 GPU 请求路由调度器实现 78

6.5 Prefill-Decode 双阶段联合执行流程 78

6.6 本章小结 80

第七章 总结与展望 81
7.1 本文工作总结 81
7.2 本文工作的局限性 83
7.3 未来工作展望 84

参考文献 85

致　　谢 94

附录 A 附录 96
A.1 研究生期间发表论文 96
A.2 研究生期间参与科研项目 96
A.3 研究生期间获得的比赛奖项 97

第一章　绪论

本章主要介绍研究背景、相关工作与不足、关键挑战、主要研究内容以及全文组织结构，为后续章节展开具体方法与实验分析奠定基础。

1.1　研究背景

近年来，大语言模型在智能问答、代码生成、检索增强生成、智能体协同以及复杂推理等场景中得到广泛应用。随着模型能力、上下文长度和任务复杂度的持续提升，推理系统的服务性能逐渐成为制约大模型落地效果的关键因素。国际数据公司（International Data Corporation，IDC）报告显示，2024 年下半年中国生成式 AI 基础设施市场同比增长 165%，达到 87.4 亿元，并首次在整体 AI IaaS 市场中占比超过七成；同时，IDC 预计 AI 算力将进入“训推”拐点，需求将从训练驱动逐步转向推理驱动[1]。这意味着，推理系统正在成为大模型能力转化为真实应用价值的“最后一公里”。与训练阶段相比，推理阶段更直接面向真实用户请求，其时延、吞吐、资源利用率以及多租户稳定性会直接影响系统可用性和部署成本。因此，围绕大语言模型推理系统开展系统级优化研究，已成为推动模型能力高效落地的重要方向。

在大语言模型生成式推理过程中，系统通常需要将输入请求划分为 Prefill 和 Decode 两个阶段。Prefill 阶段面向整段输入提示词并行计算各层注意力中的 Key 和 Value 张量，具有较高的并行度，但会一次性产生规模可观的中间状态；Decode 阶段则按照自回归方式逐 token 生成输出，每一步都依赖历史上下文，具有明显的串行性和尾部拖长效应。为了避免在每一步重复计算历史 token 的注意力键值，推理系统一般会为每个请求维护 KV Cache，从而显著降低重复计算开销。以 Kwon 等人在 PagedAttention 工作中报告的 LLaMA-13B 为例，单条序列的 KV Cache 占用最高可达 1.7 GB；若同时服务 8 条同规模序列，仅 KV Cache 即约需 13.6 GB 显存[2]。KV Cache 的大小会随着模型层数、隐藏维度、上下文

长度和并发请求数持续增长，其显存占用已成为制约推理系统扩展能力的核心瓶颈之一[3-5]。

现有研究已经围绕大语言模型推理的计算加速、内存管理和服务调度开展了大量工作。例如，FlashAttention 和 FlashAttention-2 通过 IO 感知的注意力计算与更优的并行划分显著提升了单次注意力执行效率[3-4]；DeepSpeed Inference、Orca 与 FastServe 则分别从多 GPU 推理、迭代级调度以及分布式服务优化角度降低了大模型在线推理的时延与作业完成时间[5-7]；vLLM 通过 PagedAttention 改善 KV Cache 的分页管理和显存碎片问题，vAttention 进一步尝试在不修改注意力算子的前提下利用虚拟内存机制实现动态内存管理[2,8]；SARATHI、DistServe、Splitwise、Llumnix、ServerlessLLM、NanoFlow 与 SpecInfer 等工作则分别从 chunked prefill、Prefill/Decode 解耦、阶段拆分、动态调度、多层级加载、设备内并行与投机服务等角度提升系统整体吞吐或端到端时延[9-15]。这些工作说明，大语言模型推理优化已经从单纯的算子级加速，逐步走向面向系统全链路的资源协同优化。

与此同时，提示工程技术、长上下文建模和知识增强范式的广泛使用，使得推理请求的输入长度持续增长，并显著增强了不同请求之间的前缀共享特征。Llama 2、Llama 3、Gemma、Qwen2.5 等开源模型的持续演进，以及 MMLU、GSM8K、BBH、LongBench、UltraFeedback 等评测或训练场景的广泛采用，使在线服务系统不仅要处理更长的输入上下文，也要承受更复杂、更不稳定的输出过程[16-24]。一方面，系统提示词、模板化提示、少样本示例以及检索增强生成所引入的外部文档，都会使 Prefill 阶段的计算量和 KV Cache 占用显著增加；另一方面，大量请求往往共享相同或高度相似的前缀，这为前缀复用提供了重要机会。Prompt Cache、ChunkAttention、SGLang、Parrot 以及 MemServe 等研究表明，通过重用共享前缀对应的 KV Cache、暴露跨请求语义关联或构建全局提示树，可以有效减少重复 Prefill 计算并提升推理效率[25-29]。然而，这类共享前缀缓存本身又会大量占据显存空间，一旦简单地全部常驻于 GPU 显存，就会迅速触及显存容量上限；若简单地迁移至主机内存、远端内存池或网络缓存，则又会带来显著的数据传输开销和缓存失配问题[29-31]。

除了 Prefill 阶段的前缀共享问题外，Decode 阶段还面临另一类重要挑战，即不同请求的响应长度通常呈现显著的长尾分布和高波动性。对于开放式生成、思维链推理以及在线强化学习采样等场景，即便输入提示词相同，不同请求的输出

长度也可能差异很大。响应长度的不确定性会直接影响 KV Cache 占用、批处理构成以及请求调度收益，使得系统难以提前做出合理的资源规划。现有大模型服务框架已经开始从公平调度、token 成本建模、阶段解耦与连续批处理等角度间接利用长度信息[6,10,12,32-33]；从经典排队与作业调度理论看，若任务规模已知或可较好估计，SRPT/SJF 一类策略在平均响应时间与上具有明显优势[34-37]。但对 LLM 而言，输出长度同时受到提示词语义、采样策略、推理链复杂度与模型内部状态的共同影响，远比传统 Web 请求或批处理作业更难估计。近期长度预测研究开始尝试复用中间层表征或熵特征估计生成长度，并进一步用于推理调度[38]，但距离形成稳定可控的系统级优化闭环仍有明显差距。

综上所述，大语言模型推理系统正同时面临两个相互关联的资源管理难题：其一，Prefill 阶段需要处理共享前缀缓存的保留、迁移与复用问题；其二，Decode 阶段需要根据请求长度差异实施更精细的资源分配和调度控制。由于两阶段在执行模式、资源瓶颈和优化目标上存在明显差异，因此需要针对大语言模型推理系统开展 Prefill-Decode 双阶段的 KV Cache 协同优化研究。

1.2 相关工作及不足

为更清晰地说明本文工作的研究定位，本节分别从大模型推理系统与 KV Cache 管理、前缀共享与异构内存协同优化、响应长度预测与长度感知调度三个方面梳理相关工作，并据此总结现有研究的不足。

1.2.1 大模型推理系统与 KV Cache 管理

面向大语言模型推理的系统优化研究，首先聚焦于注意力计算与 KV Cache 管理效率。FlashAttention 通过 IO 感知的分块设计降低注意力计算的高带宽显存访问次数，FlashAttention-2 则进一步改进并行策略，在更高批次大小和更长上下文下提升注意力算子的实际吞吐[3-4]。这类工作解决了单次注意力计算的算子瓶颈，但其优化对象主要是算子访存模式与 GPU 并行效率，并未直接回答在线服务场景中 KV Cache 如何跨请求、跨阶段和跨存储层进行管理的问题。

在系统运行时与内存管理层面，DeepSpeed Inference 通过算子融合、并行切分与 CPU/GPU/NVMe 协同扩大了可服务模型规模[5]；Orca 首先系统化提出迭代

级调度和持续调度，解决了队首阻塞问题[6]；FastServe 则利用 token 级可抢占性降低作业完成时间，并从 GPU 内存管理角度提高多请求混部效率[7]。vLLM 通过 PagedAttention 将 KV Cache 组织为逻辑连续、物理离散的分页块，显著缓解了显存碎片与过度预留问题[2]；vAttention 则尝试借助系统层虚拟内存机制，实现与连续 KV 布局兼容的动态分配方式[8]；Punica 进一步面向多租户 LoRA 服务，研究共享底座模型下适配器状态与批处理执行的高效复用[39]。这些工作显著提升了单机或集群服务能力，但它们更偏重于如何让更多请求同时跑起来，对共享前缀缓存本身的长期价值评估与生命周期管理讨论仍较少。

随着服务负载规模进一步增大，研究者开始更加明确地区分 Prefill 与 Decode 两阶段在资源需求上的差异。SARATHI 通过 chunked prefill 和 decode/prefill 协同执行提高高并发场景中的资源利用率[9]；Sarathi-Serve 进一步聚焦吞吐–时延折中问题，在保持较低首字延迟的同时提高系统吞吐[40]；DistServe 将 Prefill 与 Decode 解耦到不同资源池中分别调度，以提升面向服务质量的输出[10]；Splitwise 则从硬件特性出发，将计算密集的 Prefill 与内存密集的 Decode 映射到不同机器或不同档次 GPU 上[11]；Llumnix 关注多实例场景下的动态重排与跨实例调度[12]；ServerlessLLM 重点优化模型启动、检查点加载与实时迁移，从而降低 serverless LLM 推理的冷启动与扩缩容开销[13]；NanoFlow 与 Flover 分别从设备内并行和时间融合角度进一步压榨生成式推理吞吐[14,41]；SpecInfer 则通过投机推理与验证降低生成延迟[15]。此外，Fairness in Serving Large Language Models 与 FastSwitch 表明，在多租户场景下，仅追求吞吐最大化并不足够，公平性与上下文切换开销同样会反馈到 KV Cache 管理和调度设计之中[32-33]。尽管上述工作已经从阶段拆分、资源映射、服务公平与运行时调度等角度显著推进了大模型推理系统研究，但其关注重点仍主要是阶段级执行组织与系统吞吐优化，对于 Prefill 与 Decode 双阶段中 KV Cache 的跨阶段生命周期管理、价值演化规律以及围绕该核心状态展开的统一优化机制讨论仍不充分。

总体来看，现有大模型推理系统与 KV Cache 管理工作已经在算子加速、显存分页、连续批处理、推测执行、多租户公平和阶段解耦等方面取得了显著进展，但多数方法仍将 KV Cache 视为请求执行时的伴生状态而非可跨请求长期经营的关键资源。在前缀共享与异构内存协同并存的场景下，系统仍缺乏围绕共享前缀缓存构建的细粒度价值评估、分层驻留和主动迁移机制，因而难以直接

回答哪些缓存应常驻显存、哪些适合进入主机内存、哪些又值得提前回迁预取等关键问题。

1.2.2 前缀共享与异构内存协同优化

在前缀共享场景，研究工作主要围绕如何识别共享前缀并复用其 KV Cache 展开。Prompt Cache 从模块化注意力复用的视角出发，尝试在不同请求之间重用可复用的注意力状态，以降低重复推理开销[25]；ChunkAttention 通过前缀感知 KV Cache 与两阶段划分机制，提升共享前缀场景下的注意力执行效率[26]；SGLang 进一步把结构化程序执行、RadixAttention 与前缀复用结合起来，使具有共同前缀的请求能够在系统层被统一调度和复用[27]；Parrot 与 MemServe 则从应用级语义关联和全局提示树角度进一步提升跨请求定位共享上下文的能力[28-29]。这些工作说明，共享前缀缓存能够显著减少 Prefill 阶段的重复计算，是长提示词推理场景中的重要优化方向。

然而，前缀共享带来的收益并非没有代价。随着共享前缀数量增加，前缀树或相关索引结构中的 KV Cache 数量也会迅速增长，GPU 显存很难容纳所有可复用前缀。为扩展缓存容量，研究者开始探索异构内存与远端缓存协同。FlexGen 早期展示了在单 GPU 场景下利用 CPU 与磁盘分层放置权重、激活和缓存状态的可行性[42]；InfiniGen 进一步关注动态 KV Cache 管理[43]；ServerlessLLM 针对启动阶段的检查点加载与迁移优化了多层级存储利用[13]；Mooncake、MemServe 与 CacheGen 则分别从 KVCache-centric 解耦架构、弹性内存池、网络压缩与流式传输角度，增强了远端或分布式缓存的可用性[29-31]。这些工作证明，主机内存、SSD 乃至网络缓存都可以成为 LLM 服务系统的有效容量扩展层，但数据迁移、命中时机与准入决策会直接影响收益实现。

与前缀共享相关的另一类工作，则从 KV Cache 压缩、稀疏选择和量化角度降低显存占用。H2O 与 Scissorhands 通过关键注意力或重要性持久性假设，在固定预算下保留更关键的历史 token[44-45]；StreamingLLM 利用 attention sink 保留少量初始 token，使窗口化缓存仍能支持极长流式上下文[46]；SnapKV、PyramidKV、MiniCache、KIVI 与 Ada-KV 分别从 token 选择、层间预算分配、深度维压缩、低比特量化与注意力头自适应预算角度压缩 KV Cache[47-51]；Quest、DuoAttention、RazorAttention 与 ShadowKV 则更进一步，从面向查询的稀疏选择、检索头与流

式头分离、检索头压缩，以及低秩阴影缓存重构等角度减少长上下文解码时必须访问的历史状态[52-55]。这些方法在降低单请求或长上下文场景的 KV Cache 成本方面效果显著，但其大多面向单请求内部缓存压缩或长序列稀疏注意力，尚未直接解决多请求条件下共享前缀缓存的冷热演化、跨层级迁移与复用时机问题。

因此，从整个领域看，现有相关工作仍存在三方面不足。其一，对热度动态变化的建模与维护仍然困难，尤其在请求模式快速变化时，如何以较低开销进行准确追踪仍是难点。其二，前缀复用、异构内存扩容和缓存压缩常被分别讨论，缺乏统一的价值度量去协调保留、卸载、准入、预取、压缩之间的联动关系。其三，多数工作更强调容量扩展或瞬时带宽优化，而对 Prefill 阶段围绕共享前缀缓存建立完整闭环优化框架的研究仍然不足。上述不足也正是本文第三章和第四章重点关注的问题。

1.2.3 响应长度预测与长度感知调度

相较于 Prefill 阶段的共享前缀问题，Decode 阶段的关键矛盾在于请求响应长度的不确定性。由于生成任务通常具有长尾输出分布，不同请求即使在相同模型和相似输入下，也可能产生显著不同的输出长度。这一现象会导致批处理内的请求完成时间不一致，进而引发 GPU 空转、KV Cache 占用失衡以及尾部请求拖慢等问题。特别是在数学推理、复杂问答、长文摘要与偏好数据生成等任务中，输出长度的波动往往远大于传统在线推理任务[21-24]。

在系统研究层面，Orca、DistServe、Llumnix、Fairness in Serving Large Language Models 以及 FastSwitch 等工作已经认识到输入长度、输出长度与 token 成本不均衡会显著影响公平性、吞吐和尾时延，因此分别从持续批次调度、阶段解耦、动态迁移、公平性定义以及上下文切换优化等角度缓解长度差异带来的性能问题[6,10,12,32-33]。但这类工作大多使用运行时观测、排队规则或 token 成本近似来间接处理长度波动，真正将生成前可获得的长度先验系统化纳入调度决策的工作仍然较少。

从经典排队论和作业调度理论看，若任务大小已知或可较好估计，SRPT/SJF 类策略在平均响应时间上具有明确优势[34-37]。这一认识为长度感知调度提供了重要理论基础：如果系统能在 Decode 开始前较准确地估计未来生成长度，就有机会把任务规模转化为 KV Cache 负载估计，并进一步用于多 GPU 请求路由和

负载均衡。更早期的文本生成研究已经在摘要、机器翻译和可控生成任务中探索长度控制与长度预测问题，例如通过显式长度嵌入、长度标记、位置编码调整和辅助长度预测头，引导模型生成满足预算约束的目标序列[56-62]。这些工作虽然主要服务于摘要、翻译或可控生成，但它们共同说明：输出长度可以被视为生成前可利用的重要先验变量。

进入大语言模型阶段后，长度相关研究开始进一步关注模型内部是否隐式编码了输出长度信息，以及这些信息能否被直接抽取用于系统调度。近期工作一方面尝试利用中间层表示、熵特征或冻结隐藏状态预测输出长度，并进一步用于请求排序、长度感知调度和系统吞吐优化[38,63]；另一方面，也有研究从表征分析角度考察长度信息在 LLM 内部不同层和不同头上的分布，发现某些中间层单元对长度具有更强的可分性与可控性[64]。这一方向说明，长度信息不仅是任务语义属性，也是推理系统资源管理的重要先验。

然而，现有相关工作仍存在明显局限。第一，传统长度控制研究更多关注如何生成满足预算的文本，而非如何在系统运行前准确预测自然生成长度，因此难以直接迁移到在线 LLM 服务场景[56-58,62]。第二，机器翻译和摘要中的长度预测通常建立在任务输出较规整、长度方差较小的前提上，而开放式 LLM 生成任务往往呈现更强的长尾性与多模态性[59-61]。第三，现有系统论文更多关注公平性、吞吐或阶段划分，对预测结果如何进一步作用于多 GPU 间的 KV Cache 负载均衡讨论仍不充分；第四，长度预测与路由策略之间往往缺乏联合设计，导致即使预测结果可用，也难稳定转化为端到端性能收益。因此，从长度预测走向 KV Cache 负载感知多 GPU 路由，是 Decode 阶段优化中值得进一步深入的问题。本文第五章即在这一方向上，尝试构建响应长度预测–KV Cache 负载估计–多 GPU 请求路由的一体化优化路径。

1.3 挑战点分析

综合上述研究现状，本文面向大语言模型推理系统中的 KV Cache 优化问题，主要面临以下三方面挑战。

（1）共享前缀缓存热度动态变化，难以实现低开销且准确的价值评估。在前缀共享场景下，不同前缀对应的访问频率会随着用户请求模式、模板使用频率

以及检索内容变化而快速波动。若采用精确统计方法，往往会引入较高的维护开销；若采用简单启发式方法，又可能难以准确反映缓存的真实复用价值。如何在保证低开销的同时，对共享前缀缓存进行实时、准确的热度追踪，是 Prefill 阶段优化首先需要解决的问题。

（2）异构内存间的数据迁移代价高，缓存卸载与预取的收益难以稳定实现。当 GPU 显存不足时，将 KV Cache 卸载至主机内存是扩展缓存容量的直接方法；但若卸载和回迁策略不合理，数据传输代价可能超过节省的重复计算代价，反而使系统性能下降。此外，显存中的缓存保留策略、主机内存中的准入策略以及回迁时的预取策略相互耦合，一个环节设计不当都会削弱整体收益。因此，如何围绕卸载、准入、预取和执行重叠建立统一的异构内存优化机制，是第二个关键挑战。

（3）Decode 阶段响应长度波动显著，长度预测与多 GPU 路由难以形成有效闭环。Decode 阶段的请求长度差异会直接影响每个请求的 KV Cache 增长量、占用时长以及整体吞吐。在缺乏长度先验的情况下，系统通常只能采用轮转或按请求数均衡的路由策略，容易在生成长度高倾斜负载下把多个长请求集中到同一 GPU，造成 KV Cache 负载失衡和尾部延迟放大。即便能够获得长度预测结果，若预测特征不稳定，或者预测结果无法进一步映射为可执行的多 GPU 负载估计与路由决策，也难以转化为稳定的系统收益。因此，如何在长度预测基础上设计面向 Decode 阶段的 KV Cache 负载感知路由闭环，是本文需要解决的第三个核心挑战。

1.4 本文主要研究内容

围绕上述挑战，本文面向大模型推理 Prefill-Decode 双阶段的 KV Cache 管理问题，开展以下三方面研究。具体而言，本文从体系结构维度研究层次化 KV Cache 存储，从数据结构维度研究前缀 KV Cache 管理，从模型结构维度研究响应长度预测与 KV Cache 负载感知多 GPU 请求路由。

（1）研究面向 Prefill 阶段的 KV Cache 显存–主存卸载与准入策略。针对共享前缀缓存数量多、显存容量有限以及缓存价值难以动态刻画的问题，本文设计轻量级热度追踪机制，对前缀缓存的访问热度进行动态维护，并在此基础上构

建基于热度的卸载与准入策略，使高价值缓存优先保留于显存或进入外部存储，从而降低显存压力并提高前缀缓存的有效复用率。

（2）研究面向 Prefill 阶段的 KV Cache 主存–显存预取与执行优化策略。针对简单回迁策略带来的高传输开销和低命中效率问题，本文进一步研究主机内存到显存的预取决策与执行优化机制。通过比较异构内存中缓存热度、构建面向前缀共享的预取选择策略，并结合流水线执行与批量传输优化，提升缓存回迁的时效性和带宽利用效率，降低异构内存协同过程中的额外代价。

（3）研究面向 Decode 阶段的响应长度预测与 KV Cache 负载感知多 GPU 路由方法。针对响应长度长尾分布导致的多 GPU 负载失衡与尾部延迟放大问题，本文结合长度预测模型和 Decode 阶段资源感知路由，构建长度预测驱动的请求分配方法。该方法不再直接沿用单层末端词元表示，而是先对不同模型执行逐层逐头扫描，筛出与响应长度更相关的中间层与关键注意力头，再利用这些层–头单元的输出信息提取更稳定的提示特征，并结合不同场景的词元位置与全局表征获得更可靠的长度估计；在此基础上，再将预测长度转化为未来 KV Cache 增长量和多 GPU 负载估计，使新请求优先路由到预测负载更低的 GPU，从而缓解生成长度高倾斜负载下的尾部拖慢问题。

总体而言，本文尝试围绕大模型推理双阶段的 KV Cache 协同优化构建一条完整技术主线：在 Prefill 阶段，重点解决共享前缀缓存的保留、迁移和预取问题（本模块工作已在系统软件会议 MiddleWare 2024 与数据库系统会议 SIGMOD 2026 发表）；在 Decode 阶段，重点解决响应长度不确定条件下的长度预测、KV Cache 负载估计与多 GPU 请求路由问题。通过上述研究，本文力求在异构内存环境下提升大模型推理系统的整体执行效率和服务能力。

1.5 本文组织结构

本文各章节安排如下。图 1-1 展示了本文整体研究框架及各章节之间的衔接关系。

第一章为绪论，介绍本文的研究背景、相关工作与不足、关键挑战、主要研究内容以及全文组织结构。

第二章为背景知识与问题定义，介绍大模型推理过程、KV Cache 机制、前

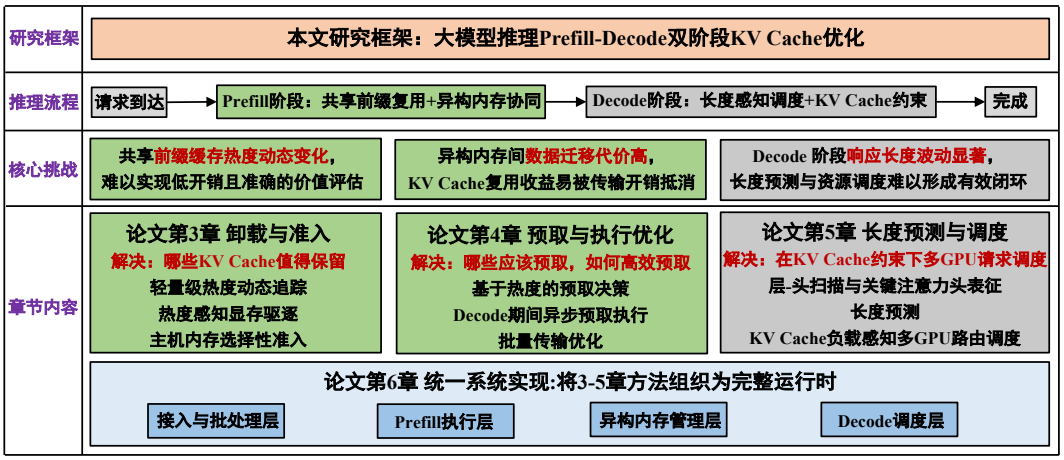


图 1-1　本文整体研究框架

缀共享、异构内存协同以及长度感知多 GPU 路由等基础内容，并给出本文关注的问题定义。

第三章研究面向 Prefill 阶段的 KV Cache 显存–主存卸载与准入策略，重点讨论热度动态追踪、缓存精准卸载以及主机内存准入机制。

第四章研究面向 Prefill 阶段的 KV Cache 主存–显存预取与执行优化策略，重点讨论预取决策、流水线执行以及批量传输优化。

第五章研究面向 Decode 阶段的响应长度预测与 KV Cache 负载感知多 GPU 路由方法，重点讨论长度预测表征驱动的预测模型、预测长度到未来 KV Cache 负载的映射，以及基于负载估计的多 GPU 请求路由策略。

第六章从系统角度串联第三章至第五章的方法，介绍统一运行时架构与关键模块实现。第七章对全文工作进行总结，并对未来研究方向进行展望。

第二章　背景知识与问题定义

本章围绕本文后续方法所依赖的基础概念展开，首先介绍大模型推理过程与 KV Cache 机制，然后说明异构内存管理与请求调度的基本原理，最后给出面向 Decode 阶段的长度感知推理优化背景与问题定义。

2.1　大模型推理过程与 KV Cache 机制

为了说明本文所关注的优化对象，本节先从模型计算流程、双阶段执行特性以及前缀共享组织方式三个方面介绍大模型推理过程中与 KV Cache 直接相关的核心基础。

2.1.1　Transformer 推理流程与显存开销

当代大语言模型大多采用仅解码器 Transformer 作为基础架构，其核心计算仍由多头自注意力和前馈网络交替堆叠构成[16-17,19,65-66]。设输入序列记为 $X = \{x_1, x_2, \cdots, x_T\}$，模型首先将 token 映射为隐藏表示，并在每一层中通过线性映射得到查询矩阵 Q、键矩阵 K 和值矩阵 V，随后使用缩放点积注意力完成上下文聚合，其基本形式为

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \tag{2-1}$$

其中 d 表示注意力头维度。该机制能够在整个上下文范围内建模 token 间依赖关系，是 Transformer 在长序列理解和生成任务中取得成功的关键原因[65]。在现代大模型中，RoPE 等位置编码进一步增强了模型对相对位置信息的建模能力，并成为长上下文推理的常用实现选择[16-17,67]。

在推理阶段，模型通常采用自回归方式生成输出，即每步基于历史上下文和已生成 token 预测下一个 token。若每一步都重新计算全部历史 token 的注意力，

计算代价会随着上下文长度快速增长。因此，推理系统普遍引入 KV Cache：将历史 token 在每层上对应的键张量和值张量缓存起来，使后续步次仅需为新 token 生成对应的 K 与 V，并与历史缓存共同完成注意力计算[2-4]。这一机制将 decode 阶段单步计算从反复重算全部历史转化为访问历史缓存并追加当前状态，是大模型在线推理可行性的基础。

从存储角度看，KV Cache 的开销通常可写为

$$M_{\mathrm{KV}} = 2 \cdot B \cdot L \cdot T \cdot H_{\mathrm{kv}} \cdot d_{\mathrm{head}} \cdot s, \tag{2-2}$$

其中 B 表示并发请求数，L 表示层数，T 表示总上下文长度，H_{kv} 表示 KV 头数，d_{head} 表示每个头的维度，s 表示单元素字节数。式中系数 2 对应键和值两类缓存。由此可见，KV Cache 容量与并发度、上下文长度和模型深度近似线性相关，而这几项恰好又是在线服务系统不断提升吞吐和模型能力时最容易增长的维度。按 Llama 2 的模型配置并以 FP16 存储估算，7B 模型在单请求、4K 上下文下的 KV Cache 约为 2 GB，扩展到 32K 上下文时将增至约 16 GB；13B 模型则约为 3.1 GB 和 25 GB[2,16]。这说明，对于 7B、13B 乃至更大规模模型，当上下文长度从数千扩展到数万 token 时，KV Cache 往往会从辅助状态演化为主导显存占用的关键资源[43,55]。

进一步地，现代大模型常通过 MQA 或 GQA 缩减 KV 头数，以降低缓存占用和带宽压力。MQA 让多个查询头共享同一组键值头，从而显著减少 decode 阶段的缓存读取和写入代价[68]；GQA 则在多头注意力与单写头之间折中，以较小精度损失换取更好的推理效率和工程兼容性[16,69]。这表明，KV Cache 并非被动存在的中间状态，而是已经反向影响了模型结构设计和推理系统实现方式。

与模型权重相比，KV Cache 还具有明显更强的动态性。模型权重在服务过程中通常保持不变，而 KV Cache 会随请求到达、前缀复用、生成推进和请求结束不断扩张、收缩与回收。对于在线服务系统而言，显存不仅要容纳模型参数，还要容纳激活、临时工作区以及不断增长的 KV Cache。在多请求并发场景下，真正限制系统吞吐和平均响应时间的，往往不再是纯算力上限，而是有限显存下能够容纳多少活跃请求及其缓存状态[2,7-8]。PagedAttention 通过分页式组织方式缓解了显存碎片与过度预留问题，vAttention 则借助虚拟内存机制实现与连续布

局兼容的动态分配[2,8]，但这些方法主要回答如何更高效地管理地址空间，并不直接回答哪些缓存更值得长期保留、跨层迁移或主动回迁等问题。

此外，KV Cache 的影响并不仅体现为容量占用，还直接反馈到调度粒度和批处理形态。当单个请求携带超长前缀或生成较长响应时，其缓存可能长期占据显存资源，压缩其他请求的可运行空间；当大量请求共享系统提示、few-shot 示例或检索文档时，若系统无法高效复用共享前缀对应的 KV Cache，则会重复触发 Prefill 计算并浪费显存带宽[25,66,70-71]。由此可见，理解 Transformer 推理流程、KV Cache 规模规律及其动态资源特征，是后续开展 Prefill–Decode 双阶段优化的前提。

2.1.2 Prefill-Decode 双阶段执行特性

大语言模型的生成式推理通常可拆分为 Prefill 和 Decode 两个阶段。Prefill 阶段负责处理输入提示词，对完整上下文执行并行 attention 计算，并为全部输入 token 构建初始 KV Cache；Decode 阶段则在此基础上逐 token 自回归生成输出，每一步仅新增一个 token 的键值并复用历史缓存[6,9-11]。两阶段共享同一套模型权重，但在计算模式、资源瓶颈、性能指标和优化目标上存在本质差异。图 2-1 直观展示了两个阶段的执行流程：Prefill 阶段一次性处理所有输入 token，经过各 Transformer 层后产生完整的 KV Cache 并生成第一个输出 token；Decode 阶段则以上一步生成的 token 和已有 KV Cache 为输入，逐步生成后续 token 并持续更新缓存，直至请求结束。

Prefill 阶段的主要特点是并行度高、计算密集、与输入长度高度相关。对于长提示词、检索增强生成、多样例提示、结构化代理调用等任务，系统往往需要一次性处理很长的输入序列，其计算时间和中间状态规模随输入长度快速增长[27,66,70-71]。从资源视角看，Prefill 更容易受到注意力计算量、显存瞬时峰值和输入复用模式的影响；从指标视角看，它与首 token 时延（time to first token, TTFT）密切相关，也是用户最容易直观感知的性能环节[10,40]。若系统能够识别并重用不同请求之间的公共前缀，则可显著减少 Prefill 阶段的重复计算和显存写入开销[25-28]。

Decode 阶段则呈现出完全不同的执行模式。由于每个新 token 的生成依赖前一时刻输出，Decode 在单请求内部天然串行，其单步计算量相对较小，但步

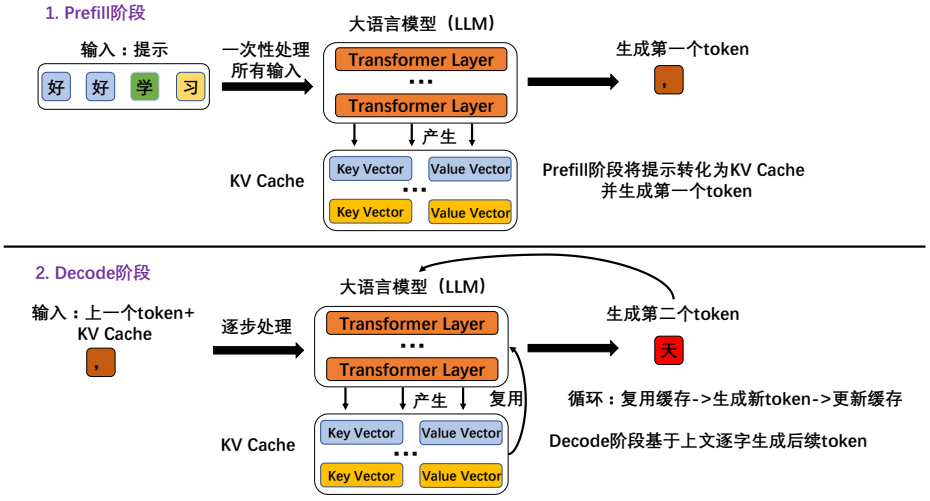


图 2-1　Prefill 与 Decode 双阶段执行流程示意

数不确定、累积时长长，并且对历史缓存读取高度敏感[6,10,12]。在批处理环境中，短请求可能很快完成，而长请求则会在后续多个步骤中持续占据批内位置，形成典型的尾部拖慢问题。系统在这一阶段更关注每输出 token 时延（Time Per Output Token, TPOT）、作业完成时间、吞吐与公平性之间的平衡[32-33,40]。

从系统优化手段看，两阶段差异进一步决定了适用方法的不同。Prefill 更适合围绕算子优化、前缀匹配、缓存复用、异构内存扩展以及数据迁移重叠展开；Decode 更适合围绕持续批次调度、长度感知调度、抢占、推测执行和 KV Cache 预算控制展开[2-4,9,15,72-73]。例如，投机解码通过先由草稿模型生成候选 token，再由目标模型并行验证，从而减少有效 decode 步数[15,72-73]；而 Splitwise、DistServe 等工作则进一步从系统部署层面拆分两阶段，将计算密集的 Prefill 与内存密集的 Decode 分配到不同资源池中[10-11]。

从资源生命周期角度看，两阶段共同决定了一次请求的显存占用曲线。请求在 Prefill 期间快速生成大段缓存，在 Decode 期间缓慢追加并长期持有这些缓存，直至请求结束后统一释放。换言之，Prefill 决定缓存如何被创建，Decode 决定缓存被占用多久。若系统只优化其中一端，整体收益往往有限：只提升 Prefill 速度而不管理 Decode 长尾，请求仍可能因长时间占用缓存而压低并发度；只优化 Decode 调度而不减少前缀重复 Prefill，则 TTFT 和显存写入成本仍然偏高。因此，本文将 Prefill 与 Decode 视为一个相互耦合的双阶段系统，分别针对其主导瓶颈设计不同的 KV Cache 优化策略。

2.1.3 前缀共享机制与前缀树组织方式

在真实应用中，不同请求经常共享相同的系统提示、角色设定、任务模板、少样本示例或检索文档，这使前缀共享成为降低 Prefill 成本的重要机会[25,66,70-71]。设两个请求的 token 序列分别为 X_a 和 X_b，其最长公共前缀长度为 $\ell_{\mathrm{lcp}}(X_a, X_b)$。若 ℓ_{lcp} 较大，则系统可以直接复用该段公共前缀对应的 KV Cache，仅对分叉之后的新 token 执行补充 Prefill。共享比例越高，节省的注意力计算与显存写入越明显[25-27]。

为高效管理共享前缀，推理系统通常采用前缀树、基数树或其他层次化索引结构组织请求前缀。前缀树中的每个节点可对应一段公共 token 序列，节点上记录该段前缀对应的 KV Cache 位置、token 区间、引用计数、访问频率与驻留位置等元数据。当新请求到达时，系统首先进行前缀匹配，定位最长可复用路径，然后在分叉位置继续扩展新的节点[27-29]。相较于简单哈希表，树结构更自然地支持部分匹配、多请求共享和层次化管理，尤其适合模板化提示、少样本提示和 RAG 场景。

从系统收益角度看，前缀共享同时降低了计算冗余和缓存写入冗余。若直接对所有请求独立执行 Prefill，则公共前缀的每个 token 都要反复计算一遍并反复写入对应层的 KV Cache；而在前缀复用场景中，系统只需为首个请求完成构建，后续请求通过引用已有缓存即可。Prompt Cache 从模块化注意力重用角度实现跨请求状态复用，ChunkAttention 通过 prefix-aware cache 改进共享前缀场景下的注意力计算执行效率，SGLang 则将结构化程序执行与前缀匹配统一到运行时框架中[25-27]。这些工作表明，前缀共享已从简单工程技巧逐步发展为系统级优化机制。

然而，前缀树组织方式也引入了新的管理难题。首先，树中节点数会随请求规模快速增长，若每个节点都长期保留对应 KV Cache，将很快耗尽 GPU 显存。其次，不同节点的复用概率高度不均衡。少数高频系统提示和模板前缀会反复命中，而大量长尾前缀命中次数极低，若对所有节点采用同等保留策略，会显著降低缓存利用效率[27,74]。再次，树中节点之间存在明显的层次价值差异：浅层节点往往被更多后代节点共享，其复用价值通常高于仅被少量请求命中的深层叶子节点。因此，前缀共享的关键不仅是能否共享，更在于哪些共享缓存应该长

期驻留、哪些适合迁移、何时回迁最合适。

更进一步地，前缀树中的共享关系还会放大缓存管理的连带效应。一个浅层公共节点若被错误驱逐，可能导致多个后续请求都失去复用机会；一个长尾叶子若被长期保留，则可能挤占更高价值节点的显存空间。由此可见，前缀共享虽然为 Prefill 阶段提供了重要优化空间，但其有效落地依赖精细的节点价值评估、异构内存放置与访问热度跟踪机制。本文第三章和第四章正是围绕这一问题展开。

2.2 大模型推理系统中的异构内存管理

在明确 KV Cache 的形成方式与资源特征之后，还需要进一步讨论其在系统层面的跨存储管理问题。本节从异构内存协同、缓存卸载与预取以及请求批处理机制三个方面介绍相关背景。

2.2.1 GPU 显存与主机内存协同

GPU 显存具有极高带宽和较低访问延迟，是执行大模型推理的核心存储层，但其容量通常远小于主机内存。当模型权重、激活张量和 KV Cache 共同争用有限显存时，系统往往无法仅依赖 GPU 本地存储支撑高并发和长上下文请求。相较之下，主机内存容量更大、成本更低，适合作为共享前缀缓存或冷数据的扩展存储层。因此，构建 GPU 显存与主机内存协同的异构内存体系，已成为提升大模型推理规模和经济性的常见手段[13,29-30,42-43]。

在异构内存体系中，GPU 显存通常承担热数据驻留和高频访问职责，主机内存则承担容量扩展和阶段性保留职责。对于短时间内高概率复用的共享前缀缓存，应优先保留在显存中，以避免重复 Prefill 或高代价回迁；对于暂时不活跃但未来仍可能复用的缓存，可卸载到主机内存；对于复用概率极低的缓存，则可以直接回收并在需要时重新计算。显存和主机内存之间通常由 PCIe、NVLink 或更高层级的互联网络连接，不同平台的带宽与时延差异会直接决定迁移策略的收益边界[30,42,75]。

值得强调的是，主机内存并非免费扩展。一方面，KV Cache 卸载和回迁会引入设备间传输开销，若迁移过于频繁，I/O 将反过来成为系统瓶颈；另一方面，主机内存虽然容量更大，但仍需维护索引、页表、元数据以及多请求共享关系，

若无选择地接纳全部被驱逐缓存，也会造成额外占用和管理负担[29-31]。因此，异构内存协同的关键不只是把缓存搬到 CPU，而是依据缓存价值、访问热度、共享程度和未来复用概率做出更精细的放置决策。

随着部署规模扩大，异构内存的边界还在继续扩展。Mooncake、MemServe、DejaVu 和 CacheGen 等工作已经开始探索远端内存池、故障容忍缓存流和压缩传输机制，使显存、主存、网络缓存乃至分布式内存池共同参与 LLM 服务[29-31,75]。这些工作说明，KV Cache 管理已不再是单机内部的数据结构问题，而是一个跨设备、跨存储层、跨服务实例的系统问题。

2.2.2 KV Cache 卸载、准入与预取基础

从系统视角看，异构内存中的 KV Cache 管理通常包含卸载、准入和预取三个基本操作。卸载是指在显存压力达到阈值或即将容纳新请求时，将部分已有缓存从 GPU 迁移至主机内存或直接回收；准入是指决定哪些被驱逐缓存值得保留在外部存储层，以便未来复用；预取则是在预测某些缓存即将再次被访问时，提前将其从主机内存回迁到显存，以提高后续请求的命中效率[29,31,42,74]。

三类操作彼此强耦合。若卸载策略仅依据最近最少使用等简单启发式进行决策，可能错误驱逐高价值共享前缀，导致后续命中率下降；若准入策略缺失，则大量低价值缓存也会被迁移至主机内存，造成无谓的 I/O 和内存占用；若预取策略过于保守，则缓存虽然被保存在主机内存中，却无法在真正需要前及时返回显存，难以转化为实际性能收益；若预取策略过于激进，又可能提前占据显存空间、挤出当前真正更热的缓存。因而，只有将卸载、准入和预取视为一个完整闭环，才能稳定发挥前缀共享场景下异构内存协同的价值。

从缓存价值建模角度看，系统往往需要同时考虑四类信息：第一，访问热度，即缓存近期和长期被命中的频率；第二，共享度，即一个缓存是否被多个请求或多个前缀子树复用；第三，迁移成本，即该缓存的体积、分块数量以及跨设备搬运代价；第四，替代代价，即若缓存被丢弃，未来重新计算该前缀需要付出的 Prefill 开销[25,27,74]。只有综合这些因素，系统才能判断某段缓存是应留在显存、应卸载到主存、应直接回收，还是应提前回迁等待命中。

在实现层面，卸载与预取还需要考虑数据组织和传输粒度。KV Cache 可以按层、按页、按块或按序列片段存放；迁移时可采用整块传输减少碎片化开销，

也可结合异步拷贝、流和事件同步机制，将数据传输与 GPU 计算重叠[2-3,31]。不同粒度将影响实际带宽利用率、调度复杂度和请求等待时间。若粒度过细，元数据维护和调度成本偏高；若粒度过粗，则可能放大无效传输。本文第四章将在此基础上进一步讨论如何结合 Prefill–Decode 执行特征实现流水式预取与批量传输优化。

除此之外，缓存压缩、量化与稀疏选择也可以与异构内存协同互补。一类方法通过重要标记选择、注意力汇点、标记稀疏化或检索头机制，减少必须保留的历史状态[44-46,52-54]；另一类方法通过低比特量化、层间预算分配或阴影缓存重构减少单个缓存块体积[48-51,55]。这些方法有助于降低容量压力，但它们大多关注单请求内部缓存压缩，而本文更关注多请求共享前缀在异构内存中的跨层级调度问题。

2.2.3 请求批处理与调度机制

批处理是提高 GPU 利用率的核心手段。通过将多个请求合并在一次或多次前向传播中共同执行，系统可以摊薄内核启动开销、提升矩阵计算并行度，并更充分地利用显存带宽。然而，大模型推理服务中的请求具有异步到达、输入长度不一、输出长度不一和共享前缀差异大的特点，传统静态批处理很难适应真实负载。因此，现代推理系统通常采用连续批处理、动态填充、阶段拆分、请求重排和跨实例迁移等技术，以兼顾吞吐、时延和公平性[2,6,9-10,12,32-33,40]。

连续批处理的核心思想是：不等待整个批次全部完成，而是在每个 decode 步骤或若干步骤后动态吸纳新请求、移除已完成请求。Orca 首先系统化展示了迭代级调度和持续批次调度在生成式模型服务中的优势，vLLM 则把这一思想与 PagedAttention 结合，显著提升了在线服务吞吐[2,6]。SARATHI 与 Sarathi-Serve 进一步通过分块 prefill 改善 Prefill 与 Decode 共存时的资源竞争关系[9,40]；DistServe 和 Splitwise 则从系统架构上拆分两阶段，减少相互干扰[10-11]。

对于 Prefill 阶段，批处理机制更关注如何合并具有相似输入长度或共享前缀的请求，以减少重复计算和冗余填充。若系统能够让共享前缀请求在相近时间进入服务队列，就更容易实现前缀树匹配、缓存复用和数据局部性提升[25,27-28]。对于 Decode 阶段，批处理机制则更关注如何避免长短请求混排导致的尾部拖慢问题，并在有限 KV Cache 预算下动态选择下一批进入解码的请求。由此可见，

批处理不仅是计算并行问题，也是缓存资源分配问题。

随着多模型、多租户服务需求上升，调度问题进一步从单模型多请求扩展到多模型、多实例、多适配器状态联合管理。Punica 研究了多租户 LoRA 服务下的共享底座模型与适配器复用[39]；MuxServe 则从空间–时间复用角度提升多 LLM 服务场景下的资源利用率[76]；Llumnix 进一步支持跨实例动态迁移与重平衡[12]。这些工作表明，调度策略已经不仅决定请求何时进入 Prefill 或 Decode，也决定哪些缓存被保留在哪个实例、何时迁移、是否抢占以及何时释放。若缺乏对 KV Cache 生命周期的认识，仅从纯计算角度安排批次，往往难以充分发挥异构内存与前缀共享的价值。

2.3 面向 Decode 阶段的长度感知推理优化

除了 Prefill 阶段的缓存驻留与迁移问题外，Decode 阶段还面临由响应长度不确定性引出的资源分配问题。本节进一步介绍长度分布特征、长度预测基础以及长度感知多 GPU 路由的形式化定义。

2.3.1 响应长度分布特征

Decode 阶段的核心不确定性来自响应长度。与输入长度不同，输出长度受到任务类型、提示内容、采样温度、停止条件以及模型内部推理路径等多种因素影响，往往呈现明显的重尾分布。对于事实问答等封闭式任务，模型输出可能只有十余个 token；而在思维链推理、代码生成、开放式对话和偏好数据生成场景中，输出长度则可能扩展至数百甚至上万 token[21-24,38]。即使输入提示几乎一致，不同采样结果之间仍可能存在明显长度波动。

长度分布的重尾特征会直接影响系统效率。若批次中混入少量超长请求，其余较短请求即使已经完成，也可能因批次等待策略而无法及时释放资源；若系统为所有请求统一预留较大 KV Cache 空间，则会造成显存浪费并降低并发度；若统一按照平均长度组织调度，则面对长尾请求时又容易出现批次失衡、频繁换批或饥饿现象[32,40]。因此，长度分布的统计规律不仅是任务语义属性，也是 Decode 阶段资源管理的重要先验。

从更细粒度的视角看，响应长度还与生成难度和推理路径复杂度相关。复杂

数学推理、长文摘要和代码合成任务通常需要更长的解码过程，而带有明确格式约束的结构化抽取、分类和固定模板生成则输出较为稳定[21-23]。这意味着长度信息并非纯随机噪声，而是可以从输入提示、任务类型乃至模型中间表示中部分估计出来。利用这一点开展长度感知调度，是缓解 Decode 长尾的重要方向。

2.3.2 响应长度预测方法基础

响应长度预测的目标是在生成开始前或生成过程中估计请求的最终输出长度或剩余生成长度。若从研究脉络看，相关工作大体可以分为两条主线：一条来自摘要、机器翻译与可控文本生成中的长度控制研究，强调显式注入目标长度信号；另一条来自在线推理与 LLM 系统优化中的长度预测研究，强调在不改变原始生成语义的前提下提前估计自然输出长度[38,56-60,62]。

在长度控制方向，早期工作往往通过额外长度嵌入、剩余长度编码或解码器控制变量，把期望输出多长显式传递给生成模型。Kikuchi 等较早将长度嵌入用于摘要生成，证明长度信息可以直接影响解码行为[56]；随后又有研究通过卷积编码器和基于注意力机制的长度控制机制增强模型在指定长度预算下的可控性[57-58,62]。这类方法的核心贡献在于说明：输出长度不是不可建模的随机变量，而是可以通过模型内部表示显式调节或近似预测的结构化属性。

在长度预测方向，机器翻译和非自回归生成研究提供了更直接的参考。由于非自回归模型需要在生成前决定目标端序列长度或长度区间，研究者通常会设计单独的长度预测器预测目标长度，或先预测长度分布再进行解码[59-60]。与完全开放式生成相比，这类任务的输出结构更规整，但它们在方法层面验证了长度预测头、长度分类器和分布式长度建模的可行性。近期在摘要领域，研究者也开始将摘要长度预测作为独立任务进行建模，以增强长短摘要生成的稳定性[61]。

进入大语言模型阶段后，长度预测开始更多依赖 LLM 自身的中间表示，而非外部独立特征。一类方法直接重用 Prefill 阶段最后层或中间层隐藏状态，通过轻量回归头预测长度；另一类方法则进一步挖掘熵、注意力模式和多层多头表示中的长度信号，以提升对长尾输出的鲁棒性[38,63-64]。其中，相关表征分析工作指出，长度信息往往并不均匀分布在所有层上，而是在部分中间层和若干显著注意力头上更集中地出现，这为本文后续采用逐层逐头筛选特征提供了背景支撑[64]。

从建模角度看，长度预测同时具有回归和分布学习属性。若直接预测单一标量长度，模型容易受长尾样本影响，在超长输出场景下误差增大；若仅将长度粗粒度离散化为少量区间，又可能损失细粒度信息，难以支持细粒度的 KV Cache 负载估计。因此，较新的方法更倾向于学习长度分布或软标签，再结合回归头输出更稳定的最终预测值。这类方法在面对复杂输出分布和多峰分布时通常更具鲁棒性[38,59,63]。

长度预测还必须兼顾系统开销。对于在线推理服务而言，预测本身不应引入明显额外时延或显存负担，否则其带来的调度收益会被抵消。因此，复用 Prefill 阶段已经产生的隐藏状态、注意力头输出或熵统计量，是更适合作为在线系统组件的实现路径[38,64]。从工程角度看，长度预测模块通常需要与调度器低耦合地集成：一方面，它应足够轻量，能够在请求进入 Decode 前快速给出长度先验；另一方面，它又应足够稳定，使路由器不会因预测抖动而在多 GPU 间做出明显错误的负载分配。

此外，长度预测与推理策略之间存在相互作用。较高温度采样、top-p 采样、拒绝采样或基于偏好的生成策略，都会放大相同输入在不同运行间的长度波动[24,38]。因此，长度预测不应被理解为对唯一真实长度的机械回归，而更适合理解为对未来资源占用规模的概率估计。本文第五章中的方法也正是沿着这一思路，先学习更稳定的长度相关特征，再将预测结果映射为多 GPU 场景下的 KV Cache 负载估计和请求路由决策。

2.3.3 长度感知路由问题定义

基于以上背景，本文将长度感知路由问题定义如下：在给定多 GPU 服务实例、每张 GPU 的显存预算、请求到达序列以及每个请求对应的前缀长度和预测响应长度的条件下，系统需要在请求进入后端执行前决定其目标 GPU，并结合各 GPU 当前已分配请求的预测 KV Cache 负载进行动态负载均衡，以优化整体吞吐、尾部延迟和整批完成时间等目标，同时避免某一 GPU 因长请求集中而成为系统瓶颈[32,34-37]。

设请求集合为 $\mathcal{R} = \{r_1, r_2, \cdots, r_n\}$，GPU 集合为 $\mathcal{G} = \{G_1, G_2, \cdots, G_K\}$。其中每个请求 r_i 包含输入长度 l_i^{in}、预测输出长度 $\hat{l}_i^{\text{out}}$ 以及相应的 KV Cache 占用函数 $m_i(l)$。若请求 r_i 被分配到 GPU G_k，则它对该 GPU 未来 Decode 阶段的预测负载

可近似表示为 $m_i(l_i^{\text{in}} + \hat{l}_i^{\text{out}})$。系统需要在任一时刻满足每张 GPU 的容量约束：

$$\sum_{r_i \in \mathcal{A}_k} m_i \left(l_i^{\text{in}} + \hat{l}_i^{\text{out}}\right) \leqslant M_k, \quad G_k \in \mathcal{G}, \tag{2-3}$$

其中 $\mathcal{A}_k$ 表示已经分配给 GPU G_k 的请求集合，M_k 表示该 GPU 可用于 Decode 阶段 KV Cache 的显存预算。长度感知路由的核心在于利用 $\hat{l}_i^{\text{out}}$ 估计请求未来的资源占用与剩余服务压力，从而更合理地选择目标 GPU。

为了刻画每张 GPU 的预测负载，还可进一步定义 GPU G_k 的负载估计为

$$\hat{L}_k = \sum_{r_i \in \mathcal{A}_k} m_i \left(l_i^{\text{in}} + \hat{l}_i^{\text{out}}\right), \tag{2-4}$$

并将其视为该 GPU 在后续 Decode 阶段可能承担的 KV Cache 压力。若忽略预测误差和调度开销，一个自然的策略是把新请求分配给当前 $\hat{L}_k$ 较小的 GPU，以避免长请求集中堆积。与经典 SRPT/SJF 更关注单队列中的服务顺序不同，本文关注的是多 GPU 服务场景下的请求分配问题：系统不仅要估计单个请求的未来长度，还要把该估计转化为不同 GPU 之间的相对负载信号，从而缓解生成长度高倾斜负载下的尾部延迟和吞吐下降问题[32-33]。

与 Decode 调度相对应，本文对 Prefill 阶段的相关问题也可作如下抽象：给定前缀树结构和异构内存层级，系统需要决定每个共享前缀节点的驻留位置、驱逐时机、主存准入以及回迁预取策略，以在有限 GPU 显存预算下最大化前缀复用收益、最小化重复 Prefill 计算和跨设备传输开销。这一问题本质上是一个同时受到热度、共享度、迁移成本和实时容量约束影响的分层缓存管理问题，也是第三章和第四章的核心研究对象。

2.4 本章小结

本章围绕本文研究所需的基础知识与问题定义展开论述。首先介绍了 Transformer 推理流程、KV Cache 的形成机理及其显存开销来源，并进一步说明 MQA、GQA 等结构设计如何从模型层面影响缓存规模；随后分析了 Prefill 与 Decode 两阶段在并行模式、资源瓶颈、性能指标和优化方法上的根本差异；在此基础上，讨论了前缀共享、前缀树组织方式及共享前缀缓存的层次化价值问题；接着介

绍了 GPU 显存与主机内存协同、KV Cache 卸载–准入–预取闭环以及请求批处理与调度机制；最后给出了响应长度分布、长度预测方法和长度感知多 GPU 路由的形式化定义，并补充说明 Prefill 阶段异构内存协同所对应的问题抽象。

第三章　面向推理 Prefill 阶段的 KV Cache 显存-主存卸载与准入策略

本章聚焦 Prefill 阶段共享前缀缓存的显存驻留问题，围绕热度追踪、显存驱逐和主存准入三个环节展开方法设计与实验分析。

3.1　问题背景与分析

在大模型在线推理中，Prefill 阶段承担着把输入上下文一次性编码成可复用状态的职责。对于带有系统提示词、任务模板、少样本示例和检索文档的真实请求而言，Prefill 阶段往往需要处理很长的输入序列，其注意力计算量、显存峰值和 KV Cache 写入量都与输入长度高度相关[27,66,70-71]。当不同请求之间共享相同或高度相似的前缀时，前缀共享可以显著减少重复 Prefill 计算；但与此同时，被共享的前缀 KV Cache 也会持续累积并快速消耗 GPU 显存[25-27]。因此，Prefill 阶段的根本矛盾并不是能否共享，而是共享之后产生的大量前缀缓存该如何管理。

前缀树中的浅层节点通常对应系统提示、模板前缀或公共文档片段，具有较强共享性；而深层节点则更接近用户特定输入或局部上下文，复用概率往往迅速下降。随着请求不断到达，前缀树节点数持续增长；若所有节点都长期驻留于显存，系统将很快触及容量上限，后续即使存在公共前缀也无法继续复用已有缓存。现有系统在显存压力升高时通常采用 LRU、LFU 或其他通用替换策略驱逐缓存，但这类方法既不理解前缀树的祖先–后代关系，也无法准确刻画节点未来的复用概率，容易把仍会被频繁命中的热点浅层节点提前驱逐[2,25,27,74]。

从在线服务视角看，这一问题还与系统目标直接绑定。真实推理系统通常既要追求高吞吐，也要满足单请求时延和批次稳定性要求；而 Prefill 阶段一旦因为显存紧张反复触发缓存抖动，就会同时恶化这三类目标。一方面，热点共享前缀被错误驱逐后，后续请求需要重新执行本可跳过的长前缀计算，直接抬高平均时延；另一方面，驱逐和卸载本身又会争用 GPU、CPU 和链路带宽，导致批次

间执行节奏被打乱。换言之，显存中的每一个前缀节点并不仅仅代表一段缓存数据，还对应着未来一批请求是否能够低成本复用这段计算结果。

一种看似直接的思路，是把被驱逐的 KV Cache 一律卸载到主机内存，用更大的主存空间换取更高的缓存容量。然而，简单的 GPU–CPU 协同方式并不可取。首先，GPU 与 CPU 之间的数据传输并非免费，若系统缺乏选择地将所有节点迁移至主存，PCIe 或 NVLink 链路很容易成为新的瓶颈[29,42-43]。其次，主机内存虽然容量更大，但也同样需要维护前缀树索引、节点元数据和缓存副本；如果低价值缓存大量堆积，后续即使再做预取，也只会更高频地传输低价值数据。也就是说，Prefill 阶段的核心问题并非是否卸载，而是哪些缓存必须离开显存、哪些缓存值得进入主存，以及系统如何持续追踪这些缓存的真实价值。以 Llama-2 13B、批大小为 8 的 MMLU 负载为例，若系统仅采用“显存不足即全部卸载到主存”的朴素策略，则总推理时延可达 2138 s，其中仅 GPU–CPU 卸载时间就约为 1200 s，占总时延的 56.1%[74]。这意味着，在共享前缀较多且请求持续到达的场景下，简单扩展主存容量并不能自然转化为性能收益；相反，若迁移对象缺乏筛选，异构内存链路本身就可能成为端到端执行的主要瓶颈。

更困难的是，这里的真实价值并不能通过单一静态指标直接给出。对于前缀树节点而言，是否值得保留至少受到四类因素共同影响：节点近期是否活跃、历史上是否被多次复用、节点位于树的浅层还是深层，以及该节点一旦失去会不会导致后续大段前缀重算。仅考虑时间特征，容易把短期活跃但整体价值不高的深层节点误判为热点；仅考虑频率特征，又可能把很早以前频繁访问、但当前负载分布已明显变化的节点继续长期保留。再考虑到在线系统必须在较低额外开销下完成这些判断，显然不能为每个节点维护复杂的完整访问历史。

进一步分析可知，这一问题至少包含三个彼此耦合的子问题。其一，是**价值评估问题**：系统需要在低在线开销前提下判断不同前缀节点的未来复用潜力，而不能仅依赖最近访问时间。其二，是**显存驱逐问题**：在显存受限时，系统应优先回收哪些节点，才能尽量不破坏后续前缀复用基础。其三，是**主存准入问题**：哪些被驱逐节点值得继续保留在主机内存，哪些节点应直接回收，以避免无效传输和主存污染。若这三个环节缺乏统一设计，即使单点策略看似合理，也可能在端到端执行中相互抵消收益。

本文的核心观察是：有限的 GPU 显存应优先留给高热度、未来更可能被再

次访问的共享前缀 KV Cache；对热度较低的节点，系统要么直接丢弃，要么在经过筛选后再写入主机内存。围绕这一目标，本文在 Prefill 阶段构建了一个逐层收缩候选空间的缓存管理链路：首先，利用动态热度追踪机制持续记录前缀树节点的访问状态；其次，基于热度感知的显存驱逐策略，从 GPU 中优先移除复用价值最低的节点；再次，利用选择性准入机制，只把具有较高复用潜力的节点保留在主机内存中；最后，再在第四章中对这些高价值节点执行主存到显存的主动回迁。

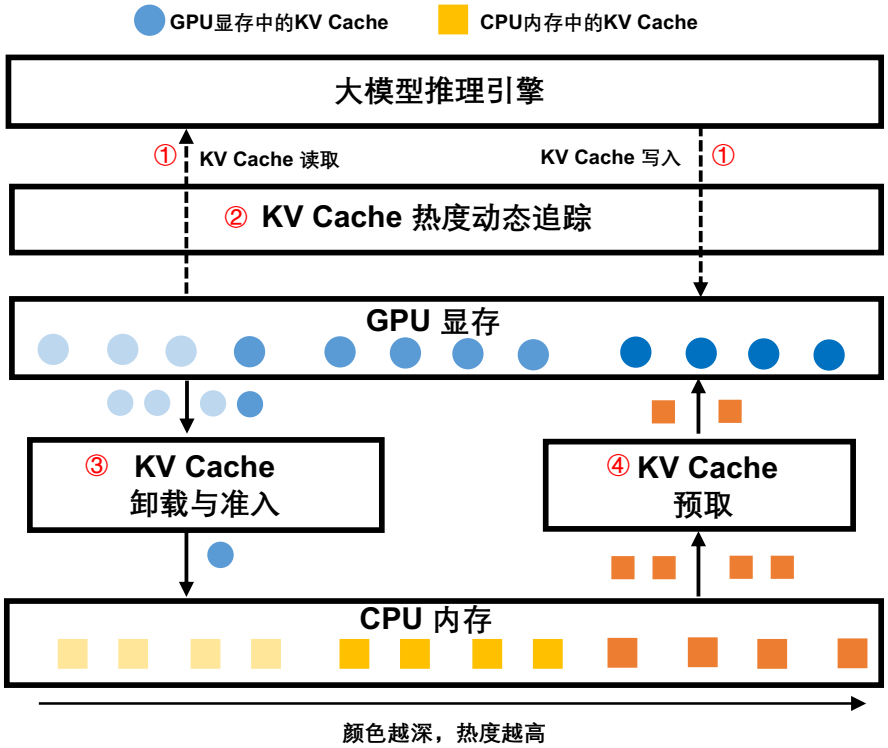


图 3-1　总体工作流程

图 3-1 给出了本文在异构内存层级中管理共享前缀 KV Cache 的整体工作流。该工作流包含三个核心环节：其一，**动态热度追踪**持续记录并更新前缀树节点的热度信息，为后续缓存管理提供低开销、实时的决策依据；其二，**选择性准入**在显存空间不足时执行热度感知驱逐，并仅将更可能被未来访问的高价值节点保留到主机内存；其三，**热度提升**在后续批次中进一步将主存中的高热度节点主动回迁到显存。

3.2 基于轻量级过滤器的 KV Cache 热度动态追踪

围绕上一节提出的价值评估问题，本节首先解决如何以较低在线开销持续刻画前缀树节点热度这一基础问题。为此，本文先给出面向前缀树节点的轻量级热度记录结构，再进一步说明该结构如何在真实请求到达和复用过程中完成动态更新，从而为后续显存驱逐与主存准入提供统一、可复用的热度依据。

3.2.1 基于布谷鸟过滤器的 KV Cache 热度记录结构

在本文的设计中，GPU 显存和主机内存中的 KV Cache 都统一挂接到同一棵前缀树上管理。图 3-2 展示了这一组织方式，其中圆形节点表示驻留在 GPU 显存中的 KV Cache，方形节点表示驻留在主机内存中的 KV Cache；父节点对应子节点的前缀，因此不同请求可沿树结构共享公共前缀。为了进一步判断哪些节点未来更可能被再次访问，系统还需要为每个前缀树节点维护低开销、可实时更新的热度记录。

若直接为所有节点维护精确统计表，虽然能够完整保留访问历史，但会带来较高的元数据开销和查询开销。为此，本文引入了增强型热度感知布谷鸟过滤器，用于记录前缀树节点的热度信息[77]。如图 3-3 所示，该过滤器底层是连续地址空间上的哈希表，由多个桶组成，每个桶包含 4 个槽位；每个槽位保存一个前缀树节点的四类元数据：指纹、时钟值、访问频次和树深度。其中，指纹用于紧凑标识节点，时钟值反映节点最近一次活跃以来的时间衰减状态，访问频次表示累计复用次数，树深度则刻画节点在前缀树中的层次位置。

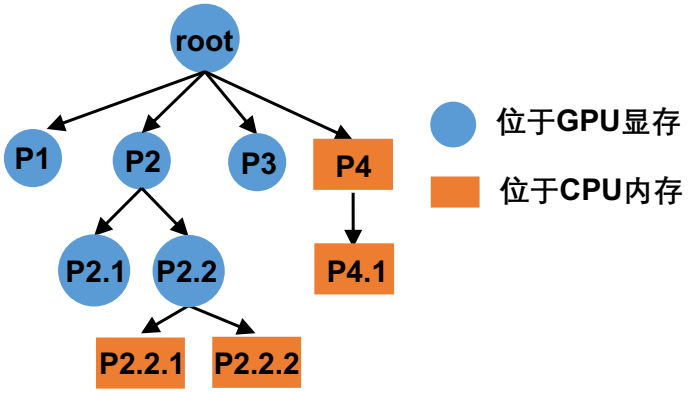


图 3-2 前缀树缓存结构

图 3-2 展示了本文采用的前缀树式 KV Cache 组织方式，其中圆形节点表示驻留在 GPU 显存中的前缀缓存，方形节点表示驻留在主机内存中的前缀缓存。前缀树中父节点表示子节点对应序列的公共前缀，因此不同请求只要共享相同的上层路径，就能够复用同一段已经生成的 KV Cache。以图中的节点 $P2$、$P2.1$ 和 $P2.2$ 为例，$P2$ 表示一段已经被缓存的公共前缀，而 $P2.1$ 与 $P2.2$ 则表示在该公共前缀基础上继续向下扩展得到的两条不同路径；也就是说，$P2.1$ 和 $P2.2$ 对应请求的前缀部分都包含 $P2$ 所代表的 token 序列，只是在后续追加内容上有所不同。因此，一旦 $P2$ 对应的 KV Cache 已经存在，后续命中 $P2.1$ 或 $P2.2$ 的请求都可以直接复用这部分前缀结果，仅需对各自新增的后缀部分继续执行计算。基于这一结构，缓存管理不再以整条请求为粒度，而是以**前缀树节点**为粒度进行热度记录、显存驱逐和主存准入；这也是本文后续所有热度判断与跨层调度都围绕树节点展开的基础。

设前缀树中新生成节点 m 的唯一标识为从根节点到该节点的 token 路径 x，则首先计算其指纹 f，再据此得到两个候选桶位置 b_1 与 b_2：

$$f = fingerprint(x), \tag{3-1}$$

$$b_1 = hash(f), \tag{3-2}$$

$$b_2 = b_1 \oplus hash(f). \tag{3-3}$$

其中，哈希函数采用 MurmurHash[78]。得到两个候选桶后，系统检查 b_1 和 b_2 中是否存在空槽；若存在空槽，则将节点 m 的指纹、时钟值、访问频次和深度写入该槽位，其中初始时钟值设为预定义的 max_age，访问频次初始化为 1，深度则记录为节点在前缀树中的当前层次。

如果两个候选桶都已满，系统会随机选择一个桶，再从该桶中随机选择一个已有槽位，把其中节点的信息迁移到它的备用桶；若备用桶也已满，则继续执行相同的重定位过程，直到找到空槽或达到最大迭代次数。若仍然无法插入，说明布谷鸟过滤器当前空间不足。不过，这种失败在实践中极少发生：一方面，布谷鸟过滤器本身具有较高装载率和较小元数据开销；另一方面，该结构存放于主机内存中，空间相对充足[77]。

图 3-3 展示了热度感知布谷鸟过滤器的内部组织形式。该结构本质上是一个

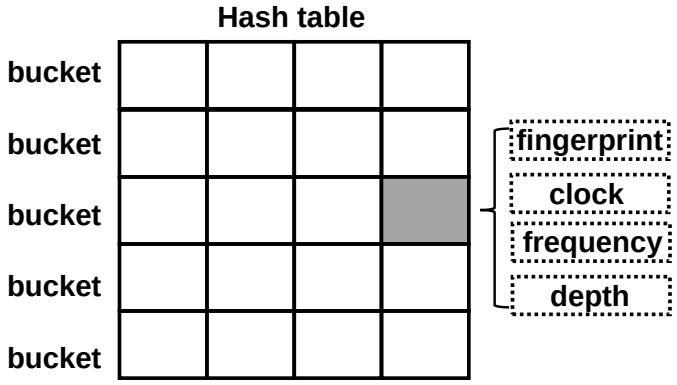


图 3-3　KV Cache 节点热度感知布谷鸟过滤器

连续地址空间上的哈希表，由多个桶组成，每个桶包含 4 个槽位；每个槽位分别记录节点指纹、时钟值、访问频次和树深度四类元数据。与传统仅判断成员是否存在的过滤器不同，本文在同一轻量级结构中同时保留了节点的紧凑标识、时间局部性、累计复用频次以及结构位置信息，从而能够在较低元数据开销下支持更精细的热度估计。也正因为这些信息被统一维护在同一结构中，后续显存驱逐、主存准入与第四章的热度提升都能够直接复用同一套热度记录，而无须额外维护独立统计副本。

3.2.2　面向动态访问模式的 KV Cache 热度实时更新

在建立热度记录结构后，本文设计了一套与在线推理过程兼容的动态更新机制，使系统能够持续追踪热点迁移。其基本思想是：当某个前缀树节点被复用时，立即更新其热度记录；而随着更多请求被处理，系统再周期性衰减全部节点的时钟值，以反映时间局部性的变化趋势。

热度更新过程可以分为两个步骤。

步骤一：复用触发更新　当某个节点 m 被后续请求复用时，系统重新收集从根到节点 m 的 token 路径 x，并利用式 (3-1)–式 (3-3) 定位其在布谷鸟过滤器中的两个候选桶 b_1 与 b_2。由于插入过程已经保证节点元数据必然位于其中一个桶内，因此只需遍历 b_1 与 b_2，即可找到节点 m 对应的槽位。随后，系统将该节点的访问频次加 1，并把时钟值重置为预定义的 max_age。例如，图 3-4 中节点内部的 “C” 表示时钟值（clock），“F” 表示访问频次（frequency）；左侧被复用的示

例节点在更新前满足 $C = 9$、$F = 5$，若设定 $max_age = 15$，则该节点在一次复用后会被更新为 $C = 15$、$F = 6$。

步骤二：周期老化更新 仅靠访问频次无法区分历史上曾经热门但近期已不活跃的节点，因此本文还设计了周期性老化机制。在处理完固定数量的用户请求后，系统由 CPU 侧后台线程遍历整个布谷鸟过滤器，将每个槽位的时钟值减 1；若某节点的时钟值已经减到 0，则继续保持为 0 而不再下降。这样一来，最近较少被访问的节点会逐渐失去时间局部性优势，而近期刚被访问的节点则因为时钟被重置，能够在后续驱逐与准入决策中获得更高优先级。

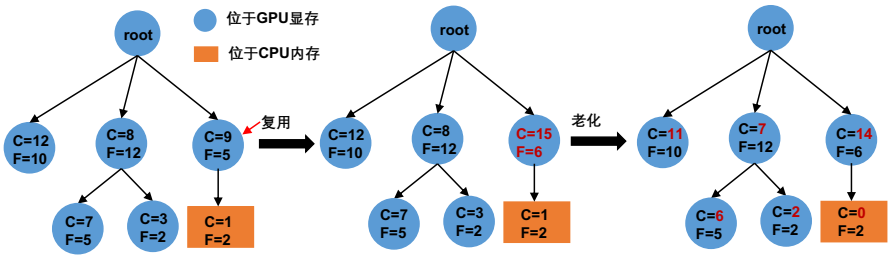


图 3-4 KV Cache 节点热度感知布谷鸟过滤器的更新过程

图 3-4 展示了热度记录的动态更新过程，其核心由“复用触发更新”和“周期老化更新”两部分组成。图中每个节点内部的“C”表示时钟值，“F”表示访问频次。以左侧示例节点为例，其在被复用前的状态为 $C = 9$、$F = 5$；当该节点被后续请求再次命中时，系统会将其时钟值重置为预设的 $max_age = 15$，同时将频次增加 1，即更新为 $C = 15$、$F = 6$。而当系统处理完固定数量请求后，后台线程会对过滤器中的全部节点执行统一老化，使各节点时钟值都减 1，从而使长期未被访问的节点逐步失去时间局部性优势。通过这两类更新，本文能够同时刻画节点的长期复用趋势与短期活跃程度，从而使热度估计既能保留稳定热点，又能及时反映访问模式迁移。

上述机制使热度定义同时吸收了两类关键信息：频次反映节点在较长时间尺度上的累计复用潜力，时钟值反映节点在最近时间窗口内的活跃程度。二者结合后，系统能够更准确地区分稳定热点、短时热点和长尾冷点。更重要的是，时钟老化由 CPU 线程执行，因此不会占用 GPU 的推理计算资源；这也是其具有低在线开销的重要原因。

3.3 基于热度的 KV Cache 精准卸载与高效准入

在具备统一热度信息之后，系统接下来需要将这些信息用于显存与主存之间的缓存放置决策。本节进一步讨论 Prefill 阶段的两个关键动作：一是当显存不足时如何选择应被移出的 GPU 节点，二是当节点离开显存后哪些值得继续保留在主机内存中。二者共同构成第三章方法从状态感知走向实际控制的核心环节。

3.3.1 基于热度的 GPU 显存 KV Cache 卸载机制

当新到达请求在 Prefill 阶段需要写入新的 KV Cache，而当前 GPU 显存已经接近容量上限时，系统必须从已有前缀缓存中选择一批节点进行驱逐。现有推理系统多采用 LRU 之类的简单策略，但这类策略无法利用前缀树的层次结构和节点热度差异，容易在乱序负载下错误驱逐高价值的共享前缀。

为此，本文提出了基于热度的显存驱逐策略。其第一个关键设计是：驱逐只在当前位于 GPU 显存中的叶子节点集合上进行，而不直接在整棵前缀树上选择候选。原因在于，前缀树中的祖先节点通常比后代节点具有更高热度，若直接驱逐浅层祖先节点，会破坏多条共享前缀路径的复用基础。这里的 GPU 叶子节点是指没有后代节点，或其后代已全部位于主机内存中的节点。

在候选叶子节点之间，本文使用如下优先级函数衡量节点继续留在显存中的价值：

$$priority = frequency + \frac{clock}{length}. \tag{3-4}$$

其中，$frequency$ 表示节点累计被访问的次数，且不局限于该节点停留在 GPU 中的时间：即使节点曾多次在 GPU 和 CPU 之间交换，其总访问频次仍然持续累积。$clock$ 表示该节点距离最近一次被装入 GPU 显存以来的时间状态，可由上一节的热度记录结构直接读取；$length$ 则表示该节点所包含的 token 数量。式 (3-4) 同时兼顾了长期复用趋势、短期时间局部性以及节点包含 token 数目，因此能够优先驱逐频次低、最近不活跃且占用空间大的冷节点。

例如，在 RAG 场景中，一些外部文档前缀可能包含数万 token，虽然偶尔会被命中，但其长度较长、占用显存较大；在热度相近时，式 (3-4) 会更倾向于优先回收这类长而冷的节点，从而提高有限显存空间的利用效率。

算法 3-1 给出了这一驱逐策略的执行流程。系统首先收集当前位于 GPU 中

算法 3-1 基于 KV Cache 节点热度的 GPU 显存 KV Cache 驱逐算法

输入： GPU 中前缀树节点 token 总量 N_{GPU}，GPU token 容量上限 C_{GPU}，前缀树根节点 $root$，新到达节点 m
输出： 完成驱逐与插入后的 GPU KV Cache 状态

while $|N_{GPU}| \geqslant C_{GPU}$ **do**
  $LeafNodes \leftarrow$ FindLeafNodes($root$)
  for each $n \in LeafNodes$ **do**
    $priority(n) \leftarrow frequency(n) + \frac{clock(n)}{length(n)}$
  end for
  while $|N_{GPU}| \geqslant C_{GPU}$ **and** $LeafNodes$ 非空 **do**
    $EvictNode \leftarrow \arg\min_{n \in LeafNodes} priority(n)$
    将 $EvictNode$ 从 GPU 显存中移除
    将 $EvictNode$ 从 $LeafNodes$ 中删除
    $N_{GPU} \leftarrow N_{GPU} - EvictNode.length$
  end while
end while
将节点 m 及其对应 KV Cache 写入 GPU 显存

的所有叶子节点，并为每个节点计算式 (3-4) 所定义的优先级；随后，只要显存占用仍高于预算上限，就不断选择优先级最低的节点执行驱逐，直到腾出的空间足以容纳新请求对应的 KV Cache。由于这一流程始终把驱逐范围限制在叶子节点集合中，因此不会轻易破坏共享前缀的祖先链；而优先级函数又能综合考虑频次、时间局部性和长度，从而显著降低错误驱逐热点前缀的概率。

3.3.2 基于热度的主机内存 KV Cache 准入方法

仅靠显存驱逐还不足以构成完整的异构内存策略。更关键的问题是：哪些被驱逐节点值得继续写入主机内存，哪些节点应该被直接丢弃。如果系统把所有被驱逐的 KV Cache 都无条件卸载到主机内存，GPU 到 CPU 的传输开销将迅速膨胀，而且主机内存中还会堆积大量未来几乎不会再次复用的低价值缓存[42-43]。

因此本文提出选择性准入机制，其背后的观察是：共享前缀中只有一部分节点会被不同请求反复复用，而大量用户特定前缀或模型生成输出对应的 KV Cache，其热度很低，继续保存几乎没有实际收益。基于这一观察，本文将主机内存从显存驱逐后的被动接收区转变为只保留高价值缓存的筛选层。图 3-5 展示了这一过程。

图 3-5 展示了选择性准入的完整决策流程。图中左侧蓝色圆形节点表示刚刚

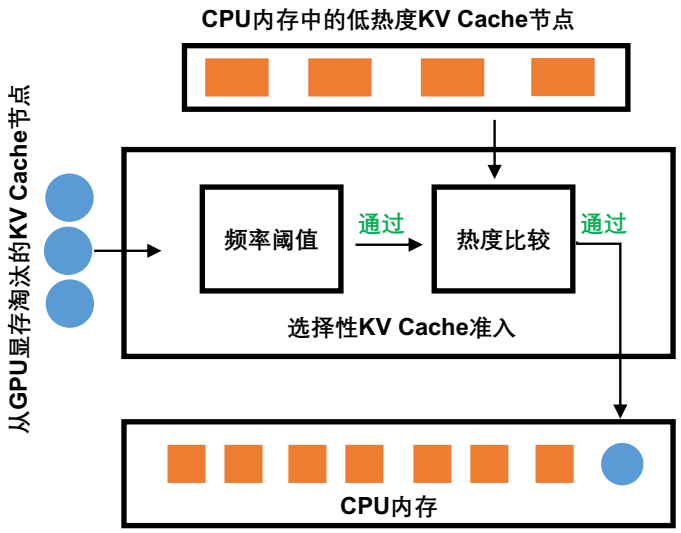


图 3-5　主机内存准入过程

从 GPU 显存中被驱逐出来、因而具备潜在准入资格的候选节点；这些节点首先经过访问频次阈值筛选，以剔除未来复用概率显著偏低的前缀缓存。随后，通过筛选的候选节点还需与主机内存中当前价值最低的一批节点进行热度比较，只有热度更高的节点才会最终被写入主机内存。由此，图 3-5 所强调的并不是“所有显存淘汰项都转储到主存”，而是主机内存只保留真正具有后续复用价值的高热度前缀缓存，从而同时降低传输开销与主存污染。

选择性准入具体分为以下四个步骤。

收集 GPU 驱逐节点　首先，系统接收由上一小节热度感知驱逐策略从 GPU 中移除的节点集合。这些节点是潜在的主存准入候选，但并不会被立即全部写入主机内存。

频次阈值过滤　随后，系统对每个候选节点做访问频次阈值检查。若某节点的累计访问频次低于预定义阈值，则将其视为复用潜力不足的低价值缓存，直接丢弃而不进入主机内存。这样可以在最早阶段过滤掉大量明显缺乏价值的节点，避免它们继续占用传输带宽和主存空间。

热度比较 对于通过频次阈值检查的候选节点，系统还要进一步与当前主机内存中最不重要的节点进行热度比较。本文在这一阶段使用如下热度定义：

$$hotness = frequency \times clock. \tag{3-5}$$

这里，$frequency$ 与 $clock$ 都直接来自热度记录结构。若当前被驱逐节点的热度低于主机内存中已有低价值节点，则说明它的未来复用潜力仍然不足，此时系统会直接丢弃该节点；只有当其热度足够高时，系统才允许其写入主机内存。通过这一步比较，主机内存中保留下来的始终是更有意义的一批节点，而不是简单承接所有来自显存的淘汰项。

写入主机内存 最终，只有同时通过频次阈值过滤和热度比较的节点，才会执行 GPU 到 CPU 的异步传输，并更新其在前缀树中的驻留位置元数据。

选择性准入带来了两个直接收益。第一，它显著降低了 GPU 与 CPU 之间的数据传输量，从而控制卸载延迟；第二，它提升了单位主存容量所承载的缓存价值，使主机内存可以在不必设置得远大于显存的前提下，仍然保存一批具有较高复用潜力的共享前缀。在具体实现中，分配给主机内存的 KV Cache 容量与 GPU 中 KV Cache 的预算空间保持同量级，但由于准入过程已经滤除了大量低热度节点，这样的容量配置已足以支撑后续复用。

3.4 实验评估与分析

为验证本章提出的热度追踪、显存驱逐与主存准入机制是否能够在真实工作负载下稳定转化为系统收益，本节从实现环境、数据集设置和关键性能指标出发，依次评估热度驱逐策略本身的有效性，以及选择性准入机制对端到端性能的实际影响。

3.4.1 实验环境与实现细节

系统实现于 SGLang 0.4.1.post1 之上，并继续沿用其按 token 粒度管理 KV Cache 的基本方式；GPU 显存与主机内存中的前缀缓存由统一前缀树管理[27]。在具体实现中，第三章只替换缓存管理相关逻辑，而尽量保持底层执行框架不变：

即各对比方法共享同一套前缀树组织方式、请求处理流程和异构内存传输路径，差异仅体现在如何记录热度、如何选择驱逐对象、以及是否允许节点进入主机内存这三个决策点上。这样做的目的，是把实验结果更直接地归因于缓存管理策略本身，而不是其他执行路径差异。

在热度追踪模块中，布谷鸟过滤器对指纹、时钟值、访问频次和深度均采用 8 bit 长度配置，max_age 设为 255；当节点被复用时，频次加 1、时钟值重置为 255，同时由 CPU 侧独立线程周期性执行时钟老化。对于未通过选择性准入的节点，系统会立即删除其元数据并释放对应显存；对于通过准入的节点，则执行 GPU 到 CPU 的 KV Cache 迁移。

硬件平台方面，实验服务器配备 256 GB DRAM 和 3.84 TB NVMe SSD，GPU 与主机之间通过 PCIe Gen 3 连接。本章实验主要基于 LLaMA-2 13B 模型展开[16]。在对比设置上，显存驱逐实验统一比较本文热度驱逐与 LRU、LFU、FIFO、2Q 四种代表性策略；准入实验则比较启用选择性准入和关闭选择性准入、全部写入主机内存两种配置。前者用于验证热度信息是否足以支撑更优驱逐判断，后者用于验证主存是否应当无条件接收所有被驱逐前缀。

3.4.2 数据集、对比方法与评价指标

实验工作负载选取了 5-shot MMLU、10-shot HellaSwag、BBH 和 C-EVAL 四类代表性基准测试，其中 BBH 与 C-EVAL 进一步引入思维链提示[20,22,79-80]。四类工作负载的平均输入长度分别约为 567、753、723 和 1673 token，覆盖了中等长度共享前缀与超长前缀两类典型场景。为了模拟在线服务环境中请求随机到达的真实情况，实验在执行前会对每个数据集中的查询进行乱序处理，以避免人工排序导致的异常前缀聚集现象；同一组乱序请求序列会被复用于所有对比方法，以保证不同策略面对完全一致的共享模式和访问顺序。

在展示方式上，本章的驱逐与准入消融图主要选取 MMLU、C-EVAL 和 HellaSwag 三个负载进行展示。这是因为这三个数据集分别覆盖了中等长度前缀、超长复杂前缀以及中等长度但共享模式较离散的场景，更适合直接观察不同缓存策略对命中率、卸载时间和总时延的影响；而 BBH 的现象会在第四章端到端实验中继续展示，用于说明完整系统闭环下的收益。

对比方法 本章对比方法分为两组。第一组用于验证显存驱逐策略本身，包括 LRU、LFU、FIFO 和 2Q 四类经典缓存替换方法，以及本文提出的热度感知驱逐。它们分别代表仅依据最近访问时间、访问频次、先进先出顺序和双队列近似重用距离进行决策的代表性方案。第二组用于验证主存准入策略，包括**无选择性准入**与**选择性准入**两种配置：前者会把所有被 GPU 驱逐的节点都写入主机内存，后者则在写入前进一步执行频次阈值过滤与热度比较，仅保留更可能再次复用的节点。由于这些方法共享完全一致的执行路径与异构内存链路，因此实验差异可以直接归因于驱逐与准入策略本身。

评价指标 为避免仅凭单一指标得出片面结论，本文同时从缓存收益、端到端性能和异构内存开销三个角度评估 Prefill 阶段策略效果。

本文重点使用以下四项指标评估 Prefill 阶段的缓存管理效果：

- **缓存命中率：**表示前缀 KV Cache 访问中有多少比例能够直接命中 GPU 显存；
- **端到端时延：**表示处理完整工作负载所需的总执行时间；
- **吞吐率：**表示系统每秒处理的 token 数量；
- **卸载时间：**表示 GPU 与 CPU 之间进行 KV Cache 传输所消耗的时间。

3.4.3 热度追踪与卸载策略有效性验证

图 3-6 对比了热度感知驱逐与 LRU、LFU、FIFO、2Q 等策略在 MMLU、C-EVAL 和 HellaSwag 上的表现。总体上，本文方法在三类工作负载上都同时取得了更高的缓存命中率、更低的端到端时延和更短的 GPU–CPU 卸载时间，而且这种优势并非只体现在某一个指标上，而是表现为命中率更高、传输更少、总时延更低的一致改进。

从具体数据看，MMLU 上本文方法的命中率为 0.8089，高于 LRU 的 0.7973、LFU 的 0.7454、FIFO 的 0.7996 和 2Q 的 0.8041；对应端到端时延为 556.07 s，相比 LRU 的 762.30 s 和 LFU 的 739.93 s 分别下降约 1.37 倍和 1.33 倍；而卸载时间仅为 25.66 s，显著低于 LRU 的 299.74 s 和 FIFO 的 316.67 s。也就是说，在 MMLU 这类共享前缀较稳定、但仍存在一定请求扰动的场景中，热度驱逐已经能把少量更准的保留直接转化为更少的回迁和更短的总时延。

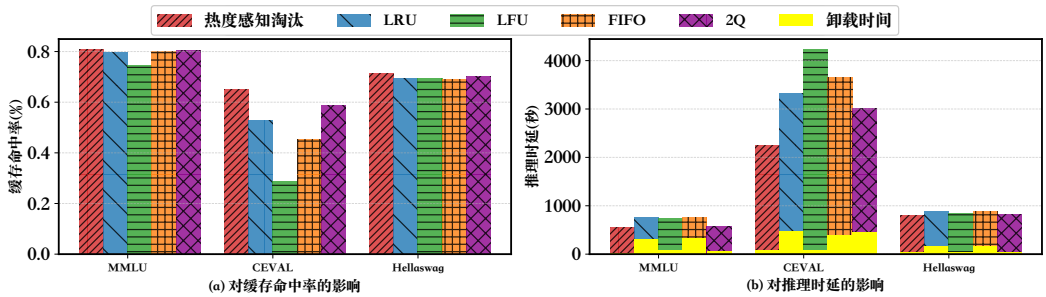


图 3-6　不同显存驱逐策略对缓存命中率和推理时延的影响，黄色部分表示 GPU 与 CPU 之间的卸载时间

在 C-EVAL 上，这一差异最明显。本文方法的命中率达到 0.6495，而 LRU、LFU、FIFO 和 2Q 分别仅为 0.5278、0.2863、0.4530 和 0.5881；对应总时延分别为 2241.64 s、3326.97 s、4234.11 s、3648.14 s 和 3005.73 s。尤其相较 LFU，本文方法在命中率上提升了约 2.27 倍，在总时延上缩短了约 1.89 倍；相较 2Q，其卸载时间也从 460.63 s 降至 68.28 s。C-EVAL 的提示更长、结构更复杂，错误驱逐一个热点前缀往往意味着后续多次 Prefill 重算或多次异构内存回迁，因此驱逐策略能否准确识别未来仍会被复用的节点会被放大到系统总时延上。

HellaSwag 上的收益虽然没有 C-EVAL 那样极端，但趋势依然一致。本文方法的命中率为 0.7144，高于 LRU 的 0.6922、FIFO 的 0.6897 和 2Q 的 0.7026；端到端时延为 789.46 s，低于 LRU 的 877.53 s、LFU 的 844.64 s 和 FIFO 的 890.36 s；卸载时间也只有 27.04 s，而 LRU 和 FIFO 分别达到 160.53 s 与 168.04 s。这说明在中等长度前缀场景中，热度驱逐依然能够稳定减少无效淘汰，只是因为负载本身的共享结构没有 C-EVAL 那么极端，所以绝对收益更温和。

上述结果说明，热度感知驱逐的收益主要来自两个方面。首先，它联合利用访问频次、时间局部性和节点深度，优先淘汰真正低价值的显存节点，减少了热点节点被误驱逐后再次回迁的情况，因此 GPU–CPU 传输时间显著降低。其次，在动态热度追踪机制支撑下，系统能更稳定地把有限显存留给高复用概率前缀，因此命中率也同步提升。换言之，图 3-6 体现的并不只是换了一种驱逐公式，而是上一节热度追踪信息确实足以支撑后续显存替换做出更优判断。

3.4.4 准入策略对推理性能的影响分析

进一步考察选择性准入机制的作用。图 3-7 比较了在启用选择性准入和关闭选择性准入两种配置下的性能表现。关闭该策略时，所有从 GPU 中驱逐出的 KV Cache 都会被无条件写入主机内存；而在启用选择性准入后，系统会对这些节点先做频次阈值过滤，再做热度比较，仅保留其中最有价值的一部分。

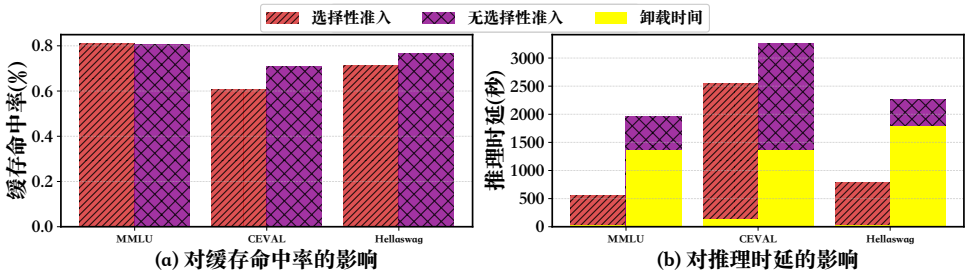


图 3-7 选择性准入策略对缓存命中率和推理时延的影响，黄色部分表示 GPU 与 CPU 之间的卸载时间

从结果看，选择性准入并不是一个单纯追求命中率最大的策略，而是一个显式控制传输成本的策略。以 MMLU 为例，关闭准入后虽然命中率略高，为 0.8064，而启用准入后为 0.8089，两者几乎持平，绝对差只有 0.25 个百分点；但总时延却从 1966.31 s 降到 556.07 s，提升约 3.54 倍，卸载时间更是从 1357.76 s 骤降到 25.66 s，下降约 52.9 倍。也就是说，在 MMLU 上，关键并不在于主存中是否存在缓存，而在于是否将过多低价值缓存迁移至主存，后者才是造成系统性能下降的主要原因。

在 C-EVAL 上，选择性准入体现出更典型的权衡关系。关闭准入时命中率为 0.7064，而启用准入后为 0.6087，绝对下降了 9.77 个百分点；但与此同时，总时延仍从 3253.40 s 降至 2546.96 s，获得约 1.28 倍改进，卸载时间也从 1357.83 s 降到 132.89 s，缩短约 10.22 倍。这说明对于超长前缀负载而言，主存容量本身并不是唯一瓶颈，更关键的是主存中保存的内容是否足够值钱。如果为了追求表面更高的命中率而盲目接收所有被驱逐节点，就会让链路带宽和主机侧管理成本迅速膨胀，最终侵蚀掉命中率带来的收益。

HellaSwag 上的现象同样支持这一判断。关闭准入时命中率为 0.7645，启用准入后为 0.7144，绝对下降约 5.01 个百分点；但端到端时延从 2261.69 s 降至 789.46 s，提升约 2.87 倍，卸载时间则从 1780.89 s 大幅降至 27.03 s，下降约 65.9 倍。这里可以看出，某些看上去可能会命中的缓存，实际上仅在主存中占用空间

并反复触发低收益传输，并没有带来与其成本相匹配的重用收益。

因此，选择性准入的价值正在于它引入了有意识的保守性。系统宁可放弃少量边际收益不明确的缓存，也不愿让大量未来几乎不会复用的节点占满主机内存并拖累传输链路。从实验数据可以更清楚地看到，在异构内存环境下，命中率并不是越高越好，更为关键的是命中率提升能否覆盖由此带来的传输与管理开销。结合上一小节可知，只有当热度追踪足够准确、显存驱逐足够合理、主存准入足够克制时，Prefill 阶段的显存–主存卸载闭环才能稳定地产生正收益。

3.5 本章小结

本章围绕 Prefill 阶段共享前缀 KV Cache 在显存受限条件下的卸载与准入问题，构建了热度追踪–显存驱逐–主存准入的基础管理链路。首先，针对前缀树节点访问模式动态变化且精确统计代价较高的问题，设计了基于增强型布谷鸟过滤器的热度记录结构，并通过复用触发更新 + 周期老化更新实现低开销的在线热度维护；其次，针对显存受限场景下如何尽量保留高价值共享前缀的问题，提出仅在 GPU 叶子节点集合上执行的热度感知驱逐策略；进一步地，针对简单全量卸载容易带来高 I/O 开销和主存拥塞的问题，提出了基于频次阈值与热度比较的选择性准入机制。

实验结果表明，本文方法能够在较小命中率代价下显著降低 GPU–CPU 传输时间，并有效改善端到端执行时延。这说明，只有先对共享前缀缓存完成有效筛选，后续的异构内存协同才具有稳定收益。因而，本章工作为第四章进一步展开的主存–显存预取与执行优化提供了高质量候选缓存集合和统一的热度基础。

第四章　面向推理 Prefill 阶段的 KV Cache 主存-显存预取与执行优化策略

在上一章解决显存驱逐与主存准入之后，本章进一步关注如何将保留在主机内存中的高价值缓存高效回迁到显存，从而有效转化前缀共享带来的性能收益。与第三章侧重回答哪些节点应被保留、哪些节点应被淘汰不同，本章更进一步讨论这些已经被识别为高价值的主存缓存，如何以合适的时机、粒度和执行方式重新进入显存层级。只有当回迁对象选择准确、传输过程与运行时执行节奏充分协调时，主机内存中的缓存保留才不只是容量扩展意义上的“暂存”，而能够真正转化为下一轮 Prefill 的命中收益与端到端性能改进。

4.1　问题背景与分析

上一章解决了共享前缀缓存哪些节点应该离开显存、哪些节点值得留在主存的问题，但这还不足以充分发挥前缀共享的收益。原因在于，保留在主机内存中的高价值 KV Cache 只是潜在可复用状态，只有当它在下一次请求命中前及时回到 GPU 显存，系统才能有效避免对应的重复 Prefill 计算。若系统始终等到新请求命中时再从主存按需加载缓存，则很可能面临 CPU 到 GPU 的传输时间大于前缀重算时间的问题；尤其在 PCIe 带宽有限、前缀长度较长、请求到达密集的条件下，简单的按需加载会显著增加后续请求时延，并使主存保留机制的理论收益受到削弱[31,42-43]。

因此，Prefill 阶段的第二个核心问题变成了应当预取什么、何时预取以及如何预取。与上一章的驱逐和准入相比，预取是一种更主动的操作：系统需要在实际命中发生之前，就判断哪些主存缓存值得回迁到显存。这意味着，预取决策不仅要比较不同存储层中的节点价值，还要额外考虑三个成本来源。第一，**显存成本**：预取会占用本就有限的显存空间，若对象选择失误，系统可能把显存再次填满低价值缓存，反而挤出更高热度的节点。第二，**带宽成本**：主存到显存的数据

迁移会消耗 PCIe/NVLink 带宽，若对象过多或粒度过碎，将明显拉高传输开销。第三，**时机成本**：若预取与当前批次的关键计算阶段发生资源竞争，还会引入新的尾部时延。

此外，预取问题还具有显著的错误代价不对称特征。若系统漏掉一个本该预取的高价值节点，后果通常是后续请求重新执行一次较长的 Prefill；而若系统错误预取了一个短期内不会再命中的节点，则不仅浪费了一次传输机会，还可能挤出显存中原本更有价值的热点前缀。因此，预取策略不能简单理解为越积极越好，而应当在命中收益和执行代价之间取得平衡。也正因为如此，第四章更强调以热度比较和执行优化共同构成闭环，而不是仅在回迁端单独提升传输速度。

从前缀树结构进一步分析，预取问题本质上是一个跨层级的冷热置换问题。主存中可被回迁的节点数通常远大于显存可容纳的数量，因此系统不可能把所有主存缓存都重新拉回显存，而必须在主存热点和显存冷点之间建立匹配关系。与上一章主要关注谁该被驱逐不同，本章更关注哪些主存节点值得提前提升、哪些显存节点应为其腾出位置。这里不仅涉及节点热度，还涉及祖先链一致性、节点大小、树深度和共享范围等因素：若一个主存节点足够热但其祖先链不完整，则单独提升并不可执行；若一个节点虽然很热但尺寸过大，则其回迁收益也可能被传输成本侵蚀。

另一方面，预取执行并不是简单的数据传输问题，而需要结合 Prefill 与 Decode 两阶段的执行差异进行设计。前缀缓存的复用主要发生在 Prefill 阶段，而 Decode 阶段新生成内容的共享概率很低；这意味着，系统可以在当前批次完成 Prefill、进入 Decode 阶段后，将下一轮更可能命中的前缀缓存从主存异步回迁到显存，并利用 Decode 阶段对这些缓存依赖较弱的特点掩蔽迁移开销[9-11]。换言之，Decode 阶段为主存–显存回迁提供了一个适合进行流水重叠的执行窗口。

基于上述分析，本章将问题进一步细化为两个相互衔接的部分。首先，是**预取决策问题**：如何在主存热点与显存冷点之间建立合理映射，使每次回迁都尽量带来更高的未来命中收益。其次，是**预取执行问题**：如何将已经确定的回迁计划以低干扰、低碎片、低额外时延的方式落实到硬件链路上。只有当预取对象选择准确、执行过程高效时，主存中的高价值 KV Cache 才能有效转化为下一批次 Prefill 的命中收益。基于这一观察，本章进一步提出面向异构内存协同的热度预取与执行优化方法。

从系统层面看，第四章实际上承担着将第三章静态保留下来的高价值节点转化为动态参与下一轮 Prefill 命中的显存布局这一任务。第三章回答的是哪些节点不应被轻易丢弃，而第四章回答的是哪些节点应在何时被提前回迁。只有将这两步统一纳入同一执行链路，异构内存才不再只是容量扩展手段，而会成为实际服务于在线推理性能的主动缓存层级。

4.2 基于异构内存热度对比的 KV Cache 预取决策

在明确预取问题的本质之后，本节首先聚焦预取什么这一决策层问题。核心思路是基于前缀树结构和统一热度记录，在主存热点与显存冷点之间建立可执行的替换映射，使回迁行为尽量服务于下一轮 Prefill 命中，而不是把有限显存再次浪费在低价值节点上。

4.2.1 异构内存间冷热 KV Cache 的识别与匹配方法

主存中可被预取的缓存数量通常远大于显存可容纳的数量，因此预取过程的第一步是建立主存热点与显存冷点之间的匹配关系。本文提出的基本原则是：优先将主存中高热度、浅深度的节点提升到显存，并使用显存中低热度、深层的节点为其腾出空间。

为了实现这一点，系统分别在两类节点上构建候选集合。其一，是主机内存中的根节点集合，即父节点位于显存、而节点本身位于主存的那些节点。这类节点可理解为主存中最靠近显存边界的热点入口，它们一旦被提升回来，就能够恢复一整段共享前缀的高效复用。其二，是 GPU 显存中的叶子节点集合，即没有子节点，或其所有后代都已位于主存中的节点。这些节点通常处在当前显存缓存树的边缘位置，若其热度又较低，则适合作为被替换对象。

上述划分与前缀树结构天然匹配。祖先节点的热度通常高于后代节点，因此从主存根节点中挑选热点节点，相当于优先恢复更有可能被多个未来请求共享的公共前缀；从 GPU 叶节点中挑选冷点节点，则能够以最小破坏代价为更高价值前缀让位。与简单地按照最近访问时间扫描整棵前缀树相比，这种主存根节点对 GPU 叶节点的比较方式既缩小了决策空间，也更符合前缀共享的语义结构。

在热度比较时，系统继续复用上一章建立的热度追踪记录。对于主存候选节

点，系统按照热度从高到低排序；对于显存候选节点，则按热度从低到高排序。随后，系统依次尝试让主存中更热的节点替换显存中更冷的节点。若某个主存节点所需的 token 空间小于等于某个显存冷节点所占空间，则可以直接一对一替换；若主存热点节点更大，则允许聚合多个显存冷节点共同让出空间，只要这些被腾出的空间之和足以容纳该主存节点即可。

这里还需处理一个特殊约束：若某个主存候选节点的父节点已经被标记为本轮驱逐对象，那么该主存节点不应再被单独提升，因为其祖先链会在提升过程中被破坏。本文方法在生成预取计划时显式检查这一一致性条件，从而避免产生逻辑上不可执行的置换方案。图 4-1 展示了基于热度的预取策略从选点、比较到生成替换计划的全过程。

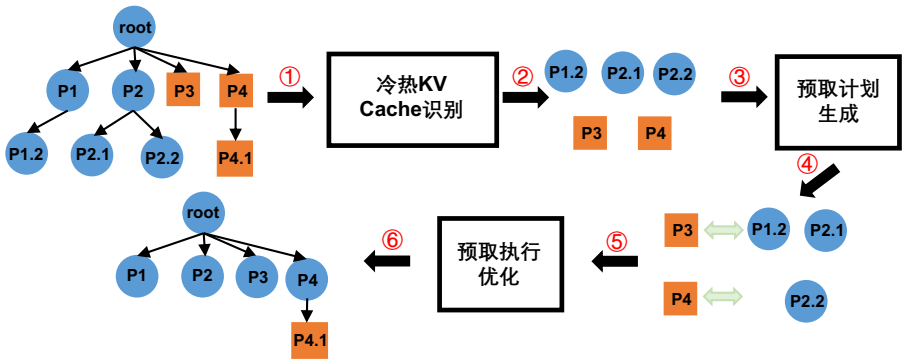


图 4-1　基于 KV Cache 节点热度的预取过程

图 4-1 给出了热度预取的完整决策过程。该过程在当前批次 Prefill 阶段结束之后启动：系统首先从主机内存中选出热度最高的一组根节点候选，再从 GPU 显存中选出热度最低的一组叶节点候选；随后，在满足容量约束与祖先链一致性约束的前提下，逐步构造“主存热点节点 → 显存冷节点集合”的替换映射。图中所表达的关键思想在于，预取并非简单地把主存中的热点缓存直接搬回显存，而是先比较主存热点与显存冷点的相对价值，并据此形成可执行的提升计划，以确保显存始终优先保留未来更可能被复用的高价值前缀。

由此，系统最终得到一组主存热点节点 → 显存冷节点集合的预取映射。与上一章的驱逐决策相比，这里更强调跨层比较：不仅要判断某个节点是否值得保留，还要判断它是否值得提前回迁到显存。

4.2.2 基于热度的 KV Cache 预取机制

在得到候选映射后，系统进入预取执行阶段，其目标不是简单的数据拷贝，而是利用热度信息周期性刷新 GPU 显存中的前缀缓存布局，使显存始终维持对未来复用更有利的状态。

具体而言，预取在每个批次 Prefill 阶段完成后触发。系统首先扫描主存中的热点根节点和显存中的冷叶节点，基于上一小节的排序与匹配规则生成可执行预取计划。随后，系统把计划中选中的显存冷节点标记为待淘汰对象，并将对应主存节点加入回迁列表。由于该计划在生成时已经保证替换前后的总 token 空间不超过既有显存预算，因此理论上不会额外增加显存峰值占用；若后续 Decode 批次本身需要更多显存空间，系统再配合第三章的热度感知驱逐策略进行补充回收。

从策略直觉看，基于热度的预取策略主要利用了三条经验。第一，浅层前缀的共享概率通常高于深层前缀，因此在热度接近时，系统更倾向于预取浅层节点。第二，超长节点会占用更多显存，只有当其热度足够高时才值得被整体回迁；否则系统宁可保留多个较短但累计收益更高的节点。第三，预取对象必须能在未来较短时间内命中，否则就只是把卸载成本提前到了当前时刻。热度排序正是对这种近期可能复用进行近似建模的手段。

预取计划生成算法流程 为将主存热点与显存冷点的比较过程转化为可执行计划，本文进一步给出预取计划生成流程，并将其形式化为算法 4-1。

算法 4-1刻画了本文预取机制中热度比较–计划生成–异步回迁的衔接过程：系统先根据热度排序确定主存热点与显存冷点的优先级，再在容量约束与祖先链一致性约束下构造可执行映射，最终为后续异步回迁提供计划输入。

4.3 基于流水线与批量传输的 KV Cache 预取执行优化

仅有合理的预取对象还不足以保证异构内存协同真正产生收益，系统还必须回答怎样搬运才能尽量不影响当前执行。因此，本节进一步从执行层面对预取过程展开优化，重点讨论如何利用 Prefill–Decode 阶段差异进行流水重叠，以及如何通过批量传输提高链路带宽利用率、降低细粒度搬运开销。

算法 4-1 预取计划生成流程

输入： 主存候选根节点集合 H，显存候选叶节点集合 C
输出： 预取计划 P

```
按热度从高到低排序 H，按热度从低到高排序 C
初始化 P ← ∅
for each h ∈ H do
  if parent(h) 已被标记为本轮待替换对象 then continue
  初始化候选替换集合 V ← ∅，累计可释放空间 S ← 0
  for each 尚未分配的 c ∈ C do
    if hotness(h) ⩽ hotness(c) then break
    将 c 加入 V，并更新 S ← S + size(c)
    if S ⩾ size(h) then
      将映射 h → V 写入 P
      将 V 中节点标记为已分配
      break
    end if
  end for
end for
return P
```

4.3.1 基于 Prefill-Decode 特性的流水线执行方法

仅有正确的预取对象还不够，系统还必须回答何时执行回迁最不影响当前批次。本文选择在当前批次 Prefill 结束之后、Decode 持续执行期间启动预取，并使用独立 CUDA Stream 将主存到显存的数据迁移与 Decode 计算并发执行[9-10]。图 4-2 给出了这一流水线设计：当前批次 Decode 持续推进时，下一批更可能复用的前缀缓存正沿 PCIe 链路异步回迁。

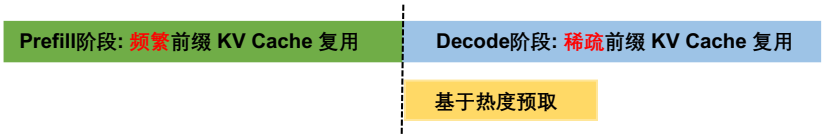


图 4-2　Decode 阶段与 KV Cache 节点热度提升过程的流水线重叠

图 4-2 展示了本文将热度提升过程与 Decode 阶段进行流水重叠的执行方式。其基本思路是：在当前批次完成 Prefill 之后，立即依据上一节生成的提升计划启动主存到显存的数据迁移，并将该迁移过程与当前批次的 Decode 计算异步并发。之所以能够这样安排，是因为前缀共享收益主要集中在 Prefill 阶段，而 Decode 阶段生成的新内容通常难以被后续请求直接复用，因此当前批次 Decode 对下一批次共享前缀缓存的回迁并不敏感。基于这一阶段差异，本文得以将主存–显存

传输开销尽量隐藏在 Decode 计算之后，从而避免其直接暴露到端到端时延上。

在流水线执行时，系统首先根据上一节生成的预取计划，将计划中待替换的 GPU 冷节点标记为不可再参与后续共享命中；随后启动异步数据迁移，把主存中的热点节点逐步拷入显存。与此同时，GPU 计算流继续推进当前批次的 Decode。只要 Decode 的计算时间足够覆盖大部分传输时间，回迁成本就能被隐藏在现有计算后面，而不会显著体现在端到端时延中。

这种流水线机制当然也带来一个代价：被标记为待替换的显存冷节点在预取执行期间暂时不可用。如果这类节点恰好被再次命中，理论上会造成一次额外缺失。但这些节点本就属于低热度候选，其短期内被再次访问的概率非常低；与之相比，把主存中的热点节点提前回迁所带来的未来命中增益更大，因此整体收益仍为正。实验结果也印证了这一点，即使考虑这一阶段性不可用成本，端到端时延仍显著下降。

4.3.2 面向传输带宽优化的批量数据传输方法

除了重叠时机，预取效率还与传输粒度密切相关。若系统为每个待回迁节点单独发起一次主存到显存的拷贝调用，PCIe 传输将受到频繁启动、同步和地址管理开销影响，难以充分利用总线带宽。为此，本文在执行层面进一步采用批量传输策略：在保证数据布局可解析的前提下，将多个待预取节点的 KV Cache 组织为更大的连续传输批次，以减少 memcpy 调用次数和碎片化传输开销[42-43]。

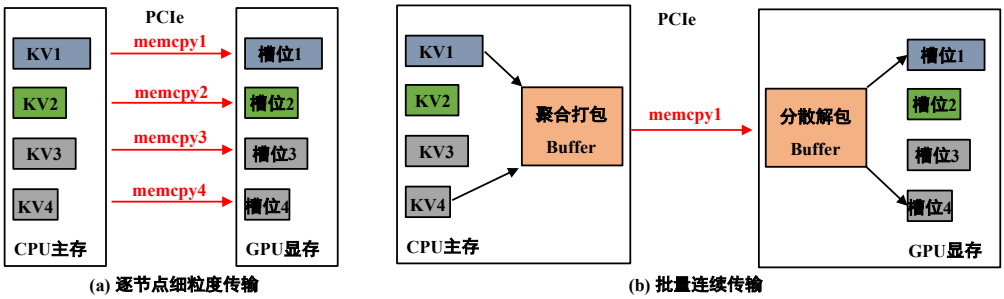


图 4-3 逐节点细粒度传输与批量连续传输的对比示意

图 4-3 展示了逐节点细粒度回迁与批量连续传输两种执行方式的差异：前者需要频繁发起小粒度搬运请求，而后者则将多个待回迁节点打包为更大的连续传输批次。

批量传输的收益主要体现在两个方面。第一，更少的传输调用意味着更低的

软件调度开销，尤其是在主存和显存之间需要频繁切换大量小块数据时，这一收益尤为明显。第二，较大的连续传输更容易接近链路峰值带宽，从而降低单位字节的迁移成本。对于共享前缀缓存而言，许多热点节点本就具有相近的生命周期和回迁时机，因此具备天然的批量组织条件。

在实现中，本系统将批量传输与流水线执行结合起来：先利用热度比较生成预取计划，再将计划中可一起搬运的节点整合为批量传输请求，最后在 Decode 期间通过独立 CUDA Stream 异步提交。由此，系统在不改变高层预取逻辑的前提下，同时优化了硬件带宽利用率和主机侧调度成本。

4.4 实验评估与分析

为验证本章提出的热度预取与执行优化是否能够把主存中的高价值缓存有效转化为下一批次的命中收益，本节从预取决策、执行优化和端到端收益三个层面展开实验分析，并进一步考察该方法在多卡和异构 GPU 平台上的扩展性与鲁棒性。

4.4.1 实验环境与评价指标

本章实验与第三章保持一致，使用前缀树统一管理 GPU 显存与主机内存中的 KV Cache，并以热度记录作为预取决策依据。为了把第四章的收益尽可能归因于回迁决策与执行优化，实验继续复用第三章已经实现的热度追踪和选择性准入机制，仅在此基础上替换预取判据、是否启用流水线重叠以及是否进行批量传输。

单卡主实验使用 LLaMA-2 13B 与 Gemma 9B[16,18]，硬件平台为单张 NVIDIA A6000。其中，LLaMA-2 13B 的主结果覆盖 MMLU、BBH、C-EVAL 和 HellaSwag 四类工作负载，脚本中对应批大小分别设为 64、64、32 和 32，用以匹配不同负载下显存占用和请求长度分布差异；Gemma 9B 用于验证收益并非只在单一模型上成立。对于单卡端到端对比，本文统一与 SGLang-LRU、SGLang-LFU、SGLang-FIFO、SGLang-2Q 以及 vLLM 进行比较，以覆盖典型的时间、频率、队列型替换以及现有推理系统基线。

多卡扩展实验使用 4 张 A6000，其中 Qwen-2 72B 用于张量并行测试，重点

观察模型并行场景下预取收益是否仍能保留；LLaMA-2 13B 用于数据并行测试，分别比较 DP1、DP2 和 DP4 三种并行规模，以考察本文方法是否会破坏数据并行下的吞吐扩展。跨硬件鲁棒性实验则使用 LLaMA-3 8B，分别部署于 RTX3090、A30、A6000 和 A100，以覆盖消费级卡、主流服务器卡和高端加速卡三类平台。

对比方法 本章同样使用两类对比方法。其一是**预取决策消融**，分别比较基于热度的预取、仅基于时间的预取、仅基于频率的预取以及仅基于深度的预取，用于验证主存热点与显存冷点之间究竟应依据哪类信号建立回迁映射。其二是**端到端系统对比**，统一比较本文方法、SGLang-LRU、SGLang-LFU、SGLang-FIFO、SGLang-2Q 与 vLLM。其中，SGLang-LRU/LFU/FIFO/2Q 表示在 SGLang 执行框架下分别采用不同缓存管理与回迁策略的系统版本，vLLM 则作为不依赖前缀树异构内存协同的主流 serving 基线。对于张量并行实验，主要与 SGLang 和 vLLM 比较，以观察在模型并行通信存在时预取收益是否仍然成立；对于数据并行实验，则比较 DP1、DP2、DP4 三种并行规模下本文方法自身的扩展趋势。

评价指标 本章的评价指标与第三章一致，采用缓存命中率、端到端时延、吞吐率和异构内存传输时间四项指标。其中，吞吐率统一以每秒处理的 token 数量衡量，并在图中以 kt/s 形式展示。对比方法之间统一复用相同工作负载顺序和同一套执行框架，从而保证不同图中的差异主要来自预取决策与执行层优化本身。

4.4.2 预取决策策略有效性验证

首先考察预取决策本身的有效性。消融实验以 MMLU、C-EVAL 和 HellaSwag 为代表工作负载，对比了热度预取、基于时间的预取、基于频率的预取以及基于深度的预取。结果表明，本文方法在三类工作负载上均取得最高缓存命中率和最低端到端时延，而且不同负载上的收益模式具有较强一致性。

从具体数据看，MMLU 上热度预取的命中率达到 0.8089，而时间、频率和深度分别只有 0.5913、0.7619 和 0.7393；对应时延分别为 556.07 s、1234.12 s、666.32 s 和 765.87 s。也就是说，单靠时间会把很多刚被访问但未必值得长期保留的节点提早拉回显存，导致时延恶化到本文方法的 2.22 倍；频率和深度虽然比时间更稳定，但仍不能同时兼顾短期活跃度与长期复用价值。

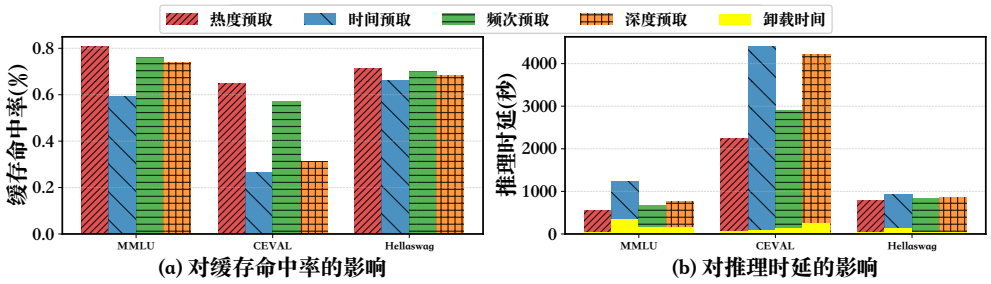


图 4-4　不同预取标准对缓存命中率和推理时延的影响，黄色部分表示 GPU 与 CPU 之间的卸载时间

在 C-EVAL 上，这一差异被进一步放大。本文方法的命中率为 0.6495，而时间仅为 0.2631，频率为 0.5708，深度为 0.3106；对应总时延分别为 2241.64 s、4408.19 s、2909.84 s 和 4210.70 s。尤其相较时间，本文方法在命中率上提升约 2.47 倍，在总时延上降低约 1.97 倍。造成这一现象的原因在于，C-EVAL 中大量前缀虽然并不呈现连续、密集的短期重复，但一旦被后续样本复用，其前缀长度和重算代价都很高，因此仅看最近有没有被访问过显然不足以支撑预取判断。

HellaSwag 上的差异相对温和，但趋势依旧清晰。本文方法的命中率为 0.7144，高于时间的 0.6629、频率的 0.7005 和深度的 0.6831；总时延为 789.46 s，低于时间的 930.93 s、频率的 824.57 s 和深度的 867.45 s；同时其卸载时间仅为 27.04 s，而时间、频率和深度分别为 129.48 s、41.73 s 和 36.17 s。这表明即便在中等长度负载上，只要主动预取对象选得足够准，系统仍然可以在不放大搬运开销的前提下稳定获得收益。

这一结果具有两层含义。其一，仅依据最近访问时间容易偏向短期活跃但总体价值不高的节点，仅依据频次又容易保留历史热点而忽略当前负载迁移趋势，仅依据深度则忽略了真实访问差异，三者都无法单独承担预取判别标准。其二，预取比驱逐更依赖未来复用概率的近似估计，因为预取本身是主动行为，一旦误判，就会把额外搬运成本直接暴露在端到端时延上。联合热度因此比单一指标更适合用于主存–显存回迁。

4.4.3　端到端性能收益分析

图 4-5 给出了 LLaMA-2 13B 在 MMLU、HellaSwag、BBH 和 C-EVAL 上的 GPU 缓存命中率对比。整体上看，本文方法在四类负载上都取得了最高命中率，而且对共享前缀越长、复用模式越复杂的工作负载，命中率优势越明显。

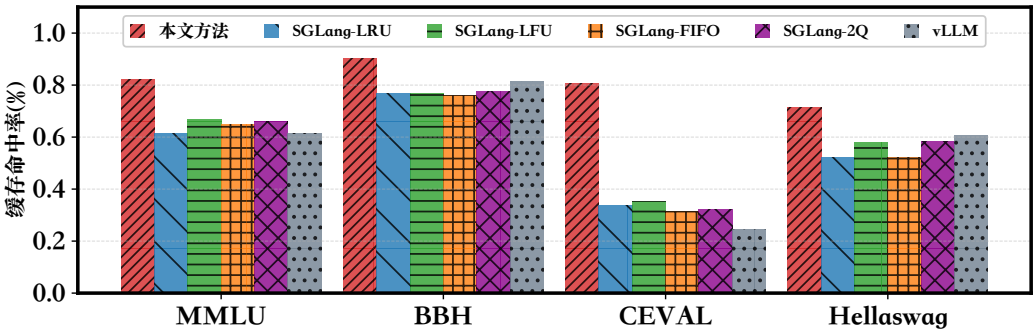


图 4-5　LLaMA-2 13B 下不同工作负载的 GPU 缓存命中率对比

从具体数据看，MMLU 上本文方法的命中率达到 0.8196，而 SGLang-LRU、SGLang-LFU、SGLang-FIFO、SGLang-2Q 和 vLLM 分别为 0.6127、0.6668、0.6494、0.6592 和 0.6125；BBH 上本文方法的命中率达到 0.9034，也明显高于各对比方法的 0.7610–0.8122 区间。更值得注意的是 C-EVAL：本文方法的命中率达到 0.8054，而 SGLang-LRU 仅为 0.3381，vLLM 更只有 0.2463，分别只有本文方法的约 42.0% 和 30.6%。这说明当前缀足够长、请求之间的共享结构更细碎时，传统基于时间的缓存保留或粗粒度共享机制很难稳定抓住高价值的前缀节点。

图 4-6 所示的端到端时延与吞吐结果进一步说明，命中率优势已经被稳定转化为系统级收益。MMLU 上，本文方法时延为 617.60 s，低于 SGLang-LRU 的 955.76 s 和 vLLM 的 1157.35 s；按图中吞吐口径换算后，吞吐率分别约为 15.08 kt/s、9.74 kt/s 和 8.04 kt/s。BBH 上，本文方法时延仅为 239.51 s，相比 SGLang-LRU 的 419.76 s 降低约 1.75 倍，吞吐率则从 14.19 kt/s 提升到 24.85 kt/s。HellaSwag 上，本文方法时延为 789.46 s，优于 SGLang-LRU 的 1304.03 s、SGLang-FIFO 的 1300.93 s 和 vLLM 的 1215.45 s，对应吞吐率达到约 11.83 kt/s，也高于各基线的 7.16–8.41 kt/s。

在 C-EVAL 上，端到端收益最为显著。本文方法将时延控制在 1982.94 s，而 SGLang-LRU 为 3958.89 s，vLLM 为 4454.86 s，分别是本文方法的约 2.00 倍和 2.25 倍；换算后的吞吐率约为 8.88 kt/s，而 SGLang-LRU 和 vLLM 分别只有约 4.45 kt/s 与 3.95 kt/s。这里的关键并不只是本文方法缓存更多，而是它能够把真正值得缓存、值得回迁的长前缀持续保留在更靠近 GPU 的层级中，因此避免了大量重复 Prefill 计算。

为了更直观地说明额外回迁开销为何没有抵消收益，表 4-2 列出了本文方

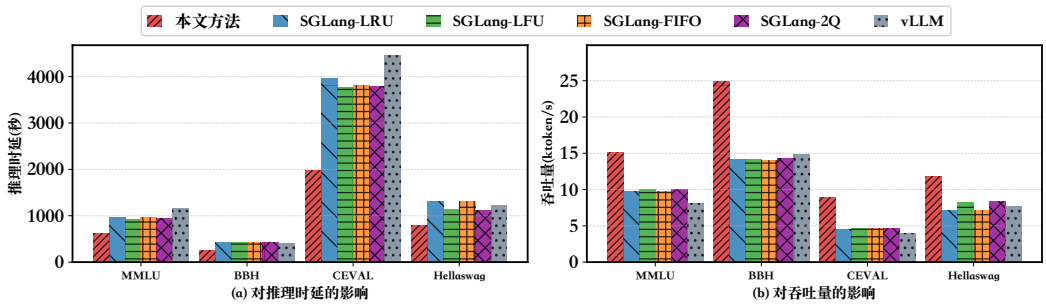


图 4-6　LLaMA-2 13B 下不同工作负载的端到端时延与吞吐对比

法在四类工作负载上的总异构内存传输时间。可以看到，尽管系统确实引入了 24.80–68.28 s 的额外搬运开销，但这些开销只占其他方法平均总时延的 1.1%–5.9%。例如，在 C-EVAL 上虽然总传输时间达到 68.28 s，是四个负载里最高的，但与 1982.94 s 的总执行时间相比仍然只是较小组成部分，而它换来的是对 vLLM 约 2.25 倍的时延改进和约 2.25 倍的吞吐提升。这说明只要预取对象选得足够准确，并且回迁过程能与 Decode 阶段有效重叠，异构内存带来的增量成本就会远小于重复 Prefill 计算造成的损失。

表 4-2　在不同工作负载上的异构内存传输时间

数据集	MMLU	BBH	C-EVAL	HellaSwag
传输时间/s	25.66	24.80	68.28	27.03

从图 4-5、图 4-6 和表 4-2 结合来看，本文方法的性能收益具有清晰的因果链条：热度驱动的多层缓存管理先提升 GPU 命中率，再通过预取流水线把额外回迁代价压缩到很低比例，最终转化为端到端时延下降与吞吐率提升。尤其在 C-EVAL 这类长前缀、复杂访问模式负载上，命中率、时延和吞吐三项指标都呈现出最强改善，说明热度筛选准确与执行层回迁高效这两个条件缺一不可。

在扩展性方面，本文方法在 4 张 A6000 上运行 Qwen-2 72B 的张量并行实验时，MMLU、BBH 和 HellaSwag 上的命中率分别达到 0.8205、0.9124 和 0.7337，而 SGLang 仅为 0.6123、0.7772 和 0.5761；对应时延分别为 1303.26 s、405.98 s 和 1772.08 s，相对 SGLang 分别降低约 2.00 倍、2.42 倍和 1.57 倍，相对 vLLM 分别降低约 2.15 倍、2.61 倍和 1.60 倍。这表明即使在张量并行下，缓存管理与回迁策略仍能带来稳定收益，而不是被模型并行通信开销完全掩盖。

在 LLaMA-2 13B 的数据并行实验中，随着并行卡数从 1 增加到 2 和 4，MMLU 上时延从 556.07 s 下降到 352.34 s 和 257.03 s，C-EVAL 上从 2241.64

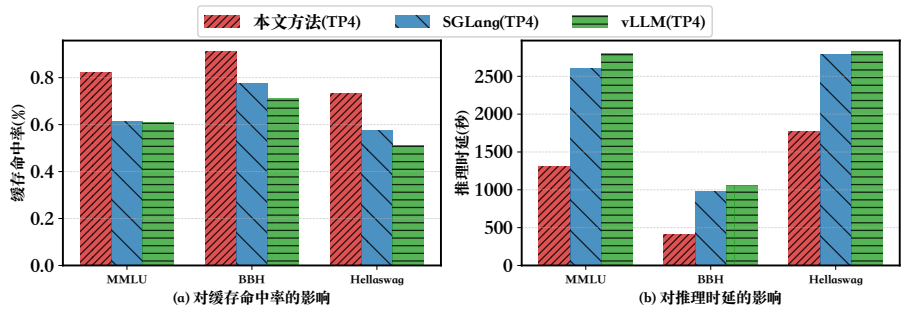


图 4-7　4 卡张量并行下的命中率与时延

s 下降到 1204.59 s 和 834.72 s，HellaSwag 上从 789.46 s 下降到 451.07 s 和 250.35 s；对应吞吐率也分别从 15.07 kt/s 提升到 23.79 kt/s 和 32.61 kt/s、从 8.88 kt/s 提升到 16.53 kt/s 和 23.85 kt/s、从 11.83 kt/s 提升到 20.70 kt/s 和 37.30 kt/s。需要指出的是，随着数据并行度增加，单卡命中率会从 0.8196 下降到 0.7253 这一级别，这是因为请求被分散到不同副本后，每张卡局部可观察到的共享模式减弱；但总体时延依然明显下降，说明本文方法并没有破坏数据并行的扩展性，反而能够在更大总缓存预算下继续受益。

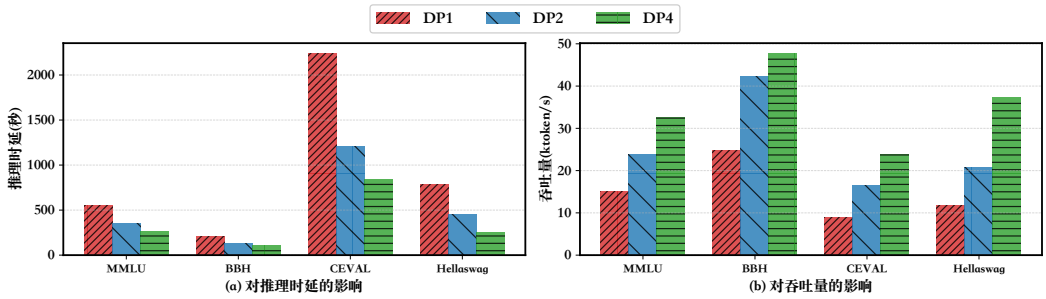


图 4-8　LLaMA-2 13B 数据并行配置的时延对比

在跨硬件鲁棒性方面，使用相同 KV Cache 容量配置时，本文方法在 RTX 3090、A30、A6000 和 A100 上均优于对比方法。例如在 A30 上，本文方法时延为 294.60 s，而 SGLang-LRU、SGLang-FIFO 和 vLLM 分别为 451.82 s、550.67 s 和 661.52 s；在 A100 上，本文方法时延进一步降至 143.70 s，相比 SGLang-LRU 的 301.83 s 和 vLLM 的 325.55 s 分别快约 2.10 倍和 2.27 倍。即便在相对较弱的 RTX 3090 上，本文方法也仍把时延控制在 488.37 s，优于 LRU 的 742.68 s 和 vLLM 的 781.01 s。说明该方法并不依赖某一特定型号 GPU 的峰值带宽或超大显存，而是在不同硬件平台上都能稳定把减少重复 Prefill 计算转化为真实收益。

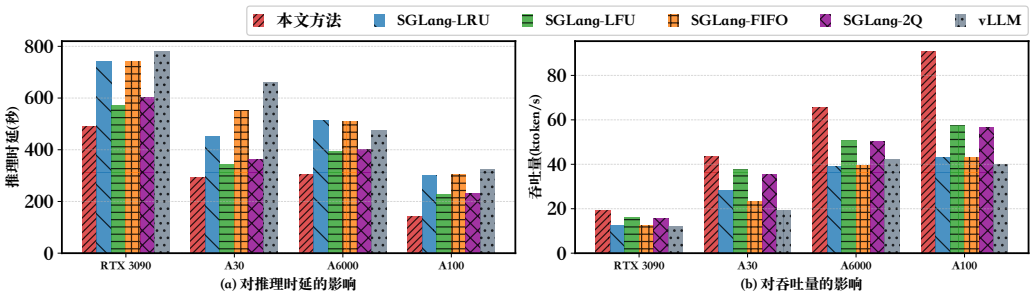


图 4-9　LLaMA-3 8B 在不同 GPU 平台上的时延与吞吐对比

4.4.4　消融实验

本文在整体组件消融中比较了四种配置：无优化基线、仅使用热度驱逐、仅使用选择性准入与热度预取，以及同时启用热度驱逐、选择性准入和热度预取的完整方案。图 4-10 表明，完整方案的收益并不是某一个组件单独拉高的，而是多个环节逐级叠加形成的。

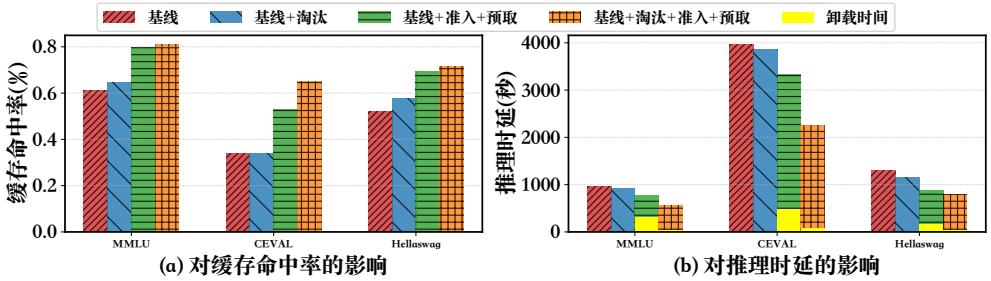


图 4-10　各优化组件的联合消融结果，黄色部分表示 GPU 与 CPU 之间的卸载时间

以 MMLU 为例，基线配置的命中率仅为 0.6127，总时延为 955.76 s；仅加入热度驱逐后，命中率只提升到 0.6468、时延仅降到 921.10 s，说明如果没有后续准入和预取，单独减少错误驱逐带来的收益仍然有限。加入选择性准入与热度预取后，命中率进一步升至 0.7973，时延降至 762.30 s；而完整方案最终达到 0.8089 和 556.07 s，说明驱逐、准入、预取与执行层优化之间存在明显协同，而不是简单线性叠加。

在 C-EVAL 上，这种协同关系更清楚。基线命中率只有 0.3381，总时延高达 3958.89 s；仅加热度驱逐后，命中率仍只有 0.3407，时延也只是轻微下降到 3868.38 s。真正明显的转折出现在启用选择性准入与热度预取之后：命中率提升到 0.5278，总时延下降到 3326.97 s；而完整方案进一步把命中率提升到 0.6495，并把总时延压到 2241.64 s。也就是说，对于超长前缀负载，仅有把坏缓存丢掉

远远不够，还必须配合把好缓存留下来并及时拉回来，系统才会出现显著收益。

HellaSwag 上也呈现出相同趋势。基线、仅驱逐、准入 + 预取、完整方案的命中率分别为 0.5206、0.5768、0.6922 和 0.7144，对应时延分别为 1304.03 s、1148.19 s、877.53 s 和 789.46 s。可以看到，仅驱逐和仅预取都能带来改善，但完整方案仍然最好，说明执行层面对传输时机与传输粒度的优化，确实把第三章形成的高价值候选缓存集合转化成了更稳定的命中收益。

这种差异在异构内存传输时间上表现得尤其直观。与仅使用准入与预取的配置相比，完整方案在 MMLU、C-EVAL 和 HellaSwag 上的传输时间分别从 299.74 s、479.98 s 和 160.53 s 降到 25.66 s、68.28 s 和 27.04 s，分别缩短约 11.68 倍、7.03 倍和 5.93 倍。换言之，问题不只是有没有预取，而是预取能否在合适时机以合适粒度完成。流水线重叠把回迁尽量隐藏在当前批次 Decode 之后，批量传输则减少了大量小块数据搬运带来的软件调度和带宽碎片损失，两者共同把本来可能十分昂贵的回迁过程压缩到了可接受范围内。

从整体工作负载看，HotPrefix 在 MMLU、BBH、C-EVAL 与 HellaSwag 上的总异构内存传输开销分别约为 25.66 s、24.80 s、68.28 s 和 27.03 s。这些开销仅相当于其他方法平均总时延的 2.6%、5.9%、1.1% 和 2.2%，但最终却换来了约 1.28 倍、1.16 倍、2.60 倍和 1.27 倍的平均时延收益。尤其在输入更长、负载更复杂的 C-EVAL 上，额外搬运开销虽然绝对值最大，但相对时延占比反而最低，这恰恰说明流水线与批量传输确实成功隐藏了回迁成本。

4.5 本章小结

本章在第三章卸载与准入策略的基础上，进一步研究了主机内存到 GPU 显存的 KV Cache 预取与执行优化问题，形成了 Prefill 阶段异构内存协同的第二个关键环节。首先，提出基于异构内存热度对比的预取决策机制，通过比较主存热点根节点与显存冷叶节点，生成一致且可执行的回迁计划；其次，结合 Prefill 与 Decode 两阶段执行特性，设计了在 Decode 期间异步执行的流水线预取机制，并通过批量数据传输降低小粒度迁移带来的额外开销。

实验结果表明，热度预取、流水重叠和批量传输能够进一步提升 GPU 缓存命中率，并显著改善端到端时延、吞吐率和跨硬件平台的鲁棒性。至此，第三章

和第四章共同完成了围绕共享前缀缓存的热度评估–驱逐准入–主动回迁–执行优化闭环，也为后续第五章从 Decode 阶段补齐长度感知调度提供了 Prefill 侧的完整支撑。

第五章　面向推理 Decode 阶段的响应长度与 KV Cache 负载感知的多 GPU 请求调度

与前两章主要面向 Prefill 阶段的共享前缀 KV Cache 管理不同，本章转向 Decode 阶段。Decode 阶段的 KV Cache 不再主要表现为可复用前缀的驻留与回迁问题，而是随着每个请求逐词元生成持续增长，并直接决定请求在 GPU 上的后续显存占用和执行时间。因此，本章围绕响应长度预测与 KV Cache 负载感知路由展开方法设计，目标是在生成开始前估计请求未来的 Decode 负载，并据此改进多 GPU 场景下的请求分配。

5.1　问题背景与分析

在大模型在线推理中，Prefill 阶段一次性处理输入提示词，并为输入上下文构建初始 KV Cache；Decode 阶段则在此基础上逐词元生成输出，每生成一个新词元，模型都需要为其追加一组新的键值缓存，并在后续步骤中持续访问历史 KV Cache[2,6]。因此，Decode 阶段的资源消耗与响应长度天然绑定：响应越长，请求保留在系统中的时间越长，后续新增的 KV Cache 越多，对 GPU 显存、显存带宽和调度队列的压力也越大。

这一特性使 Decode 阶段的调度问题不同于传统意义上的请求数均衡。若两个 GPU 当前各自接收了相同数量的请求，但其中一个 GPU 上的请求普遍会生成更长响应，那么两张 GPU 后续承担的真实 Decode 工作量和 KV Cache 增长量仍然会显著不同。对在线服务而言，这种差异会进一步表现为尾部 GPU 长时间被长请求占据，整体完成时间被拖慢，P99 延迟升高，输出吞吐下降。换言之，在 Decode 阶段，仅平衡请求个数并不等价于平衡系统负载；更合理的目标应当是平衡未来一段时间内的 KV Cache 增长压力和逐步解码工作量。

响应长度不确定性正是造成上述问题的根源。对于长上下文问答、复杂数学推理、开放式指令生成和偏好采样等场景，不同请求的输出长度往往呈现显

著异质性：部分请求在几十个词元内即可结束，另一部分请求则可能持续生成数百甚至上千个词元[21,23-24]。在这些负载下，输入长度只能反映初始 KV Cache 的大小，却无法可靠刻画后续 Decode 阶段的增量负载。若调度器无法提前区分长短请求，默认的轮转分发或请求数均衡策略就容易把多个长请求集中到同一张 GPU 上，从而形成 KV Cache 负载倾斜。

从 KV Cache 管理角度看，这种倾斜具有两个直接后果。第一，长请求所在 GPU 的 KV Cache 会持续增长，其可用缓存空间更早受到挤压，后续请求更容易受到容量约束影响。第二，长请求会在 Decode 批次中停留更久，使同一 GPU 上的短请求更难获得稳定的执行节奏，进而放大尾部延迟。与 Prefill 阶段的共享前缀缓存不同，Decode 阶段生成出的 KV Cache 通常不具备跨请求复用价值，但它会长期占用显存并参与每一步注意力计算。因此，本章关注的不是如何复用 Decode KV Cache，而是如何在生成之前利用长度先验对其未来负载进行更合理的规划。

现有响应长度预测方法表明，模型内部表征中确实包含与最终输出长度相关的信号[38,63-64]。然而，已有工作大多把长度预测本身作为独立任务，主要比较预测误差，较少讨论预测结果如何进一步服务于 Decode 阶段的 KV Cache 负载管理。对于系统调度而言，长度预测的意义不在于精确恢复每个请求的最终词元数，而在于提供一个足够稳定的负载先验，使系统能够在请求进入长时间 Decode 前判断其未来会给哪张 GPU 带来多大 KV Cache 增长压力。

基于上述分析，本章需要解决两个相互衔接的问题。第一，**如何从模型 Prefill 阶段产生的内部表征中提取对响应长度更敏感的特征，从而获得可用于系统调度的长度估计**。第二，**如何将预测长度转化为多 GPU 场景下的 KV Cache 负载估计，并据此改进请求路由，减少 GPU 间未来 Decode 负载倾斜**。围绕这两个问题，本章先提出基于层–头表征增强的响应长度预测方法，再提出基于预测 KV Cache 负载的多 GPU 路由调度方法。

5.2 基于层-头表征增强的响应长度预测方法

为了在请求进入 Decode 长时间执行之前估计其未来 KV Cache 负载，本节首先讨论响应长度预测方法。与仅使用最后一层最后词元或最后一层平均池化

的方式不同，本文关注模型内部的层–头级别表征。核心观察是：与响应长度相关的信息并不会均匀分布在所有隐藏状态中，而是集中在少量特定层和注意力头上。本文将这些对长度预测具有更强贡献的层–头单元称为 HotHead，并围绕 HotHead 进行特征选择与场景自适应融合。

5.2.1 基于层-头预测能力扫描的关键表征选择

长度预测的第一步是确定哪些内部表征真正携带了响应长度信息。常见的 Last Token 方法直接取最后一层最后一个词元的隐藏状态，假设该位置能够汇总提示词中的全部生成需求[63-64]；Mean Pool 方法则对最后一层全部提示词词元表征做均值聚合，试图获得更稳定的全局表示[63]；EGTP 进一步利用词元熵对隐藏状态进行加权池化，以强调更不确定、更可能影响后续生成的词元[38]。这些方法实现简单，但它们共同依赖最后一层表示，容易忽略中间层和特定注意力头中已经形成的长度相关结构信号。

为此，本文对候选层–头单元进行逐一扫描。设模型共有 L 层，每层有 H 个注意力头。对于任意候选单元 (l,h)，其中 $l \in \{1,\cdots,L\}$ 表示层编号，$h \in \{1,\cdots,H\}$ 表示该层内的注意力头编号，本文提取第 l 层第 h 个注意力头在提示词词元上的输出表示，并训练一个轻量长度预测器：

$$\hat{y}_{l,h} = f_{l,h}\left(\mathbf{o}^{(l,h)}\right), \tag{5-1}$$

其中 $\mathbf{o}^{(l,h)}$ 表示第 l 层第 h 个注意力头在提示词词元上的聚合表征，$f_{l,h}(\cdot)$ 表示仅使用该层–头表征训练得到的轻量长度预测器，$\hat{y}_{l,h}$ 表示由该层–头单元单独给出的响应长度预测。所有候选注意力头使用相同训练配置，并在统一验证集上计算平均绝对误差（Mean Absolute Error, MAE），即预测响应长度与真实响应长度之间绝对差值的平均值，数值越低表示预测越准确。随后，本文将验证误差归一化为层–头预测能力得分：

$$\rho_{l,h} = 1 - \frac{e_{l,h} - e_{\min}}{e_{\max} - e_{\min}}, \tag{5-2}$$

其中 $e_{l,h}$ 表示候选层–头单元 (l,h) 的验证集 MAE，$e_{\min}$ 表示同一模型、同一场

景下所有候选层–头单元中的最小 MAE，$e_{\max}$ 表示对应的最大 MAE，$\rho_{l,h}$ 表示归一化后的预测能力得分。$\rho_{l,h}$ 越大，说明该层–头单元的验证误差越低，越适合作为长度预测特征来源。

图 5-1 给出了 Llama3.2-1B 和 Qwen2.5-3B 上的层–头预测能力热力图。每个子图内部按列给出三类 ForeLen 场景：LongSeq 表示长上下文场景，用于刻画长输入和大长度跨度下的预测能力；Reasoning 表示数学推理场景，用于刻画推理复杂度和题目约束对输出长度的影响；RL 表示开放式指令和偏好采样场景，用于刻画自由生成负载中的响应长度波动。横轴表示注意力头编号，纵轴表示模型层编号，颜色表示归一化后的长度预测能力得分，颜色越接近黄色表示该层–头单元的验证集 MAE 越低、预测能力越强。

从图中可以看到，两类模型中高分区域都不是整层连续分布，而是呈现出较明显的稀疏热点形态；同时，同一模型在 LongSeq、Reasoning 和 RL 三类场景下的热点位置也并不完全一致。这说明响应长度线索既不是某一层的统一属性，也不是跨场景固定不变的结构，而是由层位置、注意力头功能和任务场景共同决定。换言之，若仅使用最后一层表示，或者固定选取少数经验层号，容易错过真正对长度预测有贡献的中间注意力头；若在所有场景中使用完全相同的层–头组合，也会忽略不同负载类型下长度信号位置的差异。因此，后续方法需要同时考虑层–头选择和场景自适应特征组合。

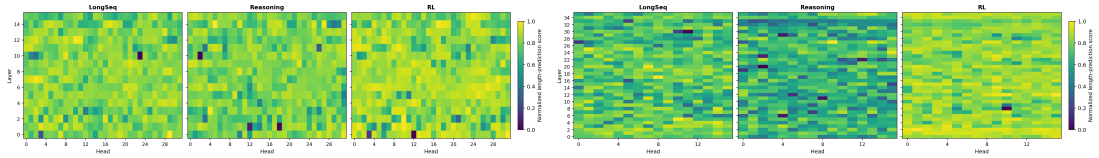


图 5-1　层-头长度预测能力热力图。左图为 Llama3.2-1B，右图为 Qwen2.5-3B；每个模型内的 LongSeq、Reasoning 和 RL 分别表示长上下文、数学推理、开放式指令与偏好采样三类场景；横轴为注意力头编号，纵轴为模型层编号，颜色越亮表示该层–头单元的长度预测能力越强。

基于这一观察，本文不再预设固定层号，而是根据验证集扫描结果为每个模型和场景选择 Top-K HotHead，其中 K 表示最终保留的关键层–头数量。设选中的 HotHead 集合为 $\mathcal{H}^*$，则对应的长度预测表征可写为

$$\mathbf{z}_{\text{hh}} = \text{Pool}\left(\left\{\mathbf{o}^{(l,h)} \mid (l,h) \in \mathcal{H}^*\right\}\right), \tag{5-3}$$

其中 $\mathcal{H}^*$ 表示由验证集预测能力排序得到的 Top-K 层–头集合，$(l,h) \in \mathcal{H}^*$ 表示第 l 层第 h 个注意力头被选为 HotHead，$\mathbf{o}^{(l,h)}$ 表示该注意力头的聚合表征，Pool$(\cdot)$ 表示对多个入选层–头表征进行拼接、加权或池化的聚合过程，$\mathbf{z}_{\text{hh}}$ 表示最终得到的长度预测特征向量。由于 $\mathcal{H}^*$ 来自验证集上的直接预测能力比较，因此该过程本质上是在模型内部筛选更接近响应长度信号的关键表征，而不是简单增加特征维度。

这种层–头扫描机制还具有较好的系统适配性。扫描和预测器训练都在离线阶段完成；在线推理时，系统只需要按照离线选定的长度预测表征配置抽取少量表征，并执行一次轻量回归预测即可得到响应长度估计。在支持 Prefill–Decode 解耦的运行时中，该过程可以进一步嵌入正式 Prefill 路径，直接复用目标模型已经产生的中间表征；而在本文的 SGLang 原型实验中，为了尽量减少对后端执行链路的侵入，采用前端旁路预测器在请求进入后端实例之前完成长度预测表征抽取和长度预测，再将预测长度交给路由器。该预测过程会带来额外开销，本文在系统级实验中将其计入端到端时延。

5.2.2 基于场景自适应特征融合的长度预测模型

上一节解决的是 “选哪些层–头” 的问题，但仅选择 HotHead 还不足以覆盖所有负载场景。原因在于，不同场景中决定响应长度的提示词结构并不相同。长上下文任务的输出长度通常同时受上下文规模、信息密度和问题定位影响；数学推理任务的输出长度更容易受到题目末尾求解目标、格式约束和推理步骤复杂度影响；开放式指令或 RL 采样场景中，开头的任务描述与末尾的具体约束都可能影响模型何时结束生成。因此，本文在 HotHead 作为核心信号的基础上，再根据场景选择合适的辅助表征和词元位置。

具体来说，本节的融合过程分为三步。第一步，离线层–头扫描得到长度预测表征 $\mathbf{z}_{\text{hh}}$，它是本文方法的主要长度信号来源。第二步，根据场景补充少量已有长度预测方法常用的全局或末端表征，避免单一层–头视角遗漏提示词整体信息。第三步，将这些表征输入同一个轻量回归器，得到最终响应长度预测。

本文使用的辅助表征包括三类。第一类是 Last Token 表征，记为 $\mathbf{z}_{\text{last}}$，它取最后一层最后一个提示词词元的隐藏状态，用于刻画提示词末端对后续生成的直接影响。第二类是 Mean Pool 表征，记为 $\mathbf{z}_{\text{mean}}$，它对最后一层全部提示词词

元的隐藏状态做平均池化，用于刻画提示词整体语义和上下文规模。第三类是 EGTP 表征，记为 $\mathbf{z}_{\text{egtp}}$，它利用词元熵对隐藏状态进行加权池化，用于强调更可能影响生成长度的高不确定性位置。在此基础上，最终用于预测的特征写为

$$\mathbf{z} = \Phi_s\left(\mathbf{z}_{\text{hh}}, \mathbf{z}_{\text{last}}, \mathbf{z}_{\text{mean}}, \mathbf{z}_{\text{egtp}}, \mathbf{z}_{\text{stat}}\right), \tag{5-4}$$

其中 s 表示当前请求所属场景，例如 Long Sequence、Reasoning 或 RL；$\Phi_s(\cdot)$ 表示场景 s 对应的特征融合函数，具体实现为按场景选择若干表征后进行拼接或加权组合；$\mathbf{z}_{\text{hh}}$ 表示由 HotHead 层–头集合得到的长度预测特征，$\mathbf{z}_{\text{last}}$ 表示 Last Token 特征，$\mathbf{z}_{\text{mean}}$ 表示 Mean Pool 特征，$\mathbf{z}_{\text{egtp}}$ 表示 EGTP 特征，$\mathbf{z}_{\text{stat}}$ 表示提示词长度、任务约束等轻量提示词侧统计特征，$\mathbf{z}$ 表示送入最终长度预测器的融合特征。

需要说明的是，式中的 $\Phi_s(\cdot)$ 并不表示每个场景都必须使用全部五类特征，而是表示 “按场景选择和组合特征” 这一过程。例如，当验证集显示长上下文场景更依赖全局信息时，$\Phi_s(\cdot)$ 会更偏向组合 $\mathbf{z}_{\text{hh}}$ 与 $\mathbf{z}_{\text{mean}}$；当验证集显示推理或开放式生成场景更依赖问题首尾约束时，$\Phi_s(\cdot)$ 会更偏向组合首尾词元上的 $\mathbf{z}_{\text{hh}}$，并视情况加入 $\mathbf{z}_{\text{last}}$ 或 $\mathbf{z}_{\text{egtp}}$。因此，5.2.2 节的核心并不是提出一个固定不变的特征堆叠方式，而是在 HotHead 为主信号的前提下，根据不同负载场景选择最适合的辅助表征。

在词元位置选择上，本文进一步发现，HotHead 的有效性不仅取决于选择哪些注意力头，也取决于从哪些提示词位置读取这些注意力头的表示。对于部分 RL 和 Reasoning 样本，原始 EGTP 实现主要使用提示词前若干词元，容易遗漏位于提示词末尾的具体问题、格式约束或回答要求。因此，本文在这些场景中采用 “前 4 个词元 + 后 4 个词元” 的 HotHead 聚合方式，使预测器同时看到任务开头的全局指令和末尾的直接约束。实验表明，这一改动在 Qwen2.5-3B 的 Reasoning 和 RL 场景中带来了明显收益。

长度预测器采用轻量回归模型或轻量神经预测头。训练目标为最小化预测长度 $\hat{y}$ 与真实响应长度 y 之间的误差：

$$\mathcal{L}_{\text{len}} = \frac{1}{N}\sum_{i=1}^{N}\left|\hat{y}_i - y_i\right|. \tag{5-5}$$

其中 $\mathcal{L}_{\text{len}}$ 表示长度预测训练损失，N 表示训练样本数，i 表示样本编号，$\hat{y}_i$ 表示第 i 条样本的预测响应长度，y_i 表示第 i 条样本的真实响应长度，$\left|\hat{y}_i - y_i\right|$ 表示单条样本的绝对预测误差。这里的真实长度 y_i 由对应目标模型分词器对生成响应计数得到。训练完成后，在线系统只保留最终预测器和场景对应的特征配置。对于后续调度而言，$\hat{y}_i$ 被解释为请求 r_i 在 Decode 阶段可能继续生成的词元数，并进一步转化为未来 KV Cache 负载估计。

需要强调的是，场景自适应融合并不改变本文方法的核心：HotHead 始终是最终预测方法中的主要组成部分。Last Token、Mean Pool、EGTP 和提示词统计特征只是围绕 HotHead 的辅助视角，用于弥补不同场景中长度信号位置不同的问题。这样的设计既保留了层–头扫描带来的模型内部可解释性，也避免把长度预测绑定到某一个固定词元位置或单一池化方式上。

5.3 基于 KV Cache 负载感知的多 GPU 路由调度方法

上一节得到的响应长度预测结果，需要进一步转化为 Decode 阶段的系统调度决策。本节讨论如何把预测长度映射为 KV Cache 负载估计，并在多 GPU 服务中用于请求路由。与只平衡请求数量的默认策略不同，本文的目标是平衡每张 GPU 后续将承担的 Decode 词元数和 KV Cache 增长量，从而减少长请求集中导致的尾部拖延。

5.3.1 基于预测响应长度的 KV Cache 负载估计

设请求 r_i 的输入长度为 l_i^{in}，预测响应长度为 $\hat{y}_i$。在 Decode 阶段，请求需要持续保留输入部分 KV Cache，并随着生成逐步追加输出部分 KV Cache。因此，请求的未来 KV Cache 规模可以近似表示为

$$\hat{c}_i = m_{\text{tok}} \left(l_i^{\text{in}} + \hat{y}_i \right), \tag{5-6}$$

其中 r_i 表示第 i 个请求，l_i^{in} 表示该请求输入提示词的词元数，$\hat{y}_i$ 表示长度预测器给出的响应词元数估计，m_{tok} 表示单位词元在目标模型上对应的 KV Cache 占用，$\hat{c}_i$ 表示请求 r_i 的预测 KV Cache 占用规模。由于同一实验中各 GPU 运行相

同目标模型，m_{tok} 对所有请求都是相同的比例系数；若只关注不同 GPU 之间的相对负载大小和路由选择结果，则可以省略该常数项，直接使用 $l_i^{\text{in}} + \hat{y}_i$ 作为请求的预测 KV Cache 负载规模。

在运行时，路由器为每张 GPU 维护一个预测负载状态。设系统共有 G 张 GPU，第 g 张 GPU 上当前已经分配但尚未完成的请求集合为 $\mathcal{Q}_g$，则其预测 KV Cache 负载可写为

$$L_g = \sum_{r_i \in \mathcal{Q}_g} \hat{c}_i = m_{\text{tok}} \sum_{r_i \in \mathcal{Q}_g} \left(l_i^{\text{in}} + \hat{y}_i \right). \tag{5-7}$$

其中 G 表示系统中的 GPU 数量，$g \in \{1, \cdots, G\}$ 表示 GPU 编号，$\mathcal{Q}_g$ 表示已经路由到第 g 张 GPU 且尚未完成的请求集合，$r_i \in \mathcal{Q}_g$ 表示请求 r_i 属于该集合，L_g 表示第 g 张 GPU 当前维护的预测 KV Cache 负载。该公式直接承接式（5-6）：单个请求的预测负载为 $\hat{c}_i$，一张 GPU 的预测负载就是其上所有未完成请求的 $\hat{c}_i$ 之和。进一步代入式（5-6）后，可以得到右侧按输入词元数和预测输出词元数展开的形式。由于同一目标模型下 m_{tok} 对所有 GPU 相同，实际路由时可省略该常数项，使用 $\sum_{r_i \in \mathcal{Q}_g} (l_i^{\text{in}} + \hat{y}_i)$ 作为相对负载指标。当请求完成后，路由器从对应 GPU 的 L_g 中扣除该请求的预测负载；当新请求被分配到某张 GPU 后，再将其预测负载加入该 GPU。这样，调度器维护的不再是简单的运行中请求数，而是对未来 Decode 负载和 KV Cache 增长压力的近似估计。

这一估计并不要求预测长度完全精确。对于路由而言，更关键的是区分不同请求的相对负载规模，尤其是避免多个长请求被连续分配到同一张 GPU。即使单个请求存在一定预测误差，只要预测器能够在整体上识别较长响应和较短响应，路由器就可以利用该信号改善多 GPU 之间的负载分布。因此，本文方法只使用请求生成前可获得的 HotHead 预测结果，不依赖事后可见的真实生成长度，可以直接用于在线路由决策。

从 KV Cache 管理角度看，式中 L_g 同时包含两层含义。其一，它近似描述第 g 张 GPU 后续需要容纳的 KV Cache 词元规模；其二，它也近似描述该 GPU 后续需要执行的 Decode 词元数。由于 Decode 阶段每个词元都对应一次 KV Cache 写入和后续注意力访问，因此这两个含义在系统层面高度相关。基于 L_g 进行路由，本质上就是把响应长度预测转化为未来 KV Cache 负载均衡问题。

5.3.2 基于预测负载均衡的多 GPU 请求路由

在得到每张 GPU 的预测负载后，本文采用一种直接的负载感知路由策略。对于新到达请求 r_i，系统首先由前端旁路预测器抽取长度预测表征，并由长度预测器输出 $\hat{y}_i$；随后计算该请求的预测 KV Cache 负载 $\hat{c}_i$，并将其分配给当前预测负载最低的 GPU：

$$g^* = \arg\min_{g \in \{1,\cdots,G\}} L_g. \tag{5-8}$$

其中 g^* 表示新请求最终被路由到的目标 GPU 编号，$\arg\min$ 表示在候选 GPU 集合中选择使目标值最小的编号，$\{1,\cdots,G\}$ 表示全部可用 GPU，L_g 表示第 g 张 GPU 当前的预测 KV Cache 负载。该公式的含义是：新请求优先发送到预测负载最小的 GPU。分配完成后，系统更新

$$L_{g^*} \leftarrow L_{g^*} + l_i^{\text{in}} + \hat{y}_i. \tag{5-9}$$

其中 L_{g^*} 表示目标 GPU 在分配前维护的预测负载，符号 $\leftarrow$ 表示状态更新，$l_i^{\text{in}} + \hat{y}_i$ 表示新请求带来的预测负载增量。更新后，L_{g^*} 同时包含原有未完成请求的预测负载和新请求的预测负载。当请求在后端 GPU 上完成后，路由器再释放其对应的预测负载。

该策略的核心不是把长度相近的请求强行放到同一个 GPU，而是利用预测长度避免未来 KV Cache 负载向某一张 GPU 倾斜。若某张 GPU 已经被分配了多个预测较长的请求，即使它当前请求数不多，L_g 也会相对较高；后续新请求会更倾向于被路由到另一张 GPU。相反，若某张 GPU 上多数请求预测较短，则即使请求数较多，其预测负载仍可能较低，可以继续接纳新请求。这样，系统平衡的是未来 Decode 负载，而不是静态请求个数。

与 SGLang 默认的多 GPU 请求分发相比，本文策略在路由层增加预测负载状态维护，并在请求进入后端实例之前引入一次旁路长度预测，不改变后端模型执行逻辑，也不要求修改模型权重或生成算法[27]。这使它更容易集成到现有服务系统中：默认调度器仍负责单个 GPU 内部的持续批处理和 KV Cache 管理，而本文方法负责在请求进入后端实例之前决定其应该落到哪张 GPU 上。两者的关系是互补的：持续批处理提高单 GPU 内部执行效率，预测负载路由则减少多

GPU 之间的 Decode/KV Cache 负载倾斜。

需要进一步说明的是，本文没有把预测长度直接作为全局排序目标。早期实验表明，若简单按照预测长度对所有请求排序或聚合，可能减少部分批内 padding，却不一定改善端到端时延；原因在于过强的长度排序可能改变请求到达顺序，引入额外等待，并在高并发下影响批次形成。因此，本文最终采用更稳健的路由目标：保持请求正常进入系统，只在 GPU 选择时使用预测 KV Cache 负载作为分配依据。这样既利用了长度预测信息，又避免把预测误差放大为全局调度错误。

5.4 实验评估与分析

为验证本章方法的有效性，本节从两个层面展开实验。首先，评估 HotHead 长度预测方法在不同模型和场景下是否能够优于常用表征基线；其次，将长度预测结果以旁路预测器形式集成到 SGLang 多 GPU 路由中，评估其能否降低端到端时延和尾部延迟，并提升输出吞吐。前者验证 KV Cache 负载估计的前提，后者验证该估计能否转化为系统收益。

5.4.1 实验环境、数据集、对比方法与评价指标

长度预测实验参考 ForeLen 基准构建三类场景：Long Sequence、Reasoning 和 RL。ForeLen 是面向大模型响应长度预测的公开基准，覆盖长序列、推理和强化学习采样类负载[38]。其中，Long Sequence 主要来自长上下文任务，考察模型在长输入和较大长度跨度下的预测能力；Reasoning 主要来自数学推理类任务，考察题目约束和推理复杂度对输出长度的影响；RL 主要来自开放式指令和偏好数据，考察开放生成场景中的响应长度波动。每条样本包含用户提示词、目标模型生成响应以及由目标模型分词器统计得到的响应词元长度。

实验使用两类目标模型：Llama3.2-1B 和 Qwen2.5-3B。选择这两类模型的原因在于，它们规模适中，便于完整完成层–头扫描、特征抽取、预测器训练和系统集成实验；同时二者结构不同，能够检验 HotHead 机制是否具有跨模型稳定性[17,19]。所有长度预测方法均使用相同训练集、验证集和测试集划分，并在相同训练预算下比较测试集 MAE。

长度预测对比方法包括 Last Token、EGTP、Mean Pool 和本文 HotHead 方法。Last Token 使用最后一层最后词元隐藏状态训练预测器，对应已有工作中直接利用末端隐藏状态预测输出长度的轻量读出思路[63-64]；Mean Pool 对最后一层提示词隐藏状态做平均池化，对应基于全局隐藏表示聚合的长度建模基线[63]；EGTP 使用熵引导的词元池化[38]；HotHead 则先进行层–头扫描，再根据场景选择长度预测表征及其融合特征配置。评价指标采用 MAE，数值越低表示预测长度与真实响应长度越接近。

系统级调度实验在改造后的 SGLang 原型上进行。实验使用多 GPU 服务设置，对比 SGLang 默认轮转路由与本文 HotHead 路由。本文原型的真实执行方式为：客户端侧旁路预测器先对待发送请求抽取长度预测表征并预测响应长度，再将预测响应长度随请求参数一并提交给 SGLang；SGLang 数据并行控制器在请求进入后端实例前读取该预测长度，按预测 KV Cache 负载选择目标 GPU。该实现没有在后端正式 Prefill 完成后再迁移 Decode 执行位置，而是以额外一次旁路预测换取请求分发前的长度先验；实验中的整批完成时延已包含旁路预测器运行时间。

工作负载使用生成长度高倾斜请求集合，以突出 Decode 阶段 KV Cache 负载倾斜对端到端性能的影响。具体而言，调度负载包含 256 条请求，由 Reasoning 和 RL 两类场景各 128 条组成；其中长响应请求 128 条，实际生成长度均为 1536 个词元，短响应请求 128 条，实际生成长度位于 2 到 64 个词元之间，均值约为 25.6 个词元。该负载使请求数量在场景与长短类别上保持均衡，但生成长度呈现明显两极分化，从而能够检验长度预测路由在 KV Cache 未来增长量不均衡时的效果。系统指标包括整批完成时延、P99 尾部延迟和输出吞吐。其中，整批完成时延表示完成整批请求的总耗时，并包含旁路长度预测的耗时；P99 尾部延迟反映尾部请求延迟；输出吞吐表示单位时间生成的输出词元数。

5.4.2 长度预测精度与现有方法的比较

图 5-2 和图 5-3 分别给出了 Llama3.2-1B 与 Qwen2.5-3B 上的四类方法对比结果。整体来看，HotHead 在两个模型、三个场景上均取得最低 MAE，说明层–头级别的关键表征选择确实能够提升响应长度预测质量。

在 Llama3.2-1B 上，HotHead 在 Long Sequence 场景中将 MAE 降至 54.79，明

表 5-1　第五章实验的主要配置

类别	参数	设置
长度预测模型	目标模型	Llama3.2-1B / Qwen2.5-3B
数据场景	任务类型	Long Sequence / Reasoning / RL
长度标签	统计方式	使用对应目标模型分词器统计响应词元数
对比方法	长度预测方法	Last Token / EGTP / Mean Pool / HotHead
训练设置	训练轮数 / 学习率	50 / 1×10^{-3}
HotHead 设置	选择方式	验证集层–头扫描，按场景固定最终特征配置
调度系统	推理框架	SGLang 0.4.1.post1 改造原型
调度负载	请求组成	Reasoning / RL 各 128 条，长 / 短响应各 128 条
调度对比	路由策略	SGLang 轮转路由 / HotHead 预测负载路由
评价指标	预测与系统指标	MAE / 整批完成时延 / P99 尾部延迟 / 输出吞吐

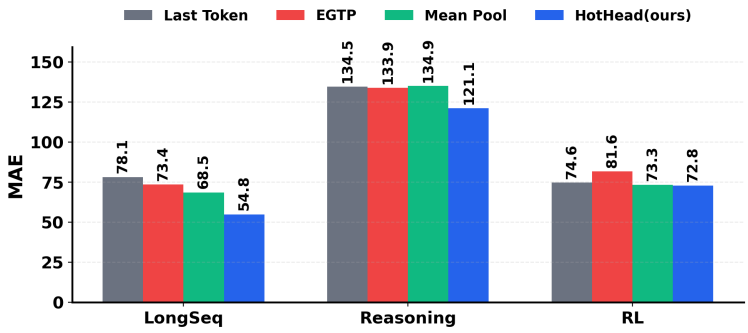


图 5-2　Llama3.2-1B 在 LongSeq、Reasoning 和 RL 不同场景下的响应长度预测 MAE 对比

显低于 Last Token 的 78.12、EGTP 的 73.38 和 Mean Pool 的 68.50；在 Reasoning 场景中，HotHead 的 MAE 为 121.15，也低于三类基线；在 RL 场景中，HotHead 的 MAE 为 72.81，略优于 Mean Pool 的 73.30，并优于 Last Token 和 EGTP。该结果说明，在小规模 Llama 模型中，HotHead 对长上下文和推理场景的收益更明显，在开放式 RL 场景中则体现为较稳健的小幅改进。

在 Qwen2.5-3B 上，HotHead 同样保持优势。Long Sequence 场景中，HotHead 将 MAE 从最佳基线 EGTP 的 90.82 降至 81.20，相对降低约 10.6%；Reasoning 场景中，HotHead 的 MAE 为 116.99，略优于 Last Token 的 118.07；RL 场景中，HotHead 将 MAE 从最佳基线 Last Token 的 71.89 降至 65.89，相对降低约 8.35%。特别是在 RL 场景中，使用提示词前 4 个词元与后 4 个词元共同构造长度预测表征后，预测误差显著下降，说明提示词末尾的任务约束和回答要求对开放式生

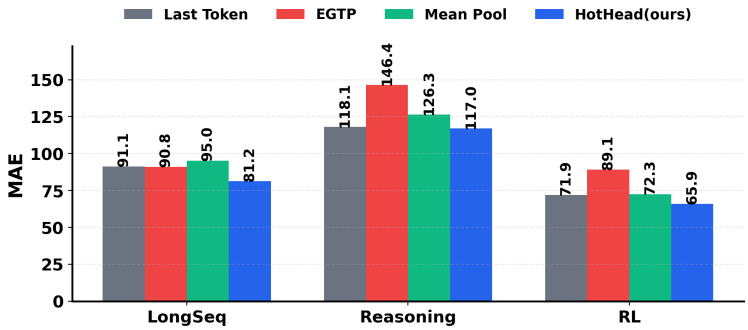


图 5-3　Qwen2.5-3B 在 LongSeq、Reasoning 和 RL 不同场景下的响应长度预测 MAE 对比

成长度具有重要影响。

表 5-2 汇总了两个模型上的最终 MAE 结果。从表中可以看到，HotHead 的优势并不来自单一模型或单一任务，而是在不同模型结构和不同负载类型下均能成立。这表明响应长度线索确实存在于模型内部少量关键层–头中，而场景自适应融合能够进一步把这些信号转化为稳定的长度估计。

表 5-2　响应长度预测 MAE 对比结果

模型	场景	Last Token	EGTP	Mean Pool	HotHead
Llama3.2-1B	Long Sequence	78.12	73.38	68.50	**54.79**
Llama3.2-1B	Reasoning	134.53	133.85	134.87	**121.15**
Llama3.2-1B	RL	74.64	81.56	73.30	**72.81**
Qwen2.5-3B	Long Sequence	91.12	90.82	95.03	**81.20**
Qwen2.5-3B	Reasoning	118.07	146.37	126.32	**116.99**
Qwen2.5-3B	RL	71.89	89.10	72.35	**65.89**

从 KV Cache 负载估计角度看，上述结果说明 HotHead 预测器能够提供比最后层表征更可靠的响应长度先验。虽然不同场景下最终融合特征配置不完全相同，但所有特征配置都以 HotHead 作为主要组成部分：Long Sequence 更依赖全局池化与层–头特征的组合，Reasoning 和 RL 更依赖首尾词元约束下的长度预测表征。这一现象也进一步说明，响应长度预测不能只依赖固定位置的隐藏状态，而需要结合模型内部关键注意力头与任务结构进行设计。

5.4.3　新调度策略与 SGLang 原始调度策略的比较

在系统级实验中，本文将 HotHead 长度预测模块以旁路预测器形式集成到 SGLang 多 GPU 路由层，并与默认轮转路由进行对比。实验使用上述 256 条生成长度高倾斜请求构造负载，以模拟 Decode 阶段长短请求混合到达且未来 KV

Cache 增长量差异显著的场景。默认策略按请求顺序在多张 GPU 间轮转分配，本文策略则在请求进入后端前通过旁路预测器得到响应长度估计，并根据每张 GPU 当前预测 KV Cache 负载选择目标 GPU。

图 5-4 给出了系统级结果。可以看到，在计入旁路预测耗时的情况下，相比 SGLang 默认轮转路由，本文方法将整批完成时延从 73.99 s 降至 49.33 s，降低约 33.3%；将 P99 尾部延迟从 66.93 s 降至 25.47 s，降低约 61.9%；同时将输出吞吐从 2483 tokens/s 提升至 3818 tokens/s，提升约 53.8%。

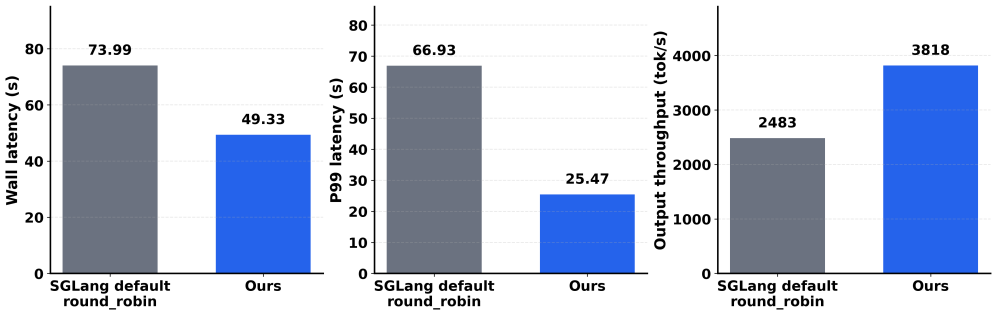


图 5-4　本文 KV Cache 负载感知路由与 SGLang 默认路由的端到端对比

上述结果表明，在生成长度高倾斜负载下，长度预测带来的收益并不局限于预测任务本身，而能够实际转化为 Decode 阶段的系统性能改进。默认轮转策略能够在请求数量上保持平均，但当请求输出长度呈现明显长短分化时，请求数量均衡并不能保证 KV Cache 负载均衡。本文方法通过预测响应长度提前估计未来 Decode 词元数和 KV Cache 增长量，从而减少长请求在单张 GPU 上的集中堆积，使多张 GPU 的后续执行压力更接近。

从不同指标看，P99 尾部延迟的下降幅度最大，说明本文方法主要缓解的是尾部 GPU 被长请求拖住的问题。整批完成时延的下降说明整体负载完成时间缩短，而输出吞吐的提升则说明负载均衡后两张 GPU 的有效生成能力得到更充分利用。需要注意的是，路由实验中的单请求预测误差仍然存在；但调度收益说明，对于多 GPU 路由而言，预测长度只需提供足够有效的相对负载信号，就能够帮助系统做出更合理的请求分配。

综合来看，HotHead 长度预测与 KV Cache 负载感知路由之间形成了直接闭环：前者通过旁路预测器从长度预测表征中提取响应长度先验，后者将该先验转化为每张 GPU 的未来 KV Cache 负载估计，并据此完成请求路由。该闭环使 Decode 阶段的 KV Cache 管理不再只依赖运行时被动增长，而能够在请求进入

后端 Decode 之前进行前瞻性负载规划。需要说明的是，当前原型采用旁路预测方式，因此会额外执行一次长度预测前向过程；在支持 Prefill–Decode 解耦或中间表征导出的服务架构中，该预测过程可以进一步与正式 Prefill 路径融合，从而减少额外计算开销。

5.5 本章小结

本章围绕 Decode 阶段 KV Cache 随生成过程持续增长、且不同请求未来增长量难以提前确定的问题，提出了响应长度预测与 KV Cache 负载感知多 GPU 路由调度方法。首先，本文通过层–头预测能力扫描发现，响应长度线索集中存在于少量关键注意力头中，并据此构造长度预测表征；随后，结合不同场景的词元位置和全局池化特征，形成场景自适应的长度预测模型，为后续调度提供响应长度先验。

在系统层面，本文进一步将预测响应长度转化为未来 KV Cache 负载估计，并在双 GPU 路由层维护每张 GPU 的预测负载状态。与只平衡请求数量的默认路由不同，本文方法根据预测 Decode 负载选择目标 GPU，从而减少长请求集中导致的 KV Cache 负载倾斜和尾部拖延。

实验结果表明，HotHead 方法在 Llama3.2-1B 和 Qwen2.5-3B 的 Long Sequence、Reasoning 和 RL 三类场景中均优于 Last Token、EGTP 和 Mean Pool 等基线；在 SGLang 双 GPU 调度实验中，本文方法相较默认轮转路由显著降低整批完成时延和 P99 尾部延迟，并提升输出吞吐。至此，本文已分别从 Prefill 阶段的共享前缀 KV Cache 管理和 Decode 阶段的生成 KV Cache 负载规划两个方面，完成对 Prefill–Decode 双阶段 KV Cache 优化的核心方法设计。下一章将在此基础上进一步给出统一系统架构与关键模块实现。

第六章　系统架构与关键模块实现

在第三章至第五章分别给出具体方法之后，本章从系统实现角度说明这些方法如何被组织为一个统一的大模型推理运行时，并进一步介绍关键模块与执行流程。

6.1　系统设计目标与总体架构

在完成第三章至第五章的方法设计之后，还需要回答一个工程问题：如何把 Prefill 阶段的异构内存管理、Prefill–Decode 交叠过程中的主动预取以及 Decode 阶段的响应长度预测与多 GPU 请求调度真正组织成一个统一可运行的推理系统。若这三部分仅以孤立算法存在，那么它们之间的状态传递、资源边界和执行时序仍不明确，难以支撑完整实现。因此，本章从系统角度说明本文方法的整体架构与关键实现方式。

本文系统的设计目标主要有三点。第一，在 Prefill 阶段统一管理 GPU 显存与主机内存中的共享前缀 KV Cache，使热度追踪、显存驱逐、主存准入和主存回迁建立在同一套前缀树状态之上。第二，在 Decode 阶段围绕请求未来响应长度建立负载先验，将响应长度预测接入多 GPU 路由调度器，把预测响应长度转化为每张 GPU 的未来 KV Cache 负载估计。第三，在全链路执行上保证 Prefill 侧缓存管理与 Decode 侧调度控制彼此协同，使缓存命中率提升和 GPU 间负载均衡最终能够转化为端到端吞吐与时延收益。

基于上述目标，本文将系统划分为接入与批处理层、Prefill 执行层、异构内存管理层以及 Decode 调度层四个部分，如图 6-1 所示。接入与批处理层负责请求接收、前缀匹配和批次组织；Prefill 执行层负责共享前缀复用与内部表征产生；异构内存管理层负责第三章、第四章中的热度追踪、卸载、准入和预取；Decode 调度层则承接第五章中的响应长度预测、KV Cache 负载估计以及多 GPU 请求路由。

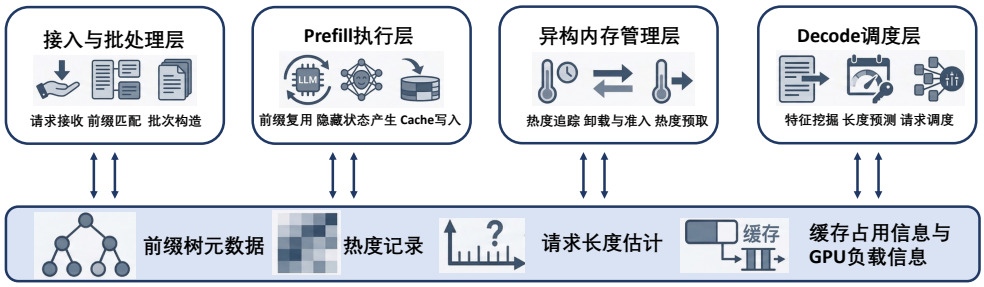


图 6-1　本文推理系统的总体架构

从状态组织方式看，系统并不为第三、第四、第五章分别维护独立的数据结构，而是尽量复用统一元数据。前缀树节点既记录其在 GPU 或 CPU 中的存储位置，也记录热度追踪所需的指纹、频次、时钟和深度；请求对象既持有输入长度、已生成长度和内部表征摘要，也持有由长度预测模块生成的长度估计结果，以及由调度器维护的设备归属和 KV Cache 占用信息。正因为底层状态是统一的，第三至第五章的方法才能在工程上真正串联起来。

进一步看，本文系统的总体实现遵循三条工程原则。第一是**状态单源化**：同一类运行时事实只在系统中保留一份权威状态，例如前缀节点驻留位置只由前缀树维护、预测响应长度只写入请求控制块，避免多个模块各自缓存一份副本后出现不一致。第二是**关键路径最短化**：凡是不直接参与当前轮 GPU 推理计算的控制逻辑，尽量放到 CPU 后台线程或独立 CUDA Stream 中异步执行，避免把元数据管理、热度衰减和数据回迁塞进同一条执行流。第三是**退化可运行**：即使某些优化模块暂时失效，例如预取计划为空、长度预测不可用或主存带宽阶段性受限，系统也应当退化回基础的共享前缀推理流程，而不是整体阻塞。

6.1.1　统一状态对象与模块接口

为了把第三章至第五章的方法真正组织成统一运行时，本文在实现上进一步抽象了三类核心状态对象。第一类是**前缀树节点对象**，负责维护词元范围、父子关系、存储层级、KV Cache 地址以及热度索引，是 Prefill 侧所有缓存管理操作的共同入口。第二类是**请求控制块**，负责维护请求输入长度、已生成长度、长度预测结果、当前阶段状态、设备归属和资源占用信息，是 Decode 侧调度器的主要输入来源。第三类是**迁移任务描述符**，负责记录待卸载节点、待回迁节点、

表 6-1　系统模块与论文章节对应关系

模块	主要职责	对应章节
前缀树与热度记录模块	统一维护 CPU/GPU 两级 KV Cache 元数据、节点热度和父子关系	第三章
卸载与准入模块	在显存压力上升时执行热度感知驱逐，并筛选值得进入主存的节点	第三章
预取与传输执行模块	构造热度提升计划，在 Decode 期间异步回迁主存热点节点	第四章
长度预测模块	基于长度预测表征估计请求响应长度；本文原型采用前端旁路预测实现	第五章
多 GPU 路由调度模块	基于输入长度、预测响应长度和各 GPU 当前状态估计未来 KV Cache 负载并分配请求	第五章
运行时协调模块	串联 Prefill、预取、Decode 三段执行并维护统一状态	本章

目标地址和异步执行状态，用于连接运行时协调模块与具体的数据传输模块。

在模块接口设计上，本文尽量采用读统一状态、写少量必要结果的方式降低耦合。例如，热度追踪模块只负责更新过滤器和节点活跃标记，不直接决定节点是否被驱逐；驱逐与准入模块只输出保留、迁移、丢弃三类决策，不直接管理后续预取；长度预测模块只把预测响应长度和置信区间写入请求控制块，不直接决定请求应被分配到哪张 GPU。这样设计的好处是，各模块的职责边界更清晰，也更便于后续替换某个局部策略而不破坏整体运行时结构。

6.2　关键数据结构与状态转换

在统一架构之外，系统能否稳定运行还取决于核心状态是否具有清晰、可维护的转换关系。为此，本文进一步从前缀树节点和请求控制块两个层面，说明系统如何维护运行时状态演化。

6.2.1　前缀树节点状态转换

对 Prefill 侧而言，一个前缀树节点在运行时通常经历以下几种状态：其一是驻留 GPU 且可命中，表示该节点当前位于显存中，能够直接服务后续共享前缀复用；其二是驻留 CPU 且可回迁，表示节点已通过第三章的选择性准入，被保存在主机内存中，可作为第四章预取候选；其三是迁移中，表示该节点已被某一

轮卸载或预取计划选中，正在等待异步传输完成；其四是失效待回收，表示节点未通过准入或已被后续决策淘汰，将在元数据清理后被彻底移除。

系统在实现时并不允许节点在这些状态之间任意跳转，而是通过受控状态转换维持一致性。例如，节点只有在显存压力触发驱逐后，才可能从驻留 GPU 且可命中转为迁移中或失效待回收；只有在 GPU–CPU 传输成功并完成元数据更新后，才会进入驻留 CPU 且可回迁；而主存节点只有在通过第四章预取计划并完成 CPU–GPU 回迁后，才会重新回到驻留 GPU 且可命中状态。通过明确这些状态边界，系统可以避免同一节点同时被多个模块视作不同驻留层级，从而减少复杂运行时中的一致性错误。

6.2.2 请求控制块状态转换

与节点状态对应，请求对象在运行时也具有清晰的阶段划分。一个请求在进入系统后，通常依次经历等待接入、执行 Prefill、等待 Decode 调度、执行 Decode、完成释放五类状态。每次状态变化都会伴随关键字段更新：例如，在 Prefill 完成时写入长度预测结果和初始显存占用，在 Decode 过程中持续更新已生成长度和当前 KV Cache 占用，在请求完成时触发资源释放和统计信息归档。

这种请求状态机设计有两个直接收益。其一，它把长度预测、KV Cache 负载估计和多 GPU 请求路由都绑定到统一的请求生命周期上，使第五章方法能够自然嵌入运行时，而不是额外维护一套平行队列。其二，它使系统更容易处理动态变化：当某些请求比预测更快结束时，调度器可以立即根据请求状态变化回收对应 GPU 上的 KV Cache 预算；当某些请求明显偏长时，系统也可以在后续路由决策中提高该 GPU 的负载估计，减少新的长请求继续流入同一设备。

6.3 面向 Prefill 阶段的异构内存协同实现

在总体架构明确之后，本节先从 Prefill 侧展开，说明第三章和第四章的方法在系统内部是如何落到统一运行时上的。具体而言，重点包括两部分：其一，是前缀树与热度记录如何作为共享状态被持续维护；其二，是卸载、准入与预取如何被组织成一条连续的异构内存执行链路。

6.3.1 前缀树与热度记录模块实现

本文系统基于 SGLang 0.4.1.post1 扩展实现，并继续沿用按词元粒度进行 KV Cache 分配与释放的基本策略。在此基础上，本文将 CPU 主存与 GPU 显存中的前缀缓存统一挂接到同一棵前缀树中，使每个节点都具备以下几类状态：节点对应词元范围、父子关系、当前驻留层级、KV Cache 地址、热度记录索引以及是否参与本轮迁移计划等标志位。这样一来，前缀树不仅是 Prefix Sharing 的逻辑索引结构，也成为异构内存管理与调度协同的核心状态载体。

热度记录模块则复用第三章中的增强型布谷鸟过滤器。具体实现时，当请求命中某个共享前缀节点，系统立即执行两项操作：一是更新该节点访问频次，并将时钟值重置为预设上限；二是同步刷新节点在前缀树中的活跃标记，便于后续驱逐与预取阶段快速收集候选集合。与此同时，后台 CPU 线程周期性衰减时钟值，使热度记录始终能够同时反映长期访问频率和短期活跃程度。由于这一过程主要发生在 CPU 侧元数据层面，因此不会显著侵占 GPU 计算资源。

为提高实现灵活性，本文进一步将驱逐优先级、主存热度比较和预取热度排序统一抽象为可配置策略项，并以同一组基础字段作为输入，从而允许系统在不改动整体架构的情况下对公式做局部替换。这种实现方式对科研原型尤其重要，因为它方便后续快速验证不同热度定义对整体系统行为的影响。

从运行时一致性角度看，前缀树模块还承担了跨层状态同步的职责。每当节点在 GPU 和 CPU 之间迁移时，系统不仅更新其驻留层级和内存地址，还会同步刷新该节点是否可参与共享命中、是否处于本轮迁移计划中、是否仍具备完整祖先链等标志位。这样一来，无论是第三章的驱逐、第四章的预取，还是后续请求到达时的前缀匹配，读取到的都是同一份一致状态，而不会出现节点已在主存中，但前缀匹配仍认为它驻留显存的逻辑冲突。

6.3.2 卸载、准入与预取执行链路实现

在 Prefill 执行链路中，第三章和第四章的方法实际上组成了一条连续的数据通路。当新请求写入前缀树后，如果显存预算仍充足，则节点和其 KV Cache 直接保留在 GPU；若显存不足，则由热度感知驱逐模块从 GPU 叶子节点集合中选择优先级最低的一批节点执行卸载。这里的卸载并不天然等价于迁移到主存，

而是先进入选择性准入判断：若候选节点未通过频次过滤或热度比较，则其元数据会被立即删除，对应显存空间直接回收；只有通过准入的节点才会真正触发 GPU 到 CPU 的 KV Cache 传输，并将前缀树节点状态更新为驻留主存。

预取执行则发生在当前批次 Prefill 完成之后。此时运行时协调模块收集主存根节点与 GPU 叶节点，调用第四章的热度提升逻辑生成 Promotion Plan。计划生成完成后，系统不会立即阻塞式搬运数据，而是把待替换的 GPU 节点与待回迁的 CPU 节点打包成一批异步传输任务，提交给专门的 CUDA Stream 执行。这样设计的目的是让回迁 I/O 尽量与 Decode 计算重叠，从而把异构内存带来的附加开销压缩到较低水平。

需要强调的是，这条 Prefill 侧实现链路中每一步都依赖同一棵前缀树和同一套热度记录：驱逐模块利用叶节点集合和热度优先级做替换，准入模块利用同一热度记录判断主存价值，预取模块再利用这些记录和树结构关系判断哪些主存节点值得被提升回显存。也正因如此，第三章和第四章不是两个割裂的算法集合，而是一个完整的异构内存缓存运行时。

在工程实现上，这条链路还刻意避免了边决策边立即执行的强耦合方式。系统通常先完成当前轮候选节点收集与计划生成，再统一触发迁移任务提交，从而把决策阶段和执行阶段在时间上适度拆开。这样做有两个直接好处：一是可以在正式传输前检查显存预算、祖先链完整性和任务冲突，避免中途回滚；二是有利于把多个小规模迁移任务聚合成更适合异步执行的大粒度任务，提高链路利用率。

6.4 面向 Decode 阶段的长度预测与多 GPU 路由实现

与 Prefill 侧主要处理共享前缀缓存不同，Decode 侧更关注请求未来生成过程带来的 KV Cache 持续增长。本节进一步说明第五章的方法如何被集成为运行时中的常驻模块，包括长度预测服务如何接入请求生命周期，以及路由调度器如何结合预测响应长度与各 GPU 当前状态完成请求分配。需要特别说明的是，本文当前原型为了保持与 SGLang 后端执行链路的低耦合，采用前端旁路预测方式：请求进入后端实例之前，旁路预测器先基于提示词抽取长度预测表征并预测响应长度，再把预测结果写入请求参数，供 SGLang 数据并行路由器使用。

6.4.1 长度预测服务实现

第五章的方法要真正服务系统调度，关键在于把长度预测变成运行时中的常驻服务，而不是单独离线执行的分析模型。本文的实现思路分为离线和在线两部分：离线阶段先对不同模型完成逐层逐头扫描，确定各自有效层以及层内高分注意力头；在线阶段在本文原型中由前端旁路预测器完成特征抽取和长度预测，即在请求提交到 SGLang 后端前，根据离线选定的 HotHead 配置从提示词中抽取输出表征、词元熵信息和聚合表示，调用长度预测模块生成该请求的响应长度估计，并将其作为请求参数传给路由器。这样做的好处是无需改动后端模型权重、生成算法和单 GPU 内部持续批处理逻辑，代价是预测器会额外执行一次前向过程；本文系统级实验将这部分预测耗时计入端到端结果。

在请求进入后端实例之前，路由调度器读取请求输入长度、预测响应长度和不同 GPU 的当前负载状态，估计该请求被分配到某张 GPU 后可能带来的新增 KV Cache 压力。实现上，请求控制块中至少维护输入长度、预测响应长度、请求所属场景以及当前分配设备等字段；GPU 状态中维护正在服务请求的输入长度、已生成长度、剩余长度估计和 KV Cache 占用近似值。路由调度器只需读取这些字段即可执行负载感知分配，而不必等待请求真实生成完成后再被动修正。

从系统耦合方式看，长度预测服务并不直接操作 KV Cache 本身，而是向调度器输出预测响应长度与长度波动范围这一中间状态。这个设计与第三、第四章中的热度记录形成呼应：前者刻画请求未来还会生成多少内容，后者刻画前缀缓存未来是否还会被复用，两者分别为 Decode 阶段和 Prefill 阶段提供决策依据。对于支持 Prefill–Decode 解耦的服务架构，旁路预测器也可以替换为正式 Prefill 路径中的轻量读出模块，从而复用后端已经产生的中间表征并减少额外前向开销；但本文实验采用的是前端旁路预测实现。

为了控制预测开销，长度预测服务在运行时只提取离线阶段已经筛选出的少量高价值层和显著注意力头，而不再扫描全模型所有层头。实际执行时，旁路预测器批量处理待发送请求，生成响应长度估计后写入请求参数；后端 SGLang 实例随后按照该参数完成多 GPU 分发和正式 Prefill/Decode 执行。之后若请求在 Decode 期间表现出明显偏离，例如实际生成步数已接近预测上界，系统也可以选择基于当前已生成长度做一次轻量修正。这样既保留了第五章方法的主要收

益，也避免了把长度预测模块做成一个高耦合的后端子系统。

6.4.2 多 GPU 请求路由调度器实现

在 Decode 执行层，路由调度器接收的核心输入包括：新请求的输入长度、旁路预测器给出的预测响应长度、单位词元 KV Cache 开销以及各 GPU 当前承载的预测负载。本文实现中，运行时先根据请求输入长度与预测响应长度估计其未来 KV Cache 需求，再分别计算该请求放入不同 GPU 后的负载增量；随后，路由调度器选择预计总负载较低且能够保持负载均衡的 GPU 作为目标设备。

为了避免路由调度器与底层内存管理器脱节，请求分配会直接更新目标 GPU 的预测 KV Cache 负载。当某请求提前完成时，系统会立刻释放其对应的 KV Cache 空间，并从该 GPU 的负载统计中扣除相应占用；当某请求预测响应长度明显偏长时，系统会在后续分配中提高该 GPU 的预计负载，避免多个长请求继续集中到同一设备。这样，长度预测结果就不再只是一个离线评估指标，而是能够直接改变多 GPU 服务中请求去向的控制信号。

调度器实现中还需要处理一个实际问题：请求长度预测并不是静态不变的资源声明，而是带有误差的近似估计。因此，系统不会把预测负载直接当作精确显存占用，而会在计算 GPU 负载时保留安全余量，用于吸收请求尾部生成和偶发预测偏差带来的波动。与此同时，路由策略并不追求让每张 GPU 的请求个数完全相同，而是优先平衡由长请求引起的未来 KV Cache 增长压力。这些细节虽然不改变第五章的核心思想，却直接影响在线系统能否稳定复现论文中的收益。

6.5 Prefill-Decode 双阶段联合执行流程

在分别说明 Prefill 侧和 Decode 侧关键模块之后，本节进一步从端到端运行视角梳理请求在系统中的完整执行路径，重点展示共享前缀复用、异构内存协同、长度预测与多 GPU 路由调度是如何在统一时序上相互衔接的。

图 6-2 进一步从运行时视角给出了本文系统的端到端执行过程。与图 6-1 强调整体模块划分不同，图 6-2 更关注一次请求在接入、旁路长度预测、多 GPU 路由、Prefill、缓存管理、预取以及 Decode 执行之间的流转关系，以及 GPU 显存、CPU 主存和调度控制逻辑在统一时序上的协同方式。

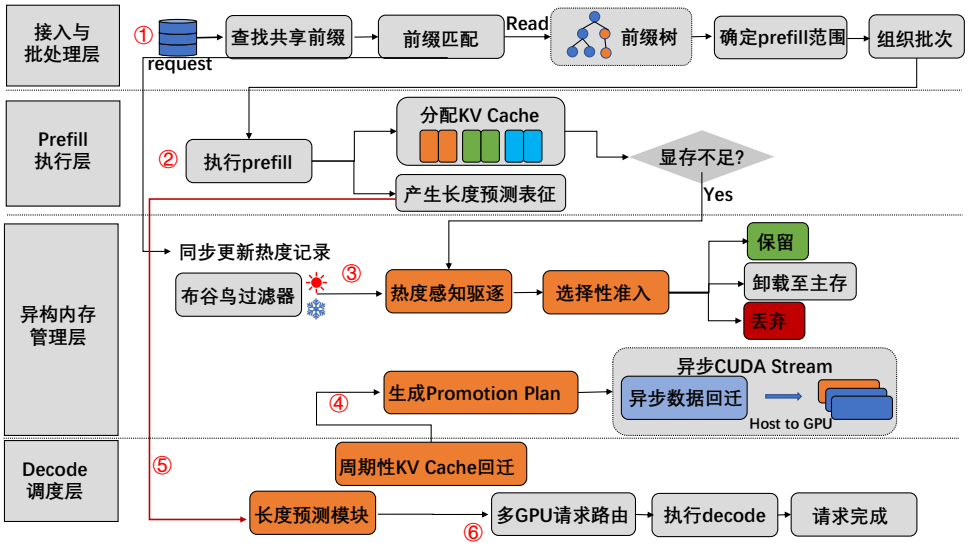


图 6-2　双阶段联合执行流程示意

综合第三章至第五章，本文系统的端到端执行流程可概括为如下六个阶段：

1) 如图 6-2 所示，请求到达后首先进入接入层；在本文当前 SGLang 路由原型中，前端旁路预测器会先批量抽取长度预测表征，生成响应长度估计，并将这些结果写入请求参数；
2) 多 GPU 路由调度器读取输入长度、预测响应长度和各 GPU 当前预测负载，估计请求进入不同设备后的未来 KV Cache 压力，并将请求分配给预计负载更合适的后端实例；
3) 请求进入目标 GPU 后执行正式 Prefill；系统沿前缀树查找可复用的共享前缀，按词元粒度分配 GPU KV Cache，并同步把新生成或被命中的前缀节点写回统一前缀树状态；
4) 若图中 GPU 显存侧的可用预算不足，则运行时会触发第三章中的热度感知驱逐与选择性准入：一部分低价值节点被直接丢弃，一部分高价值节点被卸载到 CPU 主存，从而在释放显存空间的同时保留后续可能复用的热点前缀；
5) 当前批次 Prefill 完成后，系统依据第四章中的热度提升策略，从图中主存侧候选节点中构造回迁计划，并通过独立 CUDA Stream 在 Decode 期间异步执行 CPU 到 GPU 的预取传输，以隐藏跨层数据搬移带来的额外开销；
6) 最后，后端实例执行正式 Decode；在请求完成后，系统及时释放其 KV Cache 与控制状态，并从路由器维护的预测负载中扣除对应占用，使后续

请求能够继续复用被回收出的预算空间。

结合图 6-2 可以看出，本文系统并不是简单地把 Prefill 优化和 Decode 优化前后拼接起来，而是通过统一的请求状态、统一的前缀树元数据以及统一的运行时调度将二者联结起来。第三章和第四章主要负责让共享前缀缓存以更高价值的方式存活下来，第五章则在当前原型中通过前端旁路预测，在请求进入后端 Decode 长时间执行之前估计其未来 KV Cache 增长，并据此完成多 GPU 负载感知路由，两者最终共同作用于端到端吞吐与时延。

6.6 本章小结

本章从系统角度将第三章至第五章提出的方法串联为一个统一的大模型推理运行时。首先，给出了由接入与批处理层、Prefill 执行层、异构内存管理层和 Decode 调度层构成的总体架构；随后，说明了前缀树、热度记录、卸载准入、异步预取、长度预测和多 GPU 负载感知路由等关键模块的具体实现方式；进一步地，梳理了请求从进入系统到完成 Decode 的双阶段联合执行流程。通过这一系统化实现，第三章和第四章的异构内存缓存优化与第五章的响应长度预测和 KV Cache 负载感知调度不再是分散的局部技术，而是共同构成面向大模型推理 Prefill-Decode 双阶段协同优化的完整系统。

第七章　总结与展望

本章对全文工作进行归纳总结，在回顾主要研究贡献的基础上分析当前方案的局限性，并对后续可继续深入的研究方向进行展望。

7.1　本文工作总结

本文围绕大模型推理系统中 Prefill-Decode 双阶段的 KV Cache 优化问题展开研究，从体系结构维度研究异构内存协同，从数据结构维度研究前缀共享，从模型结构维度研究响应长度预测与 KV Cache 负载感知请求路由，形成了较为完整的多维度融合优化框架。全文的主要工作可归纳为以下三个方面。

第一，针对前缀共享场景下显存容量受限、热点前缀难以稳定保留的问题，本文在第三章提出了面向 Prefill 阶段的 KV Cache 显存–主存卸载与准入策略。该方法基于增强型布谷鸟过滤器实现低开销热度追踪，并结合前缀树层次结构设计热度感知显存驱逐策略与选择性主存准入策略。实验结果表明，该方法能够在较小命中率损失下显著降低卸载时间，缓解简单全量卸载带来的 I/O 时延过高的问题。

第二，针对主存中高价值缓存难以及时回迁、回迁过程容易与当前计算相互干扰的问题，本文在第四章提出了面向 Prefill 阶段的主存–显存预取与执行优化策略。该方法通过比较主存热点根节点和显存冷叶节点构建可执行的预取计划，并结合 Decode 阶段流水线重叠和批量数据传输，在不显著增加显存峰值的前提下提升缓存回迁效率。综合第三章与第四章的 Prefill 阶段优化结果，本文方法相较启用前缀共享的 vLLM 最高可实现 2.25 倍的时延优化与吞吐提升，相较 SGLang 最高可实现 2 倍的时延下降与吞吐提升。

第三，针对 Decode 阶段输出长度长尾分布导致的多 GPU 负载失衡与 KV Cache 资源浪费问题，本文在第五章提出了面向推理 Decode 阶段的响应长度与 KV Cache 负载感知的多 GPU 请求调度方法。该方法通过层–头预测能力扫描筛

选与响应长度强相关的长度预测表征，并结合不同生成场景的词元位置与全局表征构造轻量长度预测器；在此基础上，将预测响应长度转化为未来 KV Cache 负载估计，用于多 GPU 服务中的请求路由与负载均衡。实验结果表明，所提出的长度预测方法在 Llama3.2-1B 和 Qwen2.5-3B 的 Long Sequence、Reasoning 与 RL 场景中均优于主要对比方法；在生成长度高倾斜的多 GPU 路由负载上，本文方法相较 SGLang 默认路由可降低 33.3% 的整体完成时延、降低 61.9% 的 P99 时延，并提升 53.8% 的输出吞吐。

总体而言，本文以 KV Cache 为主线，将大模型推理系统的资源管理问题划分为 Prefill 阶段的保留、卸载、准入、预取和 Decode 阶段的长度预测、负载估计、请求路由两大类任务，并针对其差异化瓶颈分别设计优化策略。实验结果表明，围绕不同执行阶段开展有针对性的系统优化，能够在不改变模型本身的前提下显著提升在线推理性能：本文在 Prefill 阶段最高实现 2.25 倍的时延与吞吐改进，在 Decode 阶段通过更准确的响应长度预测支撑多 GPU 负载感知路由，并在高倾斜负载下实现整体完成时延、P99 时延和输出吞吐的同步改善。

从研究路线看，本文工作的一个核心特点，是没有把 KV Cache 仅仅看作静态的内存占用对象，而是把它视为贯穿 Prefill 与 Decode 双阶段的动态系统资源。在 Prefill 阶段，KV Cache 是否被正确保留、卸载和回迁，决定了系统能否充分发挥前缀共享带来的重复计算消除收益；在 Decode 阶段，KV Cache 又体现为请求持续占用显存预算的生命周期资源，其增长速度和设备分布直接影响多 GPU 负载均衡与系统吞吐。也正因为此，本文从两个阶段分别切入、再在系统层统一收束，形成了较完整的问题分析与方法设计闭环。

从方法层面看，本文的贡献可以进一步概括为三点。第一，提出了基于热度追踪的前缀缓存价值评估框架，使系统能够在较低在线开销下区分高价值共享前缀与低价值长尾前缀，为后续驱逐、准入和预取提供统一依据。第二，提出了面向异构内存协同的主动缓存管理机制，把传统缓存空间不足时再被动加载的思路扩展为先筛选、再保留、后提升的多层级协同流程。第三，提出了与运行时调度直接耦合的响应长度预测和 KV Cache 负载感知路由方案，使长度预测结果不再停留于离线评估，而是进一步成为多 GPU 请求分配和未来缓存负载均衡的核心信号。

从工程意义看，本文也说明了一点：大模型推理系统的性能瓶颈并不总是来

自模型计算本身，很多时候更来自围绕 KV Cache 的状态管理、存储层级切换和请求调度方式。随着上下文长度持续增加、前缀共享更加普遍、在线服务负载更加异构，这些非模型参数层面的系统问题会越来越成为影响真实部署效率的关键因素。因此，围绕 KV Cache 展开的系统级研究，不仅具有局部优化意义，也为今后理解和设计大模型推理运行时提供了更清晰的分析视角。

7.2 本文工作的局限性

尽管本文在 Prefill 与 Decode 两阶段分别提出了较完整的优化方法，但从研究边界上看，仍存在若干局限。首先，第三章和第四章的方法主要建立在请求之间确实存在较强前缀共享这一前提之上。对于公共前缀较短、用户输入高度离散或请求几乎无共享结构的负载，热度追踪、选择性准入与主动预取的收益都会自然减弱。也就是说，本文方法更适合模板化提示、检索增强、长系统提示和思维链场景，而未必能在所有在线负载上同样显著地提升性能。

其次，第五章的响应长度预测与 KV Cache 负载感知路由虽然在生成长度高倾斜负载上表现出明显优势，但它仍依赖对请求未来生成长度的较稳定估计。在多轮交互、工具调用、复杂采样和智能体推理等场景中，请求长度常常受到外部环境反馈影响而剧烈波动，此时仅依据生成前可获得的提示词表征可能仍不够充分。此外，本文当前 SGLang 原型采用前端旁路预测方式，虽然已将预测耗时计入端到端结果，但仍会带来额外前向开销；未来若能与 Prefill–Decode 解耦架构或后端中间表征导出机制结合，可进一步减少这部分开销。因此，本文目前更多验证了长度预测、负载估计与多 GPU 路由耦合这一方向的有效性，而尚未完全解决极端动态生成场景下的长度建模问题。

再次，从系统实现角度看，本文原型主要面向单节点异构内存协同和典型多卡扩展场景，尚未深入覆盖更大规模分布式推理系统中跨节点 KV Cache 一致性、远端回迁时延和全局队列协调等问题。随着模型规模和部署规模进一步扩大，这些因素都可能成为新的关键瓶颈。因此，本文工作可以看作是面向单机和小规模多卡场景的一套完整方法框架，但距离更大规模生产系统仍有进一步工程化和理论化空间。

7.3 未来工作展望

尽管本文已经在 Prefill 与 Decode 两阶段给出了较完整的 KV Cache 优化方案，但仍有若干值得进一步深入的方向。

第一，可面向更大规模与更复杂场景扩展本文方法。当前工作主要关注单节点异构内存协同和典型文本生成负载，而在多 GPU、分布式推理、多模态与长上下文场景下，KV Cache 的放置、迁移、副本一致性以及不同模态之间的共享模式都会更加复杂。未来可从全局资源视角进一步研究跨设备 KV Cache 管理机制，并结合多模态输入特性与超长上下文需求，对本文方法进行适配与扩展。

第二，可进一步研究统一且自适应的全生命周期 KV Cache 优化框架。本文分别从 Prefill 阶段的异构内存管理和 Decode 阶段的长度预测与负载感知路由切入问题，但在真实系统中，前缀缓存保留、主存回迁、多 GPU 请求分配和服务质量保障实际上是相互耦合的。未来可以进一步把热度、长度、服务等级目标与在线监控信号纳入同一优化过程，构建统一的跨阶段决策机制，并推动系统从离线静态配置优化进一步走向在线自适应优化。

第三，可加强对长度预测与系统收益边界的理论建模。随着大模型逐渐用于长链推理、多轮工具调用、复杂采样和智能体场景，请求输出长度与缓存访问行为都会表现出更强的不确定性与长尾性。未来可进一步提升长度预测在复杂场景中的鲁棒性，引入更准确的不确定度估计与任务结构先验；同时建立更系统的分析模型，研究不同负载分布、缓存预算和链路条件下的性能收益边界，为热度驱逐、主动预取和 KV Cache 负载感知路由提供更坚实的理论依据。

综上，围绕大模型推理 KV Cache 的系统级优化仍有广阔研究空间。随着模型规模、上下文长度和应用复杂度的持续增长，面向真实服务场景的阶段感知、数据感知和资源感知优化方法将具有越来越重要的理论意义与工程价值。

参考文献

[1] 科技日报. IDC 报告显示：阿里云位列 2024 中国 AI 基础设施市场份额第一[EB/OL]. 2025. https://www.stdaily.com/web/gdxw/2025-07/01/content_363487.html.

[2] KWON W, LI Z, ZHUANG S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention[C]//Proceedings of the ACM Symposium on Operating Systems Principles (SOSP). 2023: 611-626.

[3] DAO T, FU D Y, ERMON S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness[C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2022: 16344-16359.

[4] DAO T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning[C]//arXiv preprint arXiv:2307.08691. 2023.

[5] AMINABADI R Y, RAJBHANDARI S, ZHANG M, et al. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2022: 46:1-46:15.

[6] YU G I, JEONG J S, KIM G W, et al. Orca: A Distributed Serving System for Transformer-Based Generative Models[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2022: 521-538.

[7] WU B, ZHONG Y, ZHANG Z, et al. Fast Distributed Inference Serving for Large Language Models[C]//arXiv preprint arXiv:2305.05920. 2023.

[8] PRABHU R, CHEN Y, GAWRILOW E, et al. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention[C]//arXiv preprint

arXiv:2405.04437. 2024.

[9] AGRAWAL A, PANWAR A, MOHAN J, et al. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills[C]//arXiv preprint arXiv:2308.16369. 2023.

[10] ZHONG Y, LIU S, CHEN J, et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2024: 193-210.

[11] PATEL J, CHOUKSE E, ZHANG C, et al. Splitwise: Efficient Generative LLM Inference Using Phase Splitting[C]//Proceedings of the Annual International Symposium on Computer Architecture (ISCA). 2024: 118-132.

[12] SUN B, HUANG Z, ZHAO H, et al. Llumnix: Dynamic Scheduling for Large Language Model Serving[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2024: 173-191.

[13] FU Y, MENON P, HU R, et al. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models[C]//arXiv preprint arXiv:2401.14351. 2024.

[14] ZHU Y, ZHONG Y, LIU S, et al. NanoFlow: Towards Optimal Large Language Model Serving Throughput[C]//arXiv preprint arXiv:2408.12757. 2024.

[15] MIAO X, XU G, SHI Y, et al. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification[C]//arXiv preprint arXiv:2305.09781. 2024.

[16] TOUVRON H, MARTIN L, STONE K, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models[C]//arXiv preprint arXiv:2307.09288. 2023.

[17] Meta AI. The Llama 3 Herd of Models[C]//arXiv preprint arXiv:2407.21783. 2024.

[18] TEAM G, MESNARD T, HARDIN C, et al. Gemma: Open Models Based on Gemini Research and Technology[C]//arXiv preprint arXiv:2403.08295. 2024.

[19] Qwen Team. Qwen2.5 Technical Report[C]//arXiv preprint arXiv:2412.15115. 2024.

[20] HENDRYCKS D, BURNS C, BASART S, et al. Measuring Massive Multitask Language Understanding[C]//Proceedings of the International Conference on Learning Representations (ICLR). 2021.

[21] COBBE K, KOSARAJU V, BAVARIAN M, et al. Training Verifiers to Solve Math Word Problems[C]//arXiv preprint arXiv:2110.14168. 2021.

[22] SUZGUN M, SCALES N, SCHÄRLI N, et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them[C]//arXiv preprint arXiv:2210.09261. 2022.

[23] BAI Y, LV X, ZHANG J, et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding[C]//arXiv preprint arXiv:2308.14508. 2023.

[24] CUI G, YUAN L, DING N, et al. UltraFeedback: Boosting Language Models with Scaled AI Feedback[C]//Proceedings of the International Conference on Machine Learning (ICML). 2024: 9722-9744.

[25] GIM I, CHEN G, LEE S S, et al. Prompt Cache: Modular Attention Reuse for Low-Latency Inference[C]//arXiv preprint arXiv:2311.04934. 2024.

[26] YE L, TAO Z, HUANG Y, et al. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition[C]//arXiv preprint arXiv:2402.15220. 2024.

[27] ZHENG L, YIN L, XIE Z, et al. SGLang: Efficient Execution of Structured Language Model Programs[C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2024: 62557-62583.

[28] LIN Z, LI Y, YAO Z, et al. Parrot: Efficient Serving of LLM-Based Applications with Semantic Variable[C]//arXiv preprint arXiv:2405.19888. 2024.

[29] HU Y, WANG Y, ZHANG H, et al. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool[C]//arXiv preprint arXiv:2406.17565. 2024.

[30] QIN E, CHEN J, ZHANG H, et al. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving[C]//arXiv preprint arXiv:2407.00079. 2024.

[31] LIU Y, LIU H, LIU X, et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving[C]//arXiv preprint arXiv:2310.07240. 2024.

[32] SHENG Y, ZHENG L, ZHONG Y, et al. Fairness in Serving Large Language Models[C]//arXiv preprint arXiv:2401.00588. 2024.

[33] SHEN L, WANG Z, XIAO W, et al. FastSwitch: Optimizing Context Switching Efficiency in Fairness-Aware Large Language Model Serving[C]//arXiv preprint arXiv:2410.11762. 2024.

[34] SCHRAGE L. A Proof of the Optimality of the Shortest Remaining Processing Time Discipline[J]. Operations Research (OR), 1968, 16(3): 687-690.

[35] BANSAL N, HARCHOL-BALTER M. Analysis of SRPT Scheduling: Investigating Unfairness[C]//Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS). 2001: 279-290.

[36] BENDER M A, CHAKRABARTI S, MUTHUKRISHNAN S. Flow and Stretch Metrics for Scheduling Continuous Job Streams[C]//Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA). 1998: 270-279.

[37] HARCHOL-BALTER M, CROVELLA M, MURTA C D. On Choosing a Task Assignment Policy for a Distributed Server System[R]. Technical Report; discusses size-based policies including SRPT-style scheduling for web workloads. Boston University, 1998.

[38] XIE H, CHEN Y, WANG L, et al. Predicting LLM Output Length via Entropy-Guided Representations[C]//Proceedings of the International Conference on Learning Representations (ICLR). 2026.

[39] CHEN L, YE Z, WU Y, et al. Punica: Multi-Tenant LoRA Serving[C]// Proceedings of Machine Learning and Systems (MLSys): vol. 6. 2024: 1-13.

[40] AGRAWAL A, KEDIA N, PANWAR A, et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2024: 117-134.

[41] YAO Y, CAO Y, XIAO W, et al. Flover: An Efficient Generative LLM Inference with Temporal Fusion Framework[C]//arXiv preprint arXiv:2402.03271. 2023.

[42] SHENG Y, ZHENG L, YUAN B, et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU[C]//Proceedings of the International Conference on Machine Learning (ICML). 2023: 31094-31116.

[43] LEE W, LEE J, SEO J, et al. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management[C]//arXiv preprint arXiv:2406.19707. 2024.

[44] ZHANG Z, CHOWDHERY A, WANG T, et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models[C]//arXiv preprint arXiv:2306.14048. 2023.

[45] LIU Z, DESAI A, ABBEEL P, et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time[C]//arXiv preprint arXiv:2305.17118. 2023.

[46] XIAO G, TIAN Y, CHEN B, et al. StreamingLLM: Efficient Streaming Language Models with Attention Sinks[C]//arXiv preprint arXiv:2309.17453. 2023.

[47] LI Y, HUANG Y, YANG B, et al. SnapKV: LLM Knows What You Are Looking for Before Generation[C]//arXiv preprint arXiv:2404.14469. 2024.

[48] CAI Z, TU Y, ZHENG L, et al. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling[C]//arXiv preprint arXiv:2405.12532. 2024.

[49] LIU Y, JIA H, CHEN J, et al. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models[C]//arXiv preprint arXiv:2405.14366. 2024.

[50] LIU Z, DESAI A, ABBEEL P, et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache[C]//arXiv preprint arXiv:2402.02750. 2024.

[51] FENG H, ZHANG Y, LIU B, et al. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference[C]//arXiv preprint arXiv:2407.11550. 2024.

[52] TANG H, CHEN J, ZHANG H, et al. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference[C]//arXiv preprint arXiv:2406.10774. 2024.

[53] XIAO G, TIAN Y, LEWIS M, et al. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads[C]//arXiv preprint arXiv:2410.10819. 2024.

[54] TANG H, ZHANG H, STOICA I, et al. RazorAttention: Efficient KV Cache Compression Through Retrieval Heads[C]//arXiv preprint arXiv:2410.05094. 2024.

[55] SUN Y, WANG T, TU Y, et al. ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference[C]//arXiv preprint arXiv:2410.21465. 2024.

[56] KIKUCHI Y, NEUBIG G, SASANO R, et al. Controlling Output Length in Neural Encoder-Decoders[C]//Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). 2016: 1328-1338.

[57] LIU Y, LUO Z, ZHU K. Controlling Length in Abstractive Summarization Using a Convolutional Neural Network[C]//Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). 2018: 4110-4119.

[58] TAKASE S, OKAZAKI N. Positional Encoding to Control Output Sequence Length[C]//Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long and Short Papers. 2019: 3999-4004.

[59] YANG Z, GAO Y, WANG W, et al. Predicting and Using Target Length in Neural Machine Translation[C]//Proceedings of the Conference of the Asia-Pacific

Chapter of the Association for Computational Linguistics and International Joint Conference on Natural Language Processing (AACL-IJCNLP). 2020: 389-395.

[60] WANG M, JIAXIN G, WANG Y, et al. How Length Prediction Influence the Performance of Non-Autoregressive Translation?[C]//Proceedings of the BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP (BlackboxNLP). 2021: 205-213.

[61] KWON J, KAMIGAITO H, OKUMURA M. Abstractive Document Summarization with Summary-length Prediction[C]//Findings of the Association for Computational Linguistics: EACL (Findings of ACL). 2023: 618-624.

[62] LIU Y, JIA Q, ZHU K. Length Control in Abstractive Summarization by Pretraining Information Selection[C]//Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers. 2022: 6885-6895.

[63] PIOTROWSKI G, BYSTROŃSKI M, HOŁYSZ M, et al. When Will the Tokens End? Graph-Based Forecasting for LLMs Output Length[C]//Proceedings of the Annual Meeting of the Association for Computational Linguistics and International Joint Conference on Natural Language Processing: Student Research Workshop (ACL-IJCNLP SRW). 2025: 896-907.

[64] MOON S, CHOI D, KWON J, et al. Length Representations in Large Language Models[C]//Findings of the Association for Computational Linguistics: EMNLP (Findings of ACL). 2025: 19775-19793.

[65] VASWANI A, SHAZEER N, PARMAR N, et al. Attention Is All You Need [C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2017: 5998-6008.

[66] BROWN T B, MANN B, RYDER N, et al. Language Models are Few-Shot Learners[C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2020: 1877-1901.

[67] SU J, LU Y, PAN S, et al. RoFormer: Enhanced Transformer with Rotary Position

Embedding[C]//arXiv preprint arXiv:2104.09864. 2021.

[68] SHAZEER N. Fast Transformer Decoding: One Write-Head is All You Need[C]//arXiv preprint arXiv:1911.02150. 2019.

[69] AINSLIE J, LEE-THORP J, de JONG M, et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints[C]//arXiv preprint arXiv:2305.13245. 2023.

[70] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2020: 9459-9474.

[71] RAM O, LEVINE Y, DALMEDIGOS I, et al. In-Context Retrieval-Augmented Language Models[J]. Transactions of the Association for Computational Linguistics (TACL), 2023, 11: 1316-1331.

[72] LEVIATHAN Y, KALMAN M, MATIAS Y. Fast Inference from Transformers via Speculative Decoding[C]//Proceedings of the International Conference on Machine Learning (ICML). 2023: 19274-19286.

[73] CHEN C, BORGEAUD S, IRVING G, et al. Accelerating Large Language Model Decoding with Speculative Sampling[C]//arXiv preprint arXiv:2302.01318. 2023.

[74] LI Y, GU R, HUAN C, et al. HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems[J]. Proceedings of the ACM on Management of Data (SIGMOD), 2025, 3(4): 1-27.

[75] STRATI F, TANG H, CHEN J, et al. DéjàVu: KV-Cache Streaming for Fast, Fault-Tolerant Generative LLM Serving[C]//arXiv preprint arXiv:2403.01876. 2024.

[76] DUAN J, LU R, DUANMU H, et al. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving[C]//arXiv preprint arXiv:2404.02015. 2024.

[77] FAN B, ANDERSEN D G, KAMINSKY M, et al. Cuckoo Filter: Practically Better Than Bloom[C]//Proceedings of the ACM International Conference on

Emerging Networking Experiments and Technologies (CoNEXT). 2014: 75-88.

[78] APPLEBY A. MurmurHash3[Z]. Non-cryptographic hash function. 2011.

[79] ZELLERS R, HOLTZMAN A, BISK Y, et al. HellaSwag: Can a Machine Really Finish Your Sentence?[C]//Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL). 2019: 4791-4800.

[80] HUANG Y, BAI Y, ZHU Z, et al. C-Eval: A Multi-Level Multi-Discipline Chinese Evaluation Suite for Foundation Models[C]//Proceedings of the Advances in Neural Information Processing Systems (NeurIPS). 2023: 62991-63010.

致　　谢

这份致谢，我迟迟没有下笔，因为实在有些恍惚，觉得这三年的研究生生活怎么眨眼间就要结束，节奏过于紧凑了，好像经历了许多事，但又好像什么都没有发生过。

研一带着懵懂与青涩，逐渐步入了科研的旅程，水一水课，读一读文章，一年就过去了；研二上就开始焦虑能不能找到实习，同时又换了科研新方向，在赶论文与愁实习中度过；研二下开始全心准备暑期实习的笔试面试，中间穿插论文的修改。研二暑假开始实习，9 月份回到学校，又找到新的实习，经历秋招的煎熬与选择，一直到快过年才回学校；过完年回来就要写毕业论文，准备答辩。南大留给我的记忆，似乎更多的是通宵赶论文、在工位准备笔试面试的画面。似乎整段旅程都在被推着走，都在焦虑与紧张中度过，没交到太多知心朋友，没参加什么有趣的文艺活动，也没留下什么和学校有关的美好回忆。

在我的意识里，南大和我更像是彼此的过客，她是我生命中短暂的一站，我虽然和她没产生太多值得我回忆的交集，但是在她的陪伴下，在我这的这段旅途中，我收获了无限的成长。

这三年，我接触了更多不同类型的运动，网球、羽毛球、龙舟、徒步、壁球；去到了更多的地方，贵阳、横店、长春、香港、上海、苏州、杭州、台州、衢州、芜湖、蚌埠、黄山、济南、北京、天津、新疆、甘肃等等，我的探索欲和好奇心在不断生长；结识了各行各业的不同朋友，窥见了完全不同的生活态度和生活方式；经历了感情，变得更加成熟与自洽，更加明白自己真正想要的是什么；拿到了满意的工作 offer，即将在非常好的工作氛围中从事我非常感兴趣的工作；越来越能处理好家庭关系，变得更加勇敢和果断；我的内核越来越稳定，精神追求越来越清晰，在逐渐变成我越来越喜欢的模样。

相比三年前，我依旧保持着对生活的热爱，对新鲜事物的好奇，多了更多去尝试和体检的机会与底气，沉淀下了更加自信和沉稳的自己。

感谢我的导师顾荣老师，感谢您在学术上的专业指导与在生活上无微不至

的关照，跟您读研的三年，我觉得幸运且圆满，祝愿您和我们课题组都越来越好；感谢宦成颖老师、王智彬老师在论文写作上的指导和帮助；感谢我的师弟姚仁杰、王博、闵甜，同门刘非凡等，在论文实验部分给予的帮助，大家团结友爱，互相帮助，希望我们一直保持联络，相互扶持；感谢我在工作中遇到的领导和同事们，帮助我快速的成长。

感谢永远无条件爱我，永远支持和关心我的妈妈，在爱和关心呵护下成长的孩子，永远不会差；感谢勤劳正直的爸爸，用正确和淳朴的价值观一路引导着我；感谢善良可爱的妹妹，哥哥希望你快乐地成长，多体验丰富多彩的生活，做一切你想做的事情；感谢我的好朋友们，陪伴和支持，理解与体贴，是你们给我最温暖的礼物。

最后，我想感谢，那个积极向上，乐观开朗，待人真诚，努力拼搏的自己。此刻，处于人生的重要节点，抬头往前看，我无比期待新的工作，无比向往新的生活环境，无比憧憬新的体验，无比希冀去亲手书写属于我的崭新故事，Fighting!

附录 A　附录

本附录补充说明作者在研究生期间围绕本文研究方向所开展的代表性科研工作、项目参与情况与竞赛获奖情况。

A.1　研究生期间发表论文

[1] **Li Y**, Gu R*, Huan C, Wang Z, Yao R, Tian C, Chen G. HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems[J]. //Proceedings of the ACM on Management of Data（SIGMOD，CCF-A 类数据库旗舰会议）, Vol 3(4), 2025, Article No.250, pp. 1–27.（独立第一作者）

[2] **Li Y**, Gu R*, Li S, Wang B*, Yu W. ColdPurge: Effecient Metadata Cache Cleaning via Accurate Online Data Hotness Tracking[C]. //Proceedings of the 25th International Middleware Conference（Middleware，CCF-B 类系统软件会议，Full paper 投稿 184 篇，录用 46 篇，录用率 25%）, 2024, pp. 8–14.（独立第一作者）

A.2　研究生期间参与科研项目

[1] 国自然基金面上项目《基于智能化分析方法的大数据系统软件性能优化研究》（参与，项目负责人为指导老师）

[2] 南京大学-中国移动合作项目《智能算力的调度与配置技术》（参与，项目负责人为指导老师）

[3] 2024 年江苏省研究生科研创新计划项目《数据热度轻量化感知的云原生缓存优化研究》（负责人）

A.3 研究生期间获得的比赛奖项

[1] 2024 CCF 算力网系统与应用大赛一等奖，**李宇航**、刘非凡、王书麟（本人排名第一）

[2] 2025 年硕士研究生国家奖学金